POSTGRESQL AND SQL

THE COMPLETE PROFESSIONAL COURSE

From Absolute Beginner to Professional Database Practitioner

A Comprehensive Textbook Covering Relational Database Fundamentals, Modern PostgreSQL, Analytical SQL, Database Design, Performance Optimization, Administration, Python/Pandas Integration, and Applications in Data Analytics, Data Science, Data Engineering, Backend Development, and AI Engineering

Kajola Gbenga

Data Scientist & AI Engineer

Table of Contents

1. Introduction to Databases

2. Installing and Setting Up PostgreSQL

3. SQL Fundamentals

4. Creating Databases and Tables

5. Constraints

6. Inserting Data

7. Retrieving Data

8. Filtering Data

9. UPDATE and DELETE

10. SQL Functions

11. Aggregate Functions

12. Joins

13. Subqueries

14. Common Table Expressions (CTEs)

15. Window Functions

16. Set Operations

17. Views

18. Indexes

19. Query Optimization

20. Transactions
21. Database Design
22. Advanced PostgreSQL Data Types
23. Advanced SQL Expressions
24. Stored Procedures and Functions
25. Triggers
26. Temporary Tables
27. Security and Users
28. PostgreSQL Administration Concepts
29. Backup and Restore
30. Importing and Exporting Data
31. PostgreSQL + Python
32. PostgreSQL + Pandas
33. PostgreSQL for Data Analytics
34. PostgreSQL for Data Science
35. PostgreSQL for Data Engineering
36. PostgreSQL for AI Engineering
37. Advanced Analytics with PostgreSQL
38. PostgreSQL-Specific Features
39. PostgreSQL vs Other Databases
40. SQL Interview Preparation
41. SQL Coding Challenges
42. Complete Real-World Projects
43. Final Capstone Project
44. Teaching Structure
45. SQL Best Practices
46. Final Cheat Sheet

1. Introduction to Databases

1.1 What Is Data?

Data is any raw fact or value that describes something - a name, a price, a date, a quantity. On its own, a single piece of data ("Ada", "49.99", "2026-01-15") carries little meaning. Data becomes useful **information** once it is organized and given context - for example, knowing that "Ada" is the name of a customer who paid "49.99" on "2026-01-15" for a specific order.

1.2 What Is a Database?

A **database** is an organized collection of related data, stored so it can be efficiently entered, retrieved, updated, and managed. Instead of scattering facts across disconnected files, a database organizes them into structured, related collections - so that "customers," "orders," and "products" all live in one coherent, queryable system.

1.3 What Is a DBMS?

A **DBMS (Database Management System)** is the software that creates, manages, and controls access to a database. It handles storage, retrieval, security, concurrent access by many users at once, and data integrity - so that application developers never have to write low-level code to manage files on disk directly. Examples: PostgreSQL, MySQL, Oracle Database, SQL Server, SQLite, MongoDB.

1.4 What Is an RDBMS?

An **RDBMS (Relational Database Management System)** is a DBMS built specifically around the **relational model**: data is organized into **tables** (also called relations), each with a fixed set of named columns, and relationships between tables are expressed by referencing shared key values rather than nesting or duplicating data. PostgreSQL, MySQL, SQL Server, and Oracle are all RDBMSs. This entire course focuses on the relational model, using PostgreSQL as the RDBMS.

1.5 Database vs Spreadsheet

Aspect	Spreadsheet (e.g. Excel)	Database (RDBMS)
Structure	Loose grid of cells; structure is a convention, not enforced	Strict, enforced schema - every row in a table has the same defined columns and types
Data integrity	Easy to accidentally break (a formula referencing the wrong cell, inconsistent formats)	Enforced by constraints (Part 5) - invalid data can be rejected automatically
Multi-user access	Poor - concurrent editing causes conflicts, locked files	Designed for many simultaneous users, with proper concurrency control (Part 20)

Aspect	Spreadsheet (e.g. Excel)	Database (RDBMS)
Relationships between data	Manual (VLOOKUP, manual copy-paste)	Native - foreign keys formally link related tables (Part 5)
Scale	Struggles beyond tens of thousands of rows	Designed for millions to billions of rows
Querying	Formulas, filters, pivot tables	SQL - a full declarative query language

A spreadsheet is a reasonable tool for small, single-user, ad-hoc data. A database is the correct tool once data needs to be shared across many users, must stay consistent, or needs to scale.

1.6 DBMS vs RDBMS

Every RDBMS is a DBMS, but not every DBMS is relational. Non-relational ("NoSQL") DBMSs - like MongoDB (document-oriented) or Redis (key-value) - organize data differently, often trading strict structure and relational integrity for flexibility or raw speed on specific access patterns. This course is entirely about the relational model, as implemented by PostgreSQL.

1.7 SQL vs PostgreSQL

SQL (Structured Query Language) is a standardized **language** for defining, querying, and manipulating relational data. It is not tied to any one product. **PostgreSQL** is a specific, open-source **RDBMS product** that implements SQL (following the SQL standard closely, while also adding many of its own powerful extensions). The relationship is similar to "English" (the language) versus "a specific book written in English" (a specific implementation/use of it).

1.8 PostgreSQL vs MySQL vs SQL Server vs Oracle vs SQLite

Database	License	Typical Use	Notable Traits
PostgreSQL	Open source (permissive license)	General-purpose, analytics, complex applications	Extremely standards-compliant, rich data types (JSONB, arrays, ranges), extensible (pgvector, PostGIS), strong for both OLTP and analytics
MySQL	Open source (GPL, with a commercial edition)	Web applications, read-heavy workloads	Historically simpler, very widely deployed, weaker standards compliance on some features
Microsoft SQL Server	Commercial	Enterprise Windows/.NET environments	Deep Microsoft ecosystem integration, strong BI tooling

Database	License	Typical Use	Notable Traits
Oracle Database	Commercial (often expensive licensing)	Large enterprises, legacy enterprise systems	Extremely feature-rich, historically dominant in large enterprises
SQLite	Public domain	Embedded/local applications, mobile apps, small tools	Serverless - an entire database is a single file; not designed for many concurrent users

This course uses **PostgreSQL** throughout, and will clearly flag which features are standard SQL (portable to other databases) versus PostgreSQL-specific (Part 39 covers this comparison in full depth).

1.9 Why PostgreSQL Is Important

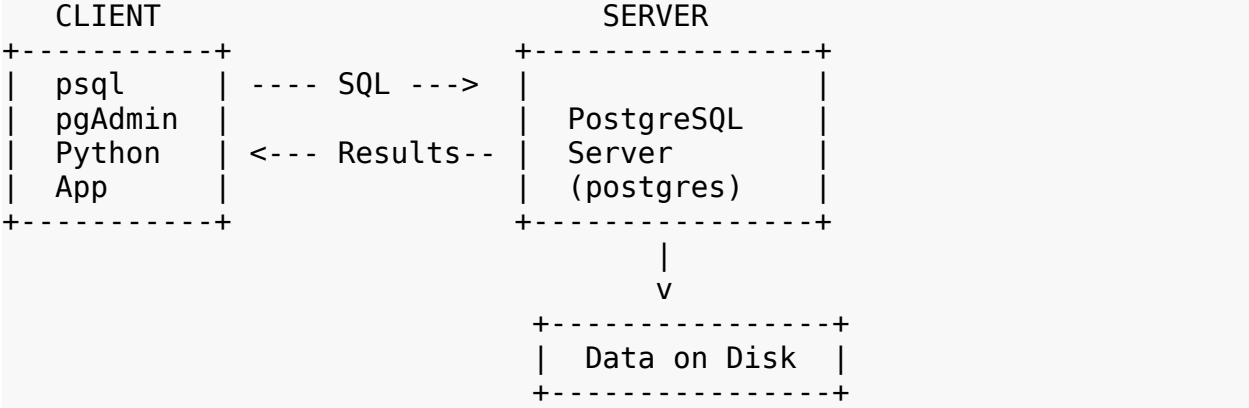
- **Standards-compliant**: PostgreSQL follows the SQL standard more closely than most competitors, so skills transfer well.
- **Free and open source**: no licensing cost, with a large, active global community.
- **Extensible**: supports custom data types, extensions (like `pgvector` for AI vector search, Part 36), and procedural languages.
- **Rich data types**: native JSON/JSONB, arrays, ranges, network types - far beyond basic SQL types.
- **Strong for both transactional and analytical workloads**: suitable for everyday application databases and heavy analytical queries.
- **Industry momentum**: increasingly the default choice for new applications, startups, and modern data/AI stacks.

1.10 PostgreSQL Use Cases

Use Case	Example
Backend application database	Storing users, orders, sessions for a web app
Data analytics	Running complex aggregations and reports over business data
Data warehousing	Consolidating data from multiple sources for analysis
Geospatial applications	Using the PostGIS extension for location data
AI/LLM applications	Using `pgvector` for embeddings and semantic search (Part 36)
Financial systems	Strong transactional guarantees (Part 20) for money-handling logic

1.11 PostgreSQL Architecture at a High Level

PostgreSQL follows a **client-server architecture**: one or more **client** programs (an application, `psql`, pgAdmin, a Python script) connect over a network (or locally) to a **PostgreSQL server process**, which manages the actual data on disk and handles every query.



The server listens on a network **port** (default `5432`), authenticates each connecting client, and executes SQL statements on the client's behalf, returning results back over the same connection.

1.12 Database, Schema, Tables, Rows, Columns - The Hierarchy

```
PostgreSQL Server
  └── Database ("company_db")
        └── Schema ("public", or a custom schema like "sales")
              └── Table ("customers")
                    ├── Column "customer_id"
                    ├── Column "name"
                    ├── Column "email"
                    └── Rows (the actual records)
```

Term	Meaning
Database	A named, isolated collection of schemas, tables, and other objects on a server; one PostgreSQL server can host many databases
Schema	A named namespace *within* a database, grouping related tables together (e.g. a `sales` schema and an `hr` schema inside one database)
Table	A structured collection of rows, all sharing the same defined set of columns
Row (also called a **record**)	One single entry in a table - e.g., one specific customer
Column (also called a **field**)	One named, typed attribute every row in a table has - e.g., `email`

```
Table: customers

 customer_id | name          | email                   | signup_date
-------------+---------------+-------------------------+-------------
 1           | Ada Lovelace  | ada@example.com         | 2024-03-01
 2           | Grace Hopper  | grace@example.com       | 2024-05-14
```

Here, `customers` is the table; `customer_id`, `name`, `email`, `signup_date` are its columns; each numbered line is a row (record); a single value like `"ada@example.com"` is one field.

1.13 Primary Keys, Foreign Keys, and Relationships

A **primary key** is a column (or combination of columns) that uniquely identifies each row in a table - no two rows can share the same primary key value, and it can never be `NULL`. A **foreign key** is a column in one table that references the primary key of another table, formally establishing a **relationship** between the two.

```
Table: customers                    Table: orders
 customer_id (PK) | name             order_id (PK) | customer_id (FK)
------------------+------           --------------+------------------
 1                | Ada               101           | 1
 2                | Grace              102           | 1
                                      103           | 2
```

Here, `orders.customer_id` is a **foreign key** referencing `customers.customer_id` - it establishes that orders 101 and 102 belong to Ada, and order 103 belongs to Grace. This is the fundamental mechanism relational databases use to connect related data without duplicating it - covered fully, along with join types built on this exact mechanism, in Parts 5 and 12.

Exercises - Section 1

Beginner 1. In your own words, explain the difference between "data" and "information," using an example from your own life. 2. List three reasons a growing business might need to move from spreadsheets to a real database.

Intermediate 3. Draw (in text) a simple two-table relationship of your own design (not customers/orders), labeling the primary key and foreign key.

Advanced 4. Research and summarize, in 2-3 sentences, one specific PostgreSQL feature (from Section 1.9) that would matter to a company choosing between PostgreSQL and MySQL for a new AI-powered application. # 2. Installing and Setting Up PostgreSQL

2.1 Key Concepts Before Installing

Term	Meaning
PostgreSQL Server	The background process (`postgres`) that stores data and executes SQL
PostgreSQL Client	Any program that connects to the server and sends SQL - `psql`, pgAdmin, or a Python script

Term	Meaning
`psql`	PostgreSQL's official command-line client
pgAdmin	PostgreSQL's official graphical (GUI) administration tool
Port	The network port the server listens on - PostgreSQL's default is `5432`
Username / Password	Credentials used to authenticate a connection
Default database	A database named `postgres`, created automatically, often used as a starting connection point
Host	The machine the server runs on - `localhost` for a local install

2.2 Installing on Windows

1. Download the installer from the official PostgreSQL website (postgresql.org), which bundles the server, `psql`, and pgAdmin together.
2. Run the installer; choose an installation directory and data directory.
3. Set a password for the default superuser account, `postgres`, when prompted.
4. Keep the default port `5432` unless you have a specific reason to change it.
5. Select the components to install (Server, pgAdmin, Command Line Tools, Stack Builder) - install all of them for this course.
6. Finish the installer; PostgreSQL will be registered as a Windows service, starting automatically.

2.3 Installing on macOS

Option A - Postgres.app (simplest for beginners): 1. Download Postgres.app from postgresapp.com. 2. Drag it into the Applications folder and open it. 3. Click "Initialize" to create a new server instance - it starts immediately.

Option B - Homebrew (common for developers):

```
brew install postgresql@16
brew services start postgresql@16
```

2.4 Installing on Linux (Debian/Ubuntu)

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

On Linux, PostgreSQL creates a default operating-system user named `postgres`; the common first step is switching to it to access the database initially:

```
sudo -i -u postgres
psql
```

2.5 Connecting to PostgreSQL - Command Line (`psql`)

```
psql -h localhost -p 5432 -U postgres -d postgres
```

Flag	Meaning
`-h`	Host (server address)
`-p`	Port
`-U`	Username
`-d`	Database name to connect to

After entering the correct password, the prompt changes to something like:

```
postgres=#
```

This confirms you are now connected and can run SQL directly.

2.6 Connecting to PostgreSQL - GUI (pgAdmin)

1. Open pgAdmin; it runs as a local web application in your browser.
2. Right-click **Servers** → **Register** → **Server...**
3. On the **General** tab, give the connection a name (e.g. "Local PostgreSQL").
4. On the **Connection** tab, enter: Host = `localhost`, Port = `5432`, Username = `postgres`, Password = (your password).
5. Click **Save** - pgAdmin will connect and display the server's databases in a tree view on the left.
6. Right-click a database → **Query Tool** to open a SQL editor and run queries visually, with results shown in a table below.

2.7 Essential `psql` Commands

All `psql` **meta-commands** (commands to `psql` itself, not SQL) begin with a backslash `\` and are **not** terminated with a semicolon.

```
-- Listing databases
\l

-- Connecting to (switching) a database
\c company_db

-- Listing tables in the current database
\dt

-- Describing a specific table's structure
\d customers

-- Listing all schemas
\dn

-- Listing users/roles
```

```
\du

-- Getting help on SQL commands
\h CREATE TABLE

-- Getting help on psql meta-commands
\?

-- Quitting psql
\q
```

Command	Purpose
`\l`	List all databases on the server
`\c dbname`	Connect to (switch to) a different database
`\dt`	List tables in the current schema
`\d tablename`	Show a table's columns, types, and constraints
`\dn`	List schemas
`\du`	List roles/users
`\q`	Exit `psql`

2.8 Creating and Dropping Databases

These are actual **SQL** statements (terminated with `;`), not `psql` meta-commands, and can be run from `psql`, pgAdmin's Query Tool, or any client.

```
CREATE DATABASE company_db;

CREATE DATABASE

DROP DATABASE company_db;

DROP DATABASE
```

Warning: `DROP DATABASE` is irreversible and deletes everything in that database permanently - there is no "undo." Always double-check the database name, and confirm a recent backup exists (Part 29) before dropping any database outside of learning/practice.

2.9 A Full Example Session

```
$ psql -U postgres
Password for user postgres: ********
psql (16.2)
Type "help" for help.

postgres=# CREATE DATABASE company_db;
CREATE DATABASE
```

```
postgres=# \c company_db
You are now connected to database "company_db" as user "postgres".

company_db=# \dt
Did not find any relations.

company_db=# CREATE TABLE customers (id INTEGER, name TEXT);
CREATE TABLE

company_db=# \dt
          List of relations
 Schema |   Name    | Type  |  Owner
--------+-----------+-------+----------
 public | customers | table | postgres

company_db=# \d customers
                Table "public.customers"
 Column |  Type   | Collation | Nullable | Default
--------+---------+-----------+----------+---------
 id     | integer |           |          |
 name   | text    |           |          |

company_db=# \q
```

Exercises - Section 2

Beginner 1. Install PostgreSQL on your own machine (or use a cloud/sandbox instance) and connect successfully using `psql`. 2. Run `\l` and identify which databases already exist by default.

Intermediate 3. Create a database named `practice_db`, connect to it, create one simple table, and confirm it appears via `\dt`.

Advanced 4. Connect using pgAdmin instead of `psql`, and reproduce the same three steps (create database, connect, create table) using only the GUI, noting any differences in workflow. # 3. SQL Fundamentals

3.1 What SQL Is

SQL (Structured Query Language) is a **declarative** language: instead of describing *how* to retrieve data step by step (as in a procedural programming language), you describe *what* data you want, and the database's query planner (Part 19) figures out the most efficient way to get it.

```sql
-- "Give me the name and email of every customer from Nigeria" - the
WHAT, not the HOW
SELECT name, email FROM customers WHERE country = 'Nigeria';
```

3.2 SQL Syntax, Statements, and Keywords

A SQL **statement** is a complete instruction, always ending in a semicolon `;`. **Keywords** (`SELECT`, `FROM`, `WHERE`, `CREATE`, `INSERT`) are reserved words with special meaning in SQL and are conventionally written in UPPERCASE, though SQL keywords are case-insensitive.

```
SELECT name FROM customers WHERE country = 'Nigeria';
```

3.3 Identifiers

Identifiers are the names of database objects: table names, column names, schema names. By default, PostgreSQL folds unquoted identifiers to lowercase. Identifiers containing spaces or requiring exact-case preservation must be wrapped in double quotes.

```
SELECT customer_id FROM customers;      -- ordinary, unquoted
identifier
SELECT "Customer_ID" FROM customers;     -- quoted - case-sensitive,
must match exactly
```

Best practice: always use lowercase, underscore-separated identifiers (`order_date`, not `OrderDate` or `"Order Date"`) - this avoids the need for quoting entirely and is the standard PostgreSQL convention.

3.4 Literals

A **literal** is a fixed value written directly into SQL. Strings use single quotes; numbers are written plainly.

```
SELECT 'Ada Lovelace';    -- a string literal
SELECT 42;                 -- a numeric literal
SELECT 19.99;               -- a decimal literal
SELECT TRUE;                  -- a boolean literal
SELECT DATE '2026-01-15';      -- a date literal
```

Important: string literals always use single quotes `'...'` in SQL, never double quotes - double quotes are reserved for identifiers (Section 3.3). Mixing these up is one of the most common beginner mistakes.

3.5 Comments

```
-- This is a single-line comment

/* This is a
   multi-line comment */

SELECT 1; -- inline comment after a statement
```

3.6 Semicolons

Every SQL statement should end with a semicolon `;`. In `psql`, the semicolon is what tells the client "this statement is complete, execute it now" - without it, `psql` simply waits for more input on the next line.

3.7 Case Sensitivity

- **Keywords** (`SELECT`, `WHERE`) are case-insensitive, but UPPERCASE is the near-universal convention for readability.
- **Unquoted identifiers** are folded to lowercase automatically, making `Customers`, `CUSTOMERS`, and `customers` all refer to the same table.
- **String literal values** (data itself) **are** case-sensitive: `'Ada'` and `'ada'` are different string values when compared.

```
SELECT * FROM CUSTOMERS;    -- works identically to:
SELECT * FROM customers;

SELECT * FROM customers WHERE name = 'Ada';    -- will NOT match a row
where name = 'ada'
```

3.8 SQL Formatting Conventions

```
-- Good formatting - readable, one clause per line, consistent
indentation
SELECT
    customer_id,
    name,
    email
FROM
    customers
WHERE
    country = 'Nigeria'
ORDER BY
    name;
```

Consistent formatting (keywords capitalized, each major clause on its own line, consistent indentation) is a professional best practice - it makes queries far easier to read, debug, and review, especially as they grow more complex through this course.

3.9 The Core Query Clauses - First Introduction

```
SELECT column1, column2       -- WHICH columns to return
FROM table_name                 -- WHICH table to read from
WHERE condition                  -- WHICH rows to include
ORDER BY column1                  -- HOW to sort the results
LIMIT 10                            -- HOW MANY rows to return
OFFSET 5;                             -- SKIP this many rows first
```

Each of these clauses gets full, dedicated treatment in Parts 7-8; this section only introduces the overall shape.

3.10 The Business Database Used Throughout This Course

To keep every example realistic and consistent, this entire course uses one running business database, built out fully in Part 4 and used in every subsequent part:

Table	Purpose
`customers`	People who place orders
`products`	Items available for sale
`orders`	An order placed by a customer
`order_items`	Individual line items within an order (which products, how many)
`employees`	Staff members
`departments`	Organizational departments employees belong to
`payments`	Payments made against orders
`stores`	Physical or regional store locations
`suppliers`	Companies supplying products

```
customers ----< orders ----< order_items >---- products
                  |                              |
                  v                              v
               payments                      suppliers

departments ----< employees
```

(An arrow `----<` reads as "one-to-many": one customer can have many orders; one order can have many order_items; one product can appear in many order_items; one supplier can supply many products; one department can have many employees.)

3.11 A First Real Query (Preview)

```sql
SELECT name, email
FROM customers
WHERE country = 'Nigeria'
ORDER BY name
LIMIT 5;
```

```
      name       |        email
-----------------+----------------------
 Ada Lovelace    | ada@example.com
 Chidi Okafor    | chidi@example.com
 Ngozi Adichie   | ngozi@example.com
(3 rows)
```

Clause	Explanation
`SELECT name, email`	Return only these two columns, not every column in the table
`FROM customers`	Read from the `customers` table

Clause	Explanation
`WHERE country = 'Nigeria'`	Only include rows where the `country` column equals the string `'Nigeria'`
`ORDER BY name`	Sort the matching rows alphabetically by `name`
`LIMIT 5`	Return at most 5 rows (here, only 3 matched, so all 3 are returned)

Exercises - Section 3

Beginner 1. Identify the keywords, identifiers, and literals in: `SELECT price FROM products WHERE category = 'Electronics';` 2. Explain why `SELECT * FROM Customers;` and `SELECT * FROM customers;` behave identically, but `WHERE name = 'ADA'` and `WHERE name = 'Ada'` do not.

Intermediate 3. Rewrite a poorly formatted query of your own choosing (all on one line, inconsistent casing) using the formatting conventions from Section 3.8.

Advanced 4. Sketch (in text, like Section 3.10's diagram) what additional table you might add to this schema to track product reviews, and explain which existing table(s) it would need to reference. # 4. Creating Databases and Tables

4.1 `CREATE DATABASE`

```
CREATE DATABASE company_db;
```

Creates a new, empty database named `company_db` on the server. Every subsequent example in this course assumes you are connected to this database (`\c company_db` in `psql`).

4.2 `CREATE SCHEMA`

A **schema** groups related tables within one database - useful for organizing a large database into logical sections (e.g., separating `sales` tables from `hr` tables) without needing separate databases.

```
CREATE SCHEMA sales;
CREATE TABLE sales.regional_targets (region TEXT, target NUMERIC);
```

Every database has a default schema named `public`; unless specified otherwise, tables are created there. This course primarily uses the `public` schema for simplicity, introducing multi-schema design in the capstone project (Part 43).

4.3 PostgreSQL Data Types - Comprehensive Reference

Numeric Types

Type	Storage	Range / Precision	When to Use
`SMALLINT`	2 bytes	-32,768 to 32,767	Small whole numbers (e.g., a rating 1-5)

Type	Storage	Range / Precision	When to Use
`INTEGER`	4 bytes	~-2.1 billion to 2.1 billion	The default choice for whole numbers (IDs, counts)
`BIGINT`	8 bytes	~-9.2 quintillion to 9.2 quintillion	Very large whole numbers (e.g., high-volume auto-incrementing IDs)
`NUMERIC(p,s)` / `DECIMAL(p,s)`	Variable	Exact, user-defined precision `p` and scale `s`	**Money and anything requiring exact decimal precision** - never use floating-point for currency
`REAL`	4 bytes	~6 decimal digits precision	Approximate scientific/measurement data where exact precision doesn't matter
`DOUBLE PRECISION`	8 bytes	~15 decimal digits precision	Higher-precision approximate scientific data

Common mistake: using `REAL`/`DOUBLE PRECISION` for money.

```
-- WRONG - floating point can introduce rounding errors in financial calculations
CREATE TABLE bad_prices (price REAL);

-- RIGHT - NUMERIC stores exact decimal values, essential for money
CREATE TABLE good_prices (price NUMERIC(10, 2));   -- up to 10 digits total, 2 after the decimal point
```

Character Types

Type	Description	When to Use
`CHAR(n)`	Fixed-length, blank-padded to exactly `n` characters	Rarely needed in PostgreSQL - fixed-format codes only (e.g., a 2-letter country code)
`VARCHAR(n)`	Variable-length, up to `n` characters, enforced	When a firm maximum length is a genuine business rule (e.g., a username limited to 50 characters)
`TEXT`	Variable-length, unlimited	**The default, idiomatic PostgreSQL choice** for most text - no meaningful

Type	Description	When to Use
		performance difference from `VARCHAR` in PostgreSQL

PostgreSQL-specific note: unlike some databases, PostgreSQL's `TEXT` and unconstrained `VARCHAR` have identical performance - many PostgreSQL professionals use `TEXT` by default and only add a `VARCHAR(n)` or a `CHECK` constraint (Part 5) when a length limit is a genuine, enforced business rule.

Boolean Type

Type	Description
`BOOLEAN`	`TRUE`, `FALSE`, or `NULL` - PostgreSQL also accepts `'t'/'f'`, `1/0`, `'yes'/'no'` as input, always displaying as `t/f`

Date and Time Types

Type	Description	Example
`DATE`	A calendar date, no time	`'2026-01-15'`
`TIME`	A time of day, no date	`'14:30:00'`
`TIMESTAMP`	Date and time, **no timezone awareness**	`'2026-01-15 14:30:00'`
`TIMESTAMPTZ`	Date and time, **timezone-aware** (stored internally in UTC, displayed in the session's timezone)	`'2026-01-15 14:30:00+01'`
`INTERVAL`	A duration/span of time	`'3 days'`, `'2 hours 30 minutes'`

Best practice: use `TIMESTAMPTZ`, not plain `TIMESTAMP`, for almost all real applications - it correctly handles users and servers in different time zones, avoiding a very common and hard-to-diagnose class of bugs.

Other PostgreSQL-Specific Types (Full Depth in Part 22)

Type	Purpose
`UUID`	Universally unique 128-bit identifiers, often used as primary keys in distributed systems
`JSON`	Stores JSON text, validated but not optimized for querying
`JSONB`	Stores JSON in an efficient **binary** format, indexable and queryable - almost always preferred over plain `JSON`
Arrays (`INTEGER[]`, `TEXT[]`, etc.)	A column holding a list of values of the same type
`ENUM`	A custom type restricted to a fixed, named set

Type	Purpose
	of values
Range types (`INT4RANGE`, `TSRANGE`, etc.)	A type representing a range of values (e.g., a date range)
Network types (`INET`, `CIDR`, `MACADDR`)	Purpose-built types for IP addresses and network data

4.4 `CREATE TABLE` - Building the Course's Business Database

```
CREATE TABLE departments (
    department_id   SERIAL PRIMARY KEY,
    department_name TEXT NOT NULL UNIQUE
);

CREATE TABLE employees (
    employee_id     SERIAL PRIMARY KEY,
    full_name       TEXT NOT NULL,
    department_id   INTEGER REFERENCES departments(department_id),
    manager_id      INTEGER REFERENCES employees(employee_id),
    salary          NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    hire_date       DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE customers (
    customer_id     SERIAL PRIMARY KEY,
    name            TEXT NOT NULL,
    email           TEXT NOT NULL UNIQUE,
    city            TEXT,
    country         TEXT,
    signup_date     DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE suppliers (
    supplier_id     SERIAL PRIMARY KEY,
    supplier_name   TEXT NOT NULL,
    country         TEXT
);

CREATE TABLE products (
    product_id      SERIAL PRIMARY KEY,
    product_name    TEXT NOT NULL,
    category        TEXT,
    price           NUMERIC(10, 2) NOT NULL CHECK (price >= 0),
    stock_quantity  INTEGER NOT NULL DEFAULT 0,
    supplier_id     INTEGER REFERENCES suppliers(supplier_id)
);

CREATE TABLE stores (
    store_id        SERIAL PRIMARY KEY,
```

```
    store_name      TEXT NOT NULL,
    city            TEXT
);

CREATE TABLE orders (
    order_id        SERIAL PRIMARY KEY,
    customer_id     INTEGER NOT NULL REFERENCES
customers(customer_id),
    store_id        INTEGER REFERENCES stores(store_id),
    order_date      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    status          TEXT NOT NULL DEFAULT 'pending'
);

CREATE TABLE order_items (
    order_item_id   SERIAL PRIMARY KEY,
    order_id        INTEGER NOT NULL REFERENCES orders(order_id),
    product_id      INTEGER NOT NULL REFERENCES products(product_id),
    quantity        INTEGER NOT NULL CHECK (quantity > 0),
    unit_price      NUMERIC(10, 2) NOT NULL
);

CREATE TABLE payments (
    payment_id      SERIAL PRIMARY KEY,
    order_id        INTEGER NOT NULL REFERENCES orders(order_id),
    payment_date    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    amount          NUMERIC(10, 2) NOT NULL,
    method           TEXT NOT NULL
);

CREATE TABLE  (x9)
```

Code Element	Explanation
`SERIAL PRIMARY KEY`	`SERIAL` auto-generates a sequential integer (1, 2, 3, ...) for every new row - combined with `PRIMARY KEY` (Part 5), this is PostgreSQL's classic pattern for a simple auto-incrementing ID
`REFERENCES departments(department_id)`	Declares a **foreign key** - `employees.department_id` must match an existing `departments.department_id`, or be `NULL`
`manager_id INTEGER REFERENCES employees(employee_id)`	A **self-referencing** foreign key - an employee's manager is also a row in the same `employees` table
`NOT NULL`	This column must always have a value (Part 5)

Code Element	Explanation
`UNIQUE`	No two rows may share the same value in this column (Part 5)
`CHECK (salary > 0)`	A business rule enforced by the database itself - salary can never be zero or negative (Part 5)
`DEFAULT CURRENT_DATE` / `DEFAULT NOW()`	If no value is given on insert, PostgreSQL fills in today's date/current timestamp automatically

Note on `SERIAL`: modern PostgreSQL (10+) also supports the SQL-standard `GENERATED ALWAYS AS IDENTITY` syntax, which many professionals now prefer:

```
CREATE TABLE departments (
    department_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    department_name TEXT NOT NULL UNIQUE
);
```

Both approaches produce an auto-incrementing integer primary key; `GENERATED ALWAYS AS IDENTITY` is the newer, SQL-standard-compliant form, while `SERIAL` remains extremely common in existing codebases and tutorials.

4.5 `ALTER TABLE`

```
-- Add a new column
ALTER TABLE customers ADD COLUMN phone TEXT;

-- Drop a column
ALTER TABLE customers DROP COLUMN phone;

-- Change a column's data type
ALTER TABLE products ALTER COLUMN stock_quantity TYPE BIGINT;

-- Rename a column
ALTER TABLE products RENAME COLUMN product_name TO name;

-- Add a constraint to an existing table
ALTER TABLE employees ADD CONSTRAINT positive_salary CHECK (salary >
0);

-- Rename a table
ALTER TABLE products RENAME TO catalog_items;
ALTER TABLE catalog_items RENAME TO products;   -- renamed back
```

4.6 `DROP TABLE`

```
DROP TABLE payments;                -- fails if other tables reference
it, or if it doesn't exist
DROP TABLE IF EXISTS payments;      -- safe - does nothing (no error)
if it doesn't exist
```

```
DROP TABLE payments CASCADE;          -- also drops anything depending
on it (use with extreme caution)
```

4.7 TRUNCATE

```
TRUNCATE TABLE order_items;
```

TRUNCATE removes **all rows** from a table instantly, without scanning them individually - much faster than DELETE FROM table; for clearing an entire large table, but (like DELETE with no WHERE) it is irreversible outside of a transaction, and resets any SERIAL/identity counters back to their starting value by default.

4.8 Table Design Guidance (Preview of Part 21)

- Every table should have a clear, single-purpose identity (one table = one type of "thing").
- Every table should have a primary key.
- Use foreign keys to express relationships rather than duplicating data across tables.
- Choose the narrowest data type that genuinely fits the data's range and precision needs.
- Full normalization theory and complete design methodology are covered in Part 21.

4.9 Common Mistakes with Data Types

Mistake	Problem	Fix
Using REAL/FLOAT for money	Rounding errors accumulate in financial calculations	Use NUMERIC(p, s)
Using TIMESTAMP instead of TIMESTAMPTZ	Ambiguous/incorrect times across timezones	Use TIMESTAMPTZ
Using VARCHAR(n) "just in case" without a real length rule	Arbitrary limits that break later when requirements change	Use TEXT, add a CHECK constraint only if there's a genuine rule
Storing dates as TEXT	Loses validation, sorting, and date-arithmetic capabilities entirely	Use DATE/TIMESTAMPTZ
Forgetting NOT NULL on required business fields	Allows silently incomplete data	Add NOT NULL deliberately wherever a value is truly required

Exercises - Section 4

Beginner 1. Create the departments and employees tables from Section 4.4 yourself, then verify with \d employees. 2. Add a phone column to customers, then remove it again using ALTER TABLE.

Intermediate 3. Design and create a reviews table (referencing both customers and products) with an appropriate primary key, foreign keys, and a CHECK constraint ensuring a rating is between 1 and 5.

Advanced 4. Explain, referencing Section 4.9, why storing a `salary` column as `REAL` instead of `NUMERIC(10,2)` could cause a real payroll discrepancy over thousands of employees, with a concrete (even approximate) example. # 5. Constraints

5.1 Why Constraints Exist

A **constraint** is a rule the database enforces automatically, rejecting any data that would violate it - moving data-quality rules out of application code (where they can be forgotten, duplicated, or bypassed) and into the database itself, where they apply universally, no matter which application or script writes the data.

The examples below use the exact tables already created in Part 4 (`departments`, `employees`, `customers`, `products`, `orders`), before they are populated with the full dataset in Part 6 - so every violation shown here is real, verified behavior against the actual course schema.

5.2 `PRIMARY KEY`

Uniquely identifies each row; implicitly enforces both `NOT NULL` and `UNIQUE`. `departments.department_id` is declared `SERIAL PRIMARY KEY` in Part 4.

```
INSERT INTO departments (department_id, department_name) VALUES (1,
'Sales'), (1, 'Engineering');

ERROR:  duplicate key value violates unique constraint
"departments_pkey"
DETAIL:  Key (department_id)=(1) already exists.
```

5.3 `FOREIGN KEY`

Ensures a column's value matches an existing value in another table's primary key (or `NULL`, if the column allows it) - this is **referential integrity** (Section 5.7). `employees.department_id` references `departments.department_id`.

```
INSERT INTO employees (full_name, department_id, salary) VALUES ('Test
Employee', 999, 50000);

ERROR:  insert or update on table "employees" violates foreign key
constraint "employees_department_id_fkey"
DETAIL:  Key (department_id)=(999) is not present in table
"departments".
```

5.4 `UNIQUE`

No two rows may share the same value in this column (NULLs are an exception - multiple `NULL`s are allowed, since `NULL` means "unknown," and two unknowns are never considered equal to each other). `departments.department_name` and `customers.email` are both declared `UNIQUE` in Part 4.

```
INSERT INTO departments (department_name) VALUES ('Sales');
INSERT INTO departments (department_name) VALUES ('Sales');
```

```
INSERT 0 1
ERROR:  duplicate key value violates unique constraint
"departments_department_name_key"
DETAIL:  Key (department_name)=(Sales) already exists.
```

```
INSERT INTO customers (name, email) VALUES ('Ada', 'ada@example.com'),
('Ada2', 'ada@example.com');

ERROR:  duplicate key value violates unique constraint
"customers_email_key"
DETAIL:  Key (email)=(ada@example.com) already exists.
```

5.5 NOT NULL

Requires every row to have a real value in this column - this is part of **domain integrity**: enforcing that a column's values stay within valid, meaningful bounds. `products.product_name` is declared `NOT NULL` in Part 4.

```
INSERT INTO products (product_name, price) VALUES (NULL, 10);

ERROR:  null value in column "product_name" of relation "products"
violates not-null constraint
DETAIL:  Failing row contains (1, null, null, 10.00, 0, null).
```

5.6 CHECK

Enforces a custom, arbitrary business rule via a boolean expression. `products.price` has `CHECK (price >= 0)` in Part 4.

```
INSERT INTO products (product_name, price) VALUES ('Widget', -10);

ERROR:  new row for relation "products" violates check constraint
"products_price_check"
DETAIL:  Failing row contains (2, Widget, null, -10.00, 0, null).
```

5.7 DEFAULT

Supplies a value automatically when none is given on insert - not itself a validation rule, but closely related, since it ensures a column is never left unexpectedly empty. `orders.order_date` defaults to `NOW()` and `orders.status` defaults to `'pending'` in Part 4.

```
INSERT INTO customers (customer_id, name, email) VALUES (100, 'Test
Customer', 'test100@example.com') RETURNING customer_id;
INSERT INTO orders (customer_id) VALUES (100) RETURNING order_id,
order_date, status;

 customer_id
-------------
         100
(1 row)
```

```
 order_id |          order_date           | status
----------+-------------------------------+---------
        1 | 2026-01-15 09:00:00.123456+00 | pending
```

(The exact `order_date` shown will match whatever moment the statement is actually run, since it comes from `NOW()`.)

5.8 Referential, Entity, and Domain Integrity - Precisely Defined

Integrity Type	Meaning	Enforced By
Entity integrity	Every row must be uniquely and reliably identifiable	`PRIMARY KEY`
Referential integrity	Relationships between tables must always point to something that actually exists	`FOREIGN KEY`
Domain integrity	Every column's values must stay within valid, meaningful bounds	`NOT NULL`, `CHECK`, data types themselves

5.9 `ON DELETE` and `ON UPDATE` Behavior

When a referenced row (e.g., a customer) is deleted, PostgreSQL needs to know what should happen to the rows that reference it (e.g., that customer's orders). By default (no option specified, as in Part 4's schema), PostgreSQL uses `ON DELETE RESTRICT` - it blocks the deletion entirely.

```
INSERT INTO customers (customer_id, name, email) VALUES (1, 'Ada',
'ada@test.com');
INSERT INTO orders (customer_id, status) VALUES (1, 'completed');
DELETE FROM customers WHERE customer_id = 1;

INSERT 0 1
INSERT 0 1
ERROR:  update or delete on table "customers" violates foreign key
constraint "orders_customer_id_fkey" on table "orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table
"orders".
```

To change this behavior, the option is specified when the foreign key is defined:

```
CREATE TABLE order_items_demo (
    order_item_id SERIAL PRIMARY KEY,
    order_id      INTEGER REFERENCES orders(order_id) ON DELETE
CASCADE,
    product_id    INTEGER REFERENCES products(product_id) ON DELETE
RESTRICT,
    quantity      INTEGER NOT NULL CHECK (quantity > 0)
);
```

Option	Behavior When the Referenced Row Is Deleted
`ON DELETE CASCADE`	Automatically delete the referencing rows too (deleting an order deletes all its `order_items`)
`ON DELETE SET NULL`	Set the foreign key column to `NULL` (a row survives, but "loses" its link)
`ON DELETE RESTRICT` (the default)	**Block** the deletion entirely if any referencing rows still exist
`ON UPDATE CASCADE`	If the referenced primary key value itself changes, automatically update the referencing rows to match

Choosing the right option is a real business decision, not just a technical one: - `CASCADE` is appropriate when child rows have no meaning without their parent (e.g., `order_items` should certainly be deleted if their `order` is deleted). - `SET NULL` is appropriate when the relationship is optional and the child record should still exist independently (e.g., an employee's `manager_id` could be set to `NULL` if the manager leaves the company, rather than deleting the employee). - `RESTRICT` (the default, and what Part 4's actual schema uses throughout) is the safest choice when accidental data loss would be worse than a blocked operation - forcing a deliberate decision before anything is removed. This is why, in the real course database, deleting a customer or product that already has orders will always be blocked unless done deliberately.

```
DROP TABLE order_items_demo;   -- cleanup - this was only for the ON
DELETE illustration above
DELETE FROM orders WHERE customer_id = 1;   -- remove the demo order
first
DELETE FROM customers WHERE customer_id = 1; -- now this succeeds

DROP TABLE
DELETE 1
DELETE 1
```

Exercises - Section 5

Beginner 1. Try inserting a product with a `NULL` price and observe the exact error PostgreSQL returns.

Intermediate 2. Add a foreign key from `payments.order_id` to `orders.order_id` with `ON DELETE CASCADE` (via `ALTER TABLE ... ADD CONSTRAINT`), and explain in one sentence why this choice makes sense for payments specifically.

Advanced 3. Design a `product_reviews` table with appropriate constraints (primary key, two foreign keys referencing `customers` and `products`, a `CHECK` on rating 1-5, and a sensible `ON DELETE` choice for each foreign key), justifying each `ON DELETE` decision in a comment. # 6. Inserting Data

6.1 `INSERT INTO` - Basic Single-Row Insert

```
INSERT INTO departments (department_name) VALUES ('Sales');

INSERT 0 1
```

`INSERT 0 1` means: 0 (an OID, essentially unused in modern PostgreSQL) and 1 row inserted.

6.2 Multiple-Row Inserts

A single `INSERT` can supply many rows at once, far more efficient than one statement per row.

```
INSERT INTO departments (department_name) VALUES
    ('Sales'),
    ('Engineering'),
    ('Marketing'),
    ('Support'),
    ('HR');

INSERT 0 5
```

6.3 Inserting Selected Columns and Relying on Defaults

Any column with a `DEFAULT` (Part 5.7) or that allows `NULL` can simply be omitted from the column list.

```
INSERT INTO customers (name, email, city, country) VALUES
    ('Ada Lovelace', 'ada@example.com', 'Lagos', 'Nigeria');
    -- signup_date is omitted - it will default to CURRENT_DATE
automatically
```

6.4 The `RETURNING` Clause (PostgreSQL / Modern SQL Standard)

`RETURNING` lets an `INSERT` (or `UPDATE`/`DELETE`) immediately return the resulting row(s), avoiding a separate `SELECT` to look up what was just written - especially valuable for retrieving an auto-generated ID.

```
INSERT INTO departments (department_name)
VALUES ('Finance')
RETURNING department_id, department_name;

 department_id | department_name
---------------+-----------------
             6 | Finance
(1 row)
```

6.5 Populating the Full Course Database

The statements below build the complete dataset used throughout the rest of this course. Run them in this exact order, since later tables reference earlier ones via foreign keys.

```
INSERT INTO departments (department_id, department_name) VALUES
    (1, 'Sales'), (2, 'Engineering'), (3, 'Marketing'), (4,
```

```
'Support'), (5, 'HR');

INSERT INTO employees (employee_id, full_name, department_id,
manager_id, salary, hire_date) VALUES
    (1, 'Efe Johnson',     5, NULL, 95000, '2019-01-15'),
    (2, 'Adaeze Obi',      1, 1,    78000, '2020-03-10'),
    (3, 'Tunde Bakare',    1, 2,    52000, '2021-06-01'),
    (4, 'Chinwe Eze',      1, 2,    50000, '2022-02-14'),
    (5, 'Grace Mensah',    2, 1,    88000, '2019-11-01'),
    (6, 'Kwame Asante',    2, 5,    72000, '2021-08-20'),
    (7, 'Fatima Bello',    2, 5,    74000, '2020-09-05'),
    (8, 'Ifeoma Nwosu',    3, 1,    65000, '2021-01-11'),
    (9, 'Samuel Kariuki',  4, 1,    48000, '2022-07-19'),
    (10,'Zainab Yusuf',    4, 9,    42000, '2023-03-01');

INSERT INTO customers (customer_id, name, email, city, country,
signup_date) VALUES
    (1,  'Ada Lovelace',    'ada@example.com',          'Lagos',
'Nigeria',        '2024-01-15'),
    (2,  'Grace Hopper',    'grace@example.com',        'London',
'United Kingdom', '2024-02-20'),
    (3,  'Chidi Okafor',    'chidi@example.com',        'Lagos',
'Nigeria',        '2024-03-05'),
    (4,  'Ngozi Adichie',   'ngozi@example.com',        'Abuja',
'Nigeria',        '2024-03-22'),
    (5,  'John Smith',      'john.smith@example.com',   'New York',
'United States',  '2024-04-01'),
    (6,  'Amara Chukwu',    'amara@example.com',        'Accra',
'Ghana',          '2024-04-18'),
    (7,  'Kwabena Owusu',   'kwabena@example.com',      'Accra',
'Ghana',          '2024-05-02'),
    (8,  'Wanjiru Kamau',   'wanjiru@example.com',      'Nairobi',
'Kenya',          '2024-05-30'),
    (9,  'Emily Davis',     'emily.davis@example.com',  'Manchester',
'United Kingdom', '2024-06-10'),
    (10, 'Tariq Hassan',    'tariq@example.com',        'Nairobi',
'Kenya',          '2024-07-01'),
    (11, 'David Chen',      'david.chen@example.com',   'Toronto',
'Canada',         '2024-08-15');

INSERT INTO suppliers (supplier_id, supplier_name, country) VALUES
    (1, 'TechSource Ltd',    'United States'),
    (2, 'GlobalGoods Inc',   'China'),
    (3, 'African Crafts Co', 'Kenya'),
    (4, 'EuroSupply GmbH',   'Germany');

INSERT INTO products (product_id, product_name, category, price,
stock_quantity, supplier_id) VALUES
    (1,  'Wireless Mouse',              'Electronics', 15.99,  200,
1),
```

```sql
    (2,  'Mechanical Keyboard',          'Electronics', 79.99,   80,
1),
    (3,  'USB-C Hub',                    'Electronics', 24.99,  150,
2),
    (4,  'Noise Cancelling Headphones',  'Electronics', 129.99,  60,
1),
    (5,  'Cotton T-Shirt',               'Clothing',    12.50,  300,
2),
    (6,  'Denim Jeans',                  'Clothing',    39.99,  120,
2),
    (7,  'Leather Wallet',               'Accessories', 29.99,   90,
3),
    (8,  'Handwoven Basket',             'Home',        34.99,   45,
3),
    (9,  'Ceramic Mug Set',              'Home',        18.99,  100,
4),
    (10, 'Desk Lamp',                    'Home',        22.50,   70,
4),
    (11, 'Python Programming Book',      'Books',       34.95,   55,
1),
    (12, 'SQL Cookbook',                 'Books',       42.00,   40,
1);

INSERT INTO stores (store_id, store_name, city) VALUES
    (1, 'Downtown Flagship', 'Lagos'),
    (2, 'Online Store',      NULL),
    (3, 'Airport Kiosk',     'Nairobi');

INSERT INTO orders (order_id, customer_id, store_id, order_date,
status) VALUES
    (1,  1, 2, '2025-01-05 10:15:00+00', 'completed'),
    (2,  2, 2, '2025-01-12 14:30:00+00', 'completed'),
    (3,  3, 1, '2025-01-20 09:00:00+00', 'completed'),
    (4,  1, 2, '2025-02-02 16:45:00+00', 'completed'),
    (5,  4, 2, '2025-02-14 11:20:00+00', 'cancelled'),
    (6,  5, 2, '2025-02-18 13:10:00+00', 'completed'),
    (7,  6, 3, '2025-02-25 08:50:00+00', 'completed'),
    (8,  2, 2, '2025-03-01 15:05:00+00', 'completed'),
    (9,  7, 3, '2025-03-10 12:40:00+00', 'completed'),
    (10, 3, 1, '2025-03-15 17:25:00+00', 'pending'),
    (11, 8, 3, '2025-03-19 10:00:00+00', 'completed'),
    (12, 1, 2, '2025-03-22 09:35:00+00', 'completed'),
    (13, 9, 2, '2025-03-28 14:15:00+00', 'completed'),
    (14, 10, 3, '2025-04-02 11:50:00+00', 'completed'),
    (15, 5, 2, '2025-04-10 16:20:00+00', 'completed');

INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES
    (1, 1, 2, 15.99), (1, 5, 1, 12.50),
    (2, 2, 1, 79.99),
```

```
    (3, 7, 1, 29.99), (3, 9, 2, 18.99),
    (4, 3, 1, 24.99),
    (5, 6, 1, 39.99),
    (6, 4, 1, 129.99),
    (7, 8, 1, 34.99), (7, 10, 1, 22.50),
    (8, 1, 1, 15.99), (8, 2, 1, 79.99),
    (9, 11, 2, 34.95),
    (10, 12, 1, 42.00),
    (11, 5, 3, 12.50),
    (12, 4, 1, 129.99), (12, 1, 1, 15.99),
    (13, 6, 2, 39.99),
    (14, 9, 1, 18.99), (14, 7, 1, 29.99),
    (15, 2, 1, 79.99), (15, 3, 1, 24.99);

INSERT INTO payments (order_id, payment_date, amount, method) VALUES
    (1,  '2025-01-05 10:20:00+00', 44.48,  'credit_card'),
    (2,  '2025-01-12 14:35:00+00', 79.99,  'paypal'),
    (3,  '2025-01-20 09:05:00+00', 67.97,  'credit_card'),
    (4,  '2025-02-02 16:50:00+00', 24.99,  'debit_card'),
    (6,  '2025-02-18 13:15:00+00', 129.99, 'credit_card'),
    (7,  '2025-02-25 08:55:00+00', 57.49,  'paypal'),
    (8,  '2025-03-01 15:10:00+00', 95.98,  'credit_card'),
    (9,  '2025-03-10 12:45:00+00', 69.90,  'debit_card'),
    (11, '2025-03-19 10:05:00+00', 37.50,  'credit_card'),
    (12, '2025-03-22 09:40:00+00', 145.98, 'paypal'),
    (13, '2025-03-28 14:20:00+00', 79.98,  'credit_card'),
    (14, '2025-04-02 11:55:00+00', 48.98,  'debit_card'),
    (15, '2025-04-10 16:25:00+00', 104.98, 'credit_card');

INSERT 0 5
INSERT 0 10
INSERT 0 11
INSERT 0 4
INSERT 0 12
INSERT 0 3
INSERT 0 15
INSERT 0 22
INSERT 0 13
```

Note: since explicit `department_id`/`employee_id`/etc. values were provided (rather than relying purely on `SERIAL`'s auto-generation), PostgreSQL's internal sequence counters need to be synced afterward so future plain inserts continue from the right number:

```
SELECT setval('departments_department_id_seq', (SELECT
MAX(department_id) FROM departments));
SELECT setval('employees_employee_id_seq', (SELECT MAX(employee_id)
FROM employees));
SELECT setval('customers_customer_id_seq', (SELECT MAX(customer_id)
FROM customers));
SELECT setval('products_product_id_seq', (SELECT MAX(product_id) FROM
products));
```

```
SELECT setval('orders_order_id_seq', (SELECT MAX(order_id) FROM
orders));
```

6.6 Common Mistakes When Inserting Data

Mistake	Problem	Fix
Omitting the column list	Fragile - breaks silently if columns are ever reordered/added	Always name columns explicitly: `INSERT INTO t (a, b) VALUES (...)`
Forgetting `RETURNING` when the new ID is needed immediately	Requires a separate, unnecessary `SELECT`	Use `RETURNING` to get it in one round trip
Inserting explicit IDs without resyncing the sequence	The next plain insert can collide with an existing ID	Call `setval(...)` after bulk-loading explicit IDs

Exercises - Section 6

Beginner 1. Insert one new supplier of your own choosing, using `RETURNING` to see its generated `supplier_id`. 2. Insert two new products, referencing that supplier.

Intermediate 3. Insert a new customer and a new order for that customer in two separate statements, using `RETURNING` on the customer insert to correctly reference the new `customer_id` in the order insert.

Advanced 4. Explain, in your own words, why the `orders`, `order_items`, and `payments` tables had to be populated in that specific order relative to `customers`, `products`, and `stores`. # 7. Retrieving Data

7.1 Selecting Specific Columns

```
SELECT product_name, price FROM products;
```

```
        product_name          | price
------------------------------+--------
 Wireless Mouse               |  15.99
 Mechanical Keyboard          |  79.99
 USB-C Hub                    |  24.99
 Noise Cancelling Headphones  | 129.99
 Cotton T-Shirt               |  12.50
 Denim Jeans                  |  39.99
 Leather Wallet               |  29.99
 Handwoven Basket             |  34.99
 Ceramic Mug Set              |  18.99
 Desk Lamp                    |  22.50
 Python Programming Book      |  34.95
 SQL Cookbook                 |  42.00
(12 rows)
```

7.2 Selecting All Columns with *

```
SELECT * FROM stores;
```

```
 store_id |    store_name     |  city
----------+-------------------+---------
        1 | Downtown Flagship | Lagos
        2 | Online Store      |
        3 | Airport Kiosk     | Nairobi
(3 rows)
```

Best practice (previewed here, covered fully in Part 45): avoid `SELECT *` in real applications and reports - it silently returns every column even after the table gains new ones, wastes bandwidth transferring unused data, and makes queries harder to reason about. Name columns explicitly whenever the result will be used by anything beyond a quick, throwaway check.

7.3 Aliases

An **alias** renames a column (or table) in the result set, using `AS` (which is itself optional, but recommended for clarity).

```
SELECT product_name AS name, price AS unit_price FROM products LIMIT
3;

        name         | unit_price
---------------------+------------
 Wireless Mouse      |      15.99
 Mechanical Keyboard |      79.99
 USB-C Hub           |      24.99
(3 rows)
```

7.4 Expressions and Calculated Columns

`SELECT` can compute new values from columns and literals, not just return stored ones directly.

```
SELECT product_name, price, price * 1.075 AS price_with_tax
FROM products
LIMIT 3;

    product_name     | price | price_with_tax
---------------------+-------+----------------
 Wireless Mouse      | 15.99 |       17.18925
 Mechanical Keyboard | 79.99 |       85.98925
 USB-C Hub           | 24.99 |       26.86425
(3 rows)
```

`price_with_tax` is not stored anywhere - it is computed fresh, for this query only, every time it runs.

7.5 `DISTINCT`

Removes duplicate rows from the result set.

```
SELECT DISTINCT category FROM products ORDER BY category;
```

```
  category
-------------
 Accessories
 Books
 Clothing
 Electronics
 Home
(5 rows)
```

Without `DISTINCT`, this query would return one row *per product* (12 rows), repeating each category name once for every product in it.

7.6 `ORDER BY` - Single and Multiple Columns

```
SELECT product_name, category, price
FROM products
ORDER BY category ASC, price DESC
LIMIT 6;
```

```
        product_name         |  category   | price
-----------------------------+-------------+--------
 Leather Wallet              | Accessories |  29.99
 SQL Cookbook                | Books       |  42.00
 Python Programming Book     | Books       |  34.95
 Denim Jeans                 | Clothing    |  39.99
 Cotton T-Shirt              | Clothing    |  12.50
 Noise Cancelling Headphones | Electronics | 129.99
(6 rows)
```

Clause	Explanation
`ORDER BY category ASC, price DESC`	First groups rows alphabetically by `category`; **within** each category, sorts by `price` from highest to lowest
`ASC` (ascending)	The default if no direction is specified - smallest/earliest first
`DESC` (descending)	Largest/latest first

7.7 `LIMIT` and `OFFSET`

```
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 3
OFFSET 2;
```

```
   product_name   | price
------------------+-------
 SQL Cookbook     | 42.00
 Denim Jeans      | 39.99
```

```
 Handwoven Basket | 34.99
(3 rows)
```

Clause	Explanation
`ORDER BY price DESC`	Sorts by price, highest first, so `LIMIT/OFFSET` operate on a well-defined order (essential - without `ORDER BY`, which rows come back for a given `LIMIT/OFFSET` is not guaranteed)
`LIMIT 3`	Return at most 3 rows
`OFFSET 2`	Skip the first 2 matching rows before starting to return results - useful for "pages" of results (e.g., page 2 of a product list)

7.8 Query Execution, Conceptually

Although written with `SELECT` first, PostgreSQL conceptually evaluates a query's clauses in roughly this logical order (not the order they're typed):

```
FROM/JOIN  ->  WHERE  ->  GROUP BY  ->  HAVING  ->  SELECT  ->
DISTINCT  ->  ORDER BY  ->  LIMIT/OFFSET
```

This explains several things that confuse beginners: a column alias defined in `SELECT` cannot be used in that same query's `WHERE` clause (because `WHERE` is conceptually evaluated *before* `SELECT`), but it **can** be used in `ORDER BY` (evaluated *after* `SELECT`). `GROUP BY` and `HAVING` are covered fully in Part 11.

Exercises - Section 7

Beginner 1. Select just `name` and `country` from `customers`, aliasing `country` as `nation`. 2. Select all distinct `status` values from `orders`.

Intermediate 3. Write a query returning `product_name` and a calculated `discounted_price` column (10% off `price`), sorted from cheapest to most expensive discounted price.

Advanced 4. Write a paginated query returning "page 2" of products (5 products per page, sorted by `product_name`), using `LIMIT/OFFSET`, and explain what `OFFSET` value corresponds to "page 3." # 8. Filtering Data

8.1 `WHERE` and Comparison Operators

```
SELECT product_name, price FROM products WHERE price > 50;

        product_name         | price
-----------------------------+--------
 Mechanical Keyboard         |  79.99
 Noise Cancelling Headphones | 129.99
(2 rows)
```

Operator	Meaning
=	Equal to
<> or !=	Not equal to (<> is the SQL-standard form; != is a widely supported alias)
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

```
SELECT product_name, category FROM products WHERE category <>
'Electronics' ORDER BY product_name LIMIT 4;

  product_name   | category
-----------------+----------
 Ceramic Mug Set | Home
 Cotton T-Shirt  | Clothing
 Denim Jeans     | Clothing
 Desk Lamp       | Home
(4 rows)
```

8.2 Logical Operators: AND, OR, NOT

```
SELECT product_name, category, price FROM products WHERE category =
'Electronics' AND price < 30;

  product_name  |  category   | price
----------------+-------------+-------
 Wireless Mouse | Electronics | 15.99
 USB-C Hub      | Electronics | 24.99
(2 rows)

SELECT product_name, category FROM products WHERE category = 'Books'
OR category = 'Accessories' ORDER BY product_name;

      product_name       |  category
-------------------------+-------------
 Leather Wallet          | Accessories
 Python Programming Book | Books
 SQL Cookbook            | Books
(3 rows)

SELECT product_name, category FROM products WHERE NOT category =
'Electronics' ORDER BY product_name LIMIT 3;

  product_name   | category
-----------------+----------
 Ceramic Mug Set | Home
 Cotton T-Shirt  | Clothing
 Denim Jeans     | Clothing
(3 rows)
```

Precedence note: `AND` binds tighter than `OR`, exactly like multiplication binds tighter than addition in arithmetic. `WHERE a OR b AND c` means `WHERE a OR (b AND c)`, not `WHERE (a OR b) AND c` - use explicit parentheses whenever combining `AND`/`OR` to avoid ambiguity and bugs.

8.3 `BETWEEN`

```
SELECT product_name, price FROM products WHERE price BETWEEN 20 AND 40
ORDER BY price;
```

```
      product_name       | price
-------------------------+-------
 Desk Lamp               | 22.50
 USB-C Hub               | 24.99
 Leather Wallet          | 29.99
 Python Programming Book | 34.95
 Handwoven Basket        | 34.99
 Denim Jeans             | 39.99
(6 rows)
```

`BETWEEN 20 AND 40` is **inclusive** on both ends - equivalent to `price >= 20 AND price <= 40`.

8.4 `IN` and `NOT IN`

```
SELECT product_name, category FROM products WHERE category IN
('Books', 'Home') ORDER BY category, product_name;
```

```
      product_name       | category
-------------------------+----------
 Python Programming Book | Books
 SQL Cookbook            | Books
 Ceramic Mug Set         | Home
 Desk Lamp               | Home
 Handwoven Basket        | Home
(5 rows)
```

```
SELECT name, country FROM customers WHERE country NOT IN ('Nigeria',
'Ghana') ORDER BY name;
```

```
     name      |    country
---------------+----------------
 Emily Davis   | United Kingdom
 Grace Hopper  | United Kingdom
 John Smith    | United States
 Tariq Hassan  | Kenya
 Wanjiru Kamau | Kenya
(5 rows)
```

Important warning (previewed here, covered fully in Part 13): `NOT IN` combined with a subquery that can return `NULL` produces confusing, often entirely empty results, due to three-

valued logic (Section 8.6). This specific `NOT IN` with a literal list is safe; `NOT IN` with a subquery deserves extra caution.

8.5 `LIKE` and `ILIKE`

`LIKE` performs pattern matching: `%` matches any sequence of characters (including none), `_` matches exactly one character. **`LIKE` is case-sensitive**; **`ILIKE` is PostgreSQL's case-insensitive version.**

```
SELECT product_name FROM products WHERE product_name LIKE '%Book%';

      product_name
-------------------------
 Python Programming Book
(1 row)
```

Notice `"SQL Cookbook"` was **not** matched - it contains lowercase `"book"`, not `"Book"`, and `LIKE` is case-sensitive. Using `ILIKE` instead catches both:

```
SELECT product_name FROM products WHERE product_name ILIKE '%book%';

      product_name
-------------------------
 Python Programming Book
 SQL Cookbook
(2 rows)
```

`ILIKE` is a PostgreSQL-specific extension - not part of the SQL standard (standard SQL would require `LOWER(column) LIKE LOWER(pattern)` to achieve the same case-insensitive effect).

8.6 `IS NULL` and `IS NOT NULL`, and Three-Valued Logic

```
SELECT store_name, city FROM stores WHERE city IS NULL;

  store_name
------------
 Online Store
(1 row)
```

Critical rule: never compare to `NULL` using `=` or `<>`. `NULL` represents "unknown," not a specific value, so comparing anything to `NULL` with `=` doesn't return `TRUE` or `FALSE` - it returns `NULL` itself, which `WHERE` treats as "exclude this row."

```
SELECT store_name FROM stores WHERE city = NULL;

 store_name
------------
(0 rows)
```

This returns **zero rows**, even though a store genuinely does have a `NULL` city - because `city = NULL` evaluates to `NULL`, not `TRUE`, for every single row. PostgreSQL's three-valued logic means

every comparison evaluates to one of TRUE, FALSE, or NULL (not just TRUE/FALSE), and WHERE only ever keeps rows where the condition evaluates to exactly TRUE.

```
SELECT NULL = NULL AS result;
```

```
 result
--------

(1 row)
```

Even NULL = NULL is NULL (unknown), not TRUE - two unknown values are never considered equal to each other. This is precisely why IS NULL exists as dedicated syntax: it is the one correct way to test for NULL, since it doesn't perform a comparison at all - it directly asks "is this value the absence of a value?"

Exercises - Section 8

Beginner 1. Find all products priced strictly between 10 and 25 using BETWEEN. 2. Find all customers whose country is 'Nigeria' or 'Kenya' using IN.

Intermediate 3. Find all products whose name contains the word "Wallet" or "Basket" using LIKE and OR, then rewrite it as a single LIKE pattern if possible, or explain why it can't be a single pattern.

Advanced 4. Explain, using Section 8.6's NULL = NULL example, exactly why a query filtering WHERE customer_id NOT IN (SELECT customer_id FROM orders WHERE customer_id IS NULL) would unexpectedly return zero rows for every customer - even ones who have never ordered anything. # 9. UPDATE and DELETE

9.1 UPDATE - Basic Syntax

```
UPDATE customers SET city = 'Lekki' WHERE customer_id = 1;
```

```
UPDATE 1
```

UPDATE 1 confirms exactly one row was changed. **UPDATE without a WHERE clause changes every row in the table** - covered as a dangerous mistake in Section 9.5.

9.2 Updating Multiple Columns at Once

```
UPDATE products
SET price = price * 1.10, stock_quantity = stock_quantity - 1
WHERE category = 'Books'
RETURNING product_name, price, stock_quantity;
```

```
      product_name       | price | stock_quantity
-------------------------+-------+----------------
 Python Programming Book | 38.45 |             54
 SQL Cookbook            | 46.20 |             39

UPDATE 2
```

Every column in a single SET clause is computed using the row's **original** values (before any change in this same statement) - `price = price * 1.10` uses each book's old price, not a partially-updated one.

9.3 Conditional Updates

```
UPDATE orders SET status = 'shipped' WHERE order_id = 10 RETURNING
order_id, status;
```

```
 order_id | status
----------+---------
       10 |  shipped

UPDATE 1
```

The WHERE clause here works exactly as it does in SELECT (Part 8) - it determines precisely which rows are affected.

9.4 RETURNING with UPDATE and DELETE

RETURNING (introduced for INSERT in Part 6) works identically for UPDATE and DELETE, returning the **new** values for updated rows, or the values of rows just before they were removed for deletes.

```
DELETE FROM order_items WHERE order_id = 5 RETURNING *;
```

```
 order_item_id | order_id | product_id | quantity | unit_price
---------------+----------+------------+----------+------------
             7 |        5 |          6 |        1 |      39.99

DELETE 1
```

9.5 Dangerous Mistakes: Forgetting WHERE

```
DELETE FROM customers;
```

would delete **every single row** in the `customers` table - not a typo-safe operation, and with no built-in "are you sure?" prompt.

```
UPDATE customers
SET status = 'Inactive';
```

would overwrite **every customer's status**, silently, with no warning - a single missing WHERE clause turning a targeted fix into a catastrophic, table-wide change.

```
BEGIN;
DELETE FROM order_items;
SELECT COUNT(*) FROM order_items;
ROLLBACK;
SELECT COUNT(*) FROM order_items;
```

```
BEGIN
DELETE 22
 count
```

```
-------
     0
(1 row)

ROLLBACK
 count
-------
    22
(1 row)
```

This demonstrates both the danger (`DELETE FROM order_items;` with no `WHERE` really does remove all 22 rows) and the safety net covered next: wrapping risky statements in an explicit transaction (Part 20) so they can be verified, and rolled back, before being made permanent.

9.6 Preventing Accidental Mass Updates/Deletions

- **Always write the `WHERE` clause first**, mentally, before writing `UPDATE`/`DELETE` - or better, write and test the equivalent `SELECT` with the same `WHERE` clause first, to see exactly which rows would be affected.
- **Wrap risky statements in a transaction** (`BEGIN; ... ; ROLLBACK;` to test, or `COMMIT;` once confident) - covered fully in Part 20 - so a mistake can be undone before it becomes permanent.
- **Use `RETURNING`** to immediately see exactly what was changed/removed, catching mistakes right away rather than discovering them later.
- Referential integrity (Part 5) also provides a safety net: attempting to delete a row that other rows still depend on is blocked by default, as shown next.

```
BEGIN;
DELETE FROM orders WHERE status = 'cancelled' RETURNING order_id;
ROLLBACK;

BEGIN

ERROR:  update or delete on table "orders" violates foreign key
constraint "order_items_order_id_fkey" on table "order_items"
DETAIL:  Key (order_id)=(5) is still referenced from table
"order_items".
```

Even a seemingly reasonable, `WHERE`-qualified deletion (removing cancelled orders) is correctly blocked here, because `order_items` still references order 5 - PostgreSQL's referential integrity prevents this specific mass-deletion mistake automatically. A safe cleanup script would need to delete the dependent `order_items` rows first, deliberately, in the correct order.

Exercises - Section 9

Beginner 1. Update a single customer's `country` value, using `RETURNING` to confirm the change.

Intermediate 2. Write an `UPDATE` that increases the `price` of every product in the `'Electronics'` category by 5%, using `RETURNING` to review the affected rows before deciding whether to `COMMIT`.

Advanced 3. Write a safe procedure (as a sequence of SQL statements, using `BEGIN/ROLLBACK/COMMIT`) for deleting a specific customer and all of their related orders, order_items, and payments, in an order that won't violate foreign key constraints. # 10. SQL Functions

10.1 String Functions

```
SELECT LOWER(name) AS lower_name, UPPER(name) AS upper_name,
LENGTH(name) AS len
FROM customers LIMIT 3;
```

```
  lower_name   |  upper_name   | len
---------------+---------------+-----
 ada lovelace | ADA LOVELACE |  12
 grace hopper | GRACE HOPPER |  12
 chidi okafor | CHIDI OKAFOR |  12
(3 rows)
```

```
SELECT TRIM('  padded  ') AS trimmed;
```

```
 trimmed
---------
 padded
(1 row)
```

```
SELECT SUBSTRING('PostgreSQL' FROM 1 FOR 8) AS sub;
```

```
   sub
----------
 PostgreS
(1 row)
```

```
SELECT REPLACE('2025-01-05', '-', '/') AS reformatted;
```

```
 reformatted
-------------
 2025/01/05
(1 row)
```

```
SELECT CONCAT(name, ' <', email, '>') AS contact FROM customers LIMIT
2;
```

```
             contact
-----------------------------------
 Ada Lovelace <ada@example.com>
 Grace Hopper <grace@example.com>
(2 rows)
```

```
SELECT CONCAT_WS(', ', city, country) AS location FROM customers LIMIT
3;

        location
------------------------
 Lagos, Nigeria
 London, United Kingdom
 Lagos, Nigeria
(3 rows)
```

`CONCAT_WS` ("concat with separator") joins its arguments using the first argument as a separator, and - unlike plain string concatenation with `||` - automatically skips any `NULL` arguments rather than making the whole result `NULL`.

Function	Purpose
`LOWER(s)` / `UPPER(s)`	Case conversion
`LENGTH(s)`	Character count
`TRIM(s)`	Removes leading/trailing whitespace
`SUBSTRING(s FROM start FOR length)`	Extracts part of a string
`REPLACE(s, old, new)`	Replaces all occurrences of a substring
`CONCAT(...)`	Joins values into one string, treating `NULL` as an empty string
`CONCAT_WS(sep, ...)`	Joins values with a separator, skipping `NULL`s entirely

10.2 Numeric Functions

```
SELECT ROUND(19.987, 2) AS r, CEIL(19.1) AS c, FLOOR(19.9) AS f, ABS(-
5) AS a, POWER(2, 10) AS p, MOD(10, 3) AS m;

   r   | c  | f  | a |  p   | m
-------+----+----+---+------+---
 19.99 | 20 | 19 | 5 | 1024 | 1
(1 row)
```

Function	Purpose
`ROUND(n, d)`	Rounds `n` to `d` decimal places
`CEIL(n)`	Rounds up to the nearest integer
`FLOOR(n)`	Rounds down to the nearest integer
`ABS(n)`	Absolute value
`POWER(base, exp)`	Exponentiation
`MOD(a, b)`	Remainder of `a / b`

10.3 Date/Time Functions

```
SELECT CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP, NOW();
```

```
 current_date |    current_time    |       current_timestamp       |
now
--------------+--------------------+-------------------------------
+-------------------------------
 2026-09-07   | 21:44:40.105807+00 | 2026-09-07 21:44:40.105807+00 |
2026-09-07 21:44:40.105807+00
(1 row)
```

`CURRENT_TIMESTAMP` and `NOW()` are equivalent - `NOW()` is PostgreSQL's function-call form of the same value.

```
SELECT DATE_TRUNC('month', order_date) AS month, order_date FROM
orders LIMIT 3;
```

```
         month          |       order_date
------------------------+------------------------
 2025-01-01 00:00:00+00 | 2025-01-05 10:15:00+00
 2025-01-01 00:00:00+00 | 2025-01-12 14:30:00+00
 2025-01-01 00:00:00+00 | 2025-01-20 09:00:00+00
(3 rows)
```

`DATE_TRUNC('month', ...)` "rounds down" a timestamp to the start of its month - extremely useful for grouping data by month (Part 33 uses this heavily for month-over-month analysis).

```
SELECT DATE_PART('year', order_date) AS yr, EXTRACT(MONTH FROM
order_date) AS mo FROM orders LIMIT 3;
```

```
  yr  | mo
------+----
 2025 |  1
 2025 |  1
 2025 |  1
(3 rows)
```

`DATE_PART('year', x)` and `EXTRACT(YEAR FROM x)` are equivalent, alternate syntaxes for pulling out a specific component of a date/timestamp.

```
SELECT order_date, order_date + INTERVAL '7 days' AS week_later FROM
orders LIMIT 2;
```

```
       order_date       |       week_later
------------------------+------------------------
 2025-01-05 10:15:00+00 | 2025-01-12 10:15:00+00
 2025-01-12 14:30:00+00 | 2025-01-19 14:30:00+00
(2 rows)
```

Adding an `INTERVAL` directly to a date/timestamp performs date arithmetic naturally - PostgreSQL correctly handles month lengths, leap years, and time zones automatically.

10.4 Conditional Functions

```
SELECT product_name, price,
       CASE
           WHEN price > 50 THEN 'Expensive'
           WHEN price > 20 THEN 'Moderate'
           ELSE 'Cheap'
       END AS tier
FROM products LIMIT 5;
```

```
        product_name         | price  |   tier
-----------------------------+--------+-----------
 Wireless Mouse              |  15.99 | Cheap
 Mechanical Keyboard         |  79.99 | Expensive
 USB-C Hub                   |  24.99 | Moderate
 Noise Cancelling Headphones | 129.99 | Expensive
 Cotton T-Shirt              |  12.50 | Cheap
(5 rows)
```

`CASE` evaluates its `WHEN` conditions **in order**, stopping at the first match - a product priced exactly 51 matches `price > 50` before ever being checked against `price > 20`.

```
SELECT store_name, COALESCE(city, 'Online/No Location') AS
city_display FROM stores;
```

```
    store_name     |    city_display
-------------------+--------------------
 Downtown Flagship | Lagos
 Online Store      | Online/No Location
 Airport Kiosk     | Nairobi
(3 rows)
```

`COALESCE(a, b, c, ...)` returns the **first non-NULL** value in its argument list - the standard, idiomatic way to supply a fallback/default value for potentially-`NULL` columns.

```
SELECT NULLIF(5, 5) AS a, NULLIF(5, 6) AS b;
```

```
 a | b
---+---
   | 5
(1 row)
```

`NULLIF(a, b)` returns `NULL` if `a` equals `b`, otherwise returns `a` - useful for turning a specific "sentinel" value (like `0` or `-1` used to mean "unknown" in old data) into a genuine `NULL`.

10.5 Type Conversion

```
SELECT CAST('123' AS INTEGER) AS cast_style, '123'::INTEGER AS
colon_style;
```

```
 cast_style | colon_style
------------+-------------
```

```
        123 |         123
(1 row)
```

`CAST(value AS type)` is the SQL-standard syntax; `value::type` is PostgreSQL's own shorthand for the identical operation - both are used constantly in real PostgreSQL code, with `::` being especially common for its brevity.

Exercises - Section 10

Beginner 1. Write a query showing each product's name in uppercase alongside its price rounded to zero decimal places.

Intermediate 2. Write a query classifying each order by month name (hint: `TO_CHAR(order_date, 'Month')`) and count orders per month.

Advanced 3. Write a query using `CASE`, `COALESCE`, and a date function together to produce a single report column describing each order's age category (e.g., "This month", "Last month", "Older"), based on comparing `order_date` to `CURRENT_DATE`. # 11. Aggregate Functions

11.1 The Five Core Aggregate Functions

```
SELECT
    COUNT(*)         AS total_orders,
    SUM(amount)       AS total_revenue,
    AVG(amount)        AS avg_payment,
    MIN(amount)         AS min_payment,
    MAX(amount)          AS max_payment
FROM payments;
```

```
 total_orders | total_revenue |     avg_payment     | min_payment |
max_payment
--------------+---------------+---------------------+-------------
+-------------
           13 |        988.21 | 76.0161538461538462 |       24.99 |
145.98
(1 row)
```

Function	Purpose
`COUNT(*)`	Number of rows
`COUNT(column)`	Number of **non-NULL** values in that column (different from `COUNT(*)` if the column has NULLs)
`COUNT(DISTINCT column)`	Number of distinct non-NULL values
`SUM(column)`	Total of a numeric column
`AVG(column)`	Arithmetic mean
`MIN(column)` / `MAX(column)`	Smallest / largest value

An aggregate function with no `GROUP BY` collapses the **entire table** into a single summary row, as shown above.

11.2 GROUP BY

GROUP BY splits rows into groups sharing the same value(s) in specified column(s), and computes aggregates **per group** instead of over the whole table.

```
SELECT category, COUNT(*) AS product_count, AVG(price) AS avg_price
FROM products
GROUP BY category
ORDER BY category;
```

```
  category   | product_count |      avg_price
-------------+---------------+---------------------
 Accessories |             1 | 29.9900000000000000
 Books       |             2 | 38.4750000000000000
 Clothing    |             2 | 26.2450000000000000
 Electronics |             4 | 62.7400000000000000
 Home        |             3 | 25.4933333333333333
(5 rows)
```

Rule: every column in SELECT that is not itself an aggregate function must appear in GROUP BY - PostgreSQL enforces this, since otherwise it would be ambiguous which specific row's value to show for a non-aggregated column within a group of many rows.

11.3 HAVING

HAVING filters **groups** (after aggregation), exactly as WHERE filters individual **rows** (before aggregation) - the fundamental difference between the two.

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category
HAVING COUNT(*) > 2
ORDER BY category;
```

```
  category   | product_count
-------------+---------------
 Electronics |             4
 Home        |             3
(2 rows)
```

11.4 WHERE vs HAVING - Both Together

```
SELECT category, AVG(price) AS avg_price
FROM products
WHERE price < 100
GROUP BY category
HAVING AVG(price) > 20
ORDER BY category;
```

```
  category   |      avg_price
-------------+---------------------
 Accessories | 29.9900000000000000
```

```
 Books       | 38.4750000000000000
 Clothing    | 26.2450000000000000
 Electronics | 40.3233333333333333
 Home        | 25.4933333333333333
(5 rows)
```

Clause	Filters	Runs (Conceptually)	Can Reference Aggregates?
`WHERE`	Individual rows	Before grouping/aggregation	No - aggregates don't exist yet at this stage
`HAVING`	Groups	After grouping/aggregation	Yes - that's its entire purpose

Notice `Electronics`'s average price changed (from `62.74` to `40.32`) between Section 11.2 and this query - `WHERE price < 100` excluded the `129.99` headphones **before** the average was computed, while `HAVING` only decides whether to keep or discard the *already-computed* group average.

11.5 Realistic Business Questions

Total sales:

```
SELECT SUM(amount) AS total_sales FROM payments;
```

```
 total_sales
-------------
      988.21
(1 row)
```

Average order value:

```
SELECT ROUND(AVG(amount), 2) AS avg_order_value FROM payments;
```

```
 avg_order_value
-----------------
           76.02
(1 row)
```

Number of customers who have ordered:

```
SELECT COUNT(DISTINCT customer_id) AS unique_customers FROM orders;
```

```
 unique_customers
------------------
               10
(1 row)
```

Revenue by country:

```
SELECT c.country, SUM(p.amount) AS revenue
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN payments p ON p.order_id = o.order_id
GROUP BY c.country
ORDER BY revenue DESC;
```

```
    country     | revenue
----------------+---------
 Nigeria        |  283.42
 United Kingdom |  255.95
 United States  |  234.97
 Ghana          |  127.39
 Kenya          |   86.48
(5 rows)
```

(Joins are formally introduced in Part 12; this preview shows exactly why they matter for realistic aggregation.)

Revenue by month:

```
SELECT DATE_TRUNC('month', payment_date) AS month, SUM(amount) AS
revenue
FROM payments
GROUP BY month
ORDER BY month;
```

```
         month          | revenue
------------------------+---------
 2025-01-01 00:00:00+00 |  192.44
 2025-02-01 00:00:00+00 |  212.47
 2025-03-01 00:00:00+00 |  429.34
 2025-04-01 00:00:00+00 |  153.96
(4 rows)
```

Best-selling products (by units sold):

```
SELECT pr.product_name, SUM(oi.quantity) AS units_sold
FROM order_items oi
JOIN products pr ON pr.product_id = oi.product_id
GROUP BY pr.product_name
ORDER BY units_sold DESC
LIMIT 5;
```

```
    product_name     | units_sold
---------------------+------------
 Cotton T-Shirt      |          4
 Wireless Mouse      |          4
 Denim Jeans         |          3
 Mechanical Keyboard |          3
 Ceramic Mug Set     |          3
(5 rows)
```

Customer purchase frequency:

```
SELECT c.name, COUNT(o.order_id) AS orders_placed
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.name
ORDER BY orders_placed DESC
LIMIT 5;
```

```
     name     | orders_placed
--------------+---------------
 Ada Lovelace |             3
 Chidi Okafor |             2
 Grace Hopper |             2
 John Smith   |             2
 Emily Davis  |             1
(5 rows)
```

Exercises - Section 11

Beginner 1. Count the number of products in each supplier's catalog, sorted from most to fewest.

Intermediate 2. Find every product category whose average price exceeds $30, using `HAVING`.

Advanced 3. Write a query showing total revenue per store (joining `orders`, `payments`, and `stores`), including stores with zero completed-order revenue (hint: this requires a join type covered in Part 12 - note the challenge here and revisit after reading it). # 12. Joins

12.1 Why Joins Exist

Relational design (Part 5, Part 21) deliberately splits data across multiple tables to avoid duplication - a customer's name is stored once in `customers`, not copied into every one of their orders. A **join** is how SQL reassembles related data back together at query time, matching rows across tables based on a shared key.

12.2 Join Keys

A **join key** is the column(s) used to match rows between two tables - almost always a foreign key in one table paired with the primary key it references.

```
customers                        orders
 customer_id (PK) | name         order_id (PK) | customer_id (FK)
------------------+------       --------------+------------------
 1                | Ada          1             | 1
 2                | Grace         2             | 2
```

Joining `orders.customer_id` to `customers.customer_id` reconnects each order with its customer.

12.3 INNER JOIN

Returns only rows where a match exists in **both** tables.

```
SELECT o.order_id, c.name, o.order_date
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_id
LIMIT 5;
```

```
 order_id |     name      |       order_date
----------+---------------+------------------------
        1 | Ada Lovelace  | 2025-01-05 10:15:00+00
        2 | Grace Hopper  | 2025-01-12 14:30:00+00
        3 | Chidi Okafor  | 2025-01-20 09:00:00+00
        4 | Ada Lovelace  | 2025-02-02 16:45:00+00
        5 | Ngozi Adichie | 2025-02-14 11:20:00+00
(5 rows)
```

Since every order genuinely has a valid `customer_id` (enforced by the `NOT NULL` and foreign key from Part 4-5), `INNER JOIN` and plain `JOIN` (its default synonym) return every order here - but if a customer had been deleted, orphaned orders would silently disappear from this result, which is precisely the difference `LEFT JOIN` addresses.

12.4 LEFT JOIN (LEFT OUTER JOIN)

Returns **every** row from the left table, matched with the right table where possible - filling in `NULL` for the right side wherever no match exists.

```
SELECT s.store_name, COUNT(o.order_id) AS order_count
FROM stores s
LEFT JOIN orders o ON o.store_id = s.store_id
GROUP BY s.store_name
ORDER BY s.store_name;
```

```
    store_name     | order_count
-------------------+-------------
 Airport Kiosk     |           4
 Downtown Flagship |           2
 Online Store      |           9
(3 rows)
```

Every store appears, even if it had zero orders - an `INNER JOIN` here would have silently dropped any store with no orders at all from the results entirely.

The single most common practical use of `LEFT JOIN`: finding what's *missing*.

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

```
    name
------------
 David Chen
(1 row)
```

This finds every customer who has **never** placed an order - `David Chen` is the only such customer in this dataset. The `WHERE o.order_id IS NULL` condition is the key idiom: it keeps only the rows where the `LEFT JOIN` found no match at all.

12.5 `RIGHT JOIN` (`RIGHT OUTER JOIN`)

The mirror image of `LEFT JOIN`: returns every row from the **right** table, matched with the left table where possible.

```
SELECT s.store_name, o.order_id
FROM orders o
RIGHT JOIN stores s ON o.store_id = s.store_id
ORDER BY s.store_name
LIMIT 5;
```

```
    store_name     | order_id
-------------------+----------
 Airport Kiosk     |       11
 Airport Kiosk     |       14
 Airport Kiosk     |        7
 Airport Kiosk     |        9
 Downtown Flagship |        3
(5 rows)
```

`A RIGHT JOIN B` always returns exactly the same result as `B LEFT JOIN A` - in practice, most professionals write `LEFT JOIN` almost exclusively (reordering the tables instead) simply because it reads more naturally, and `RIGHT JOIN` is rarely seen in real production code.

12.6 `FULL OUTER JOIN`

Returns every row from **both** tables, matched where possible, with `NULL`s filled in on whichever side has no match.

```
SELECT d.department_name, e.full_name
FROM departments d
FULL OUTER JOIN employees e ON e.department_id = d.department_id
ORDER BY d.department_name;
```

```
 department_name |   full_name
-----------------+----------------
 Engineering     | Grace Mensah
 Engineering     | Fatima Bello
 Engineering     | Kwame Asante
 HR              | Efe Johnson
 Marketing       | Ifeoma Nwosu
 Sales           | Chinwe Eze
```

```
 Sales           | Tunde Bakare
 Sales           | Adaeze Obi
 Support         | Samuel Kariuki
 Support         | Zainab Yusuf
(10 rows)
```

If any department had zero employees, it would still appear here (with `full_name` as `NULL`) - and if any employee somehow had no department, they would still appear too. `FULL OUTER JOIN` is used far less often than `INNER/LEFT JOIN` in practice, but is exactly the right tool when you need to see mismatches from *both* sides at once (e.g., a full data-reconciliation report).

12.7 `CROSS JOIN`

Returns the full **Cartesian product**: every row from the first table combined with every row from the second, with no matching condition at all.

```
SELECT s.store_name, d.department_name
FROM stores s
CROSS JOIN departments d
ORDER BY s.store_name, d.department_name
LIMIT 6;
```

```
    store_name     | department_name
-------------------+-----------------
 Airport Kiosk     | Engineering
 Airport Kiosk     | HR
 Airport Kiosk     | Marketing
 Airport Kiosk     | Sales
 Airport Kiosk     | Support
 Downtown Flagship | Engineering
(6 rows)
```

With 3 stores and 5 departments, this produces exactly 3 x 5 = 15 rows total. `CROSS JOIN` is genuinely useful for deliberately generating combinations (e.g., every product x every possible size, or a calendar table crossed with every store, for reporting scaffolding) - but is a serious **accidental** bug (Section 12.9) when it happens unintentionally.

12.8 Self Joins

A **self join** joins a table to itself - useful whenever rows in a table relate to *other rows in that same table*, such as an employee's manager also being an employee.

```
SELECT e.full_name AS employee, m.full_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id
ORDER BY e.employee_id;
```

```
    employee    |    manager
----------------+----------------
 Efe Johnson    |
```

```
 Adaeze Obi     | Efe Johnson
 Tunde Bakare   | Adaeze Obi
 Chinwe Eze     | Adaeze Obi
 Grace Mensah   | Efe Johnson
 Kwame Asante   | Grace Mensah
 Fatima Bello   | Grace Mensah
 Ifeoma Nwosu   | Efe Johnson
 Samuel Kariuki | Efe Johnson
 Zainab Yusuf   | Samuel Kariuki
(10 rows)
```

The **same table**, `employees`, is given two different aliases (`e` for "the employee," `m` for "their manager") purely so the query can refer to two conceptually different roles played by the identical underlying table. `LEFT JOIN` (rather than `INNER JOIN`) is essential here - `Efe Johnson` has no manager (`manager_id IS NULL`), and an `INNER JOIN` would have silently excluded them from the results entirely.

12.9 One-to-One, One-to-Many, and Many-to-Many Relationships

Relationship	Example	How It's Modeled
One-to-one	One `employee` has exactly one optional office-assignment record	A foreign key with a `UNIQUE` constraint on the child table
One-to-many	One `customer` has many `orders`	A plain foreign key on the "many" side (`orders.customer_id`)
Many-to-many	Many `orders` contain many `products`, and each `product` appears in many `orders`	A **junction table** in between - `order_items`, with foreign keys to both `orders` and `products`

`order_items` is precisely this course's many-to-many junction table: it is what makes it possible to say "this order contains these products" and "this product appears in these orders," without either `orders` or `products` needing a variable, awkward list of the other.

12.10 Join Duplicates

Joining to a table on the "many" side of a one-to-many relationship naturally multiplies rows - this is expected, not a bug, but it must be accounted for when aggregating.

```
SELECT o.order_id, COUNT(*) FROM orders o JOIN order_items oi ON
oi.order_id = o.order_id WHERE o.order_id IN (1, 2) GROUP BY
o.order_id;
```

```
 order_id | count
----------+-------
        1 |     2
        2 |     1
(2 rows)
```

Order 1 has 2 line items, so joining `orders` to `order_items` produces **2 rows for order 1 alone** - if you then tried to `SUM` something from the `orders` table itself (like counting orders) across this joined result without `DISTINCT` or careful aggregation, you would over-count.

12.11 Missing Matches (Preview, Full Detail in Section 12.4)

Using `INNER JOIN` where a `LEFT JOIN` was actually needed **silently drops rows** with no match - one of the most common, hardest-to-notice bugs in real SQL, precisely because the query runs without error and simply returns fewer rows than expected, with no warning.

12.12 Incorrect Joins and Accidental Cartesian Products

Forgetting the `ON` condition (or using a comma-separated `FROM` list without a `WHERE` condition linking the tables) produces an accidental Cartesian product - syntactically valid, but almost always a serious logic bug.

```
SELECT COUNT(*) FROM orders o, order_items oi;

 count
-------
   330
```

This returns `330` rows (15 orders x 22 order_items) - every order incorrectly paired with every line item, rather than each order paired only with its own line items. Always double-check that a join's `ON` condition is present and correct, especially when a query's row count looks suspiciously large.

Exercises - Section 12

Beginner 1. Write an `INNER JOIN` between `order_items` and `products` showing the product name and quantity for every line item in order 1. 2. Write a `LEFT JOIN` finding every product that has never appeared in any `order_items` row (hint: mirror Section 12.4's "missing customer" pattern).

Intermediate 3. Write a query joining `employees` to `departments` and to a self-join on `employees` (for the manager), showing each employee's name, department, and manager's name in one single result.

Advanced 4. Deliberately write a query that produces an accidental Cartesian product by joining `customers` and `orders` with no `ON` condition, confirm the row count matches `(customer count) x (order count)`, then fix it with a correct `ON` condition and explain the difference in row counts. # 13. Subqueries

13.1 What a Subquery Is

A **subquery** is a query nested inside another query - used anywhere a value, list of values, or table can appear: in `SELECT`, `WHERE`, `FROM`, or `HAVING`.

13.2 Scalar Subqueries

A **scalar subquery** returns exactly one row and one column - usable anywhere a single value is expected.

```
SELECT product_name, price, (SELECT AVG(price) FROM products) AS
avg_price
FROM products
LIMIT 3;

    product_name     | price |      avg_price
---------------------+-------+---------------------
 Wireless Mouse      | 15.99 | 40.5725000000000000
 Mechanical Keyboard | 79.99 | 40.5725000000000000
 USB-C Hub           | 24.99 | 40.5725000000000000
(3 rows)
```

The subquery `(SELECT AVG(price) FROM products)` runs once, producing a single number, repeated alongside every row.

13.3 Single-Row Subqueries in `WHERE`

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;

        product_name         | price
-----------------------------+--------
 Noise Cancelling Headphones | 129.99
 Mechanical Keyboard         |  79.99
 SQL Cookbook                |  42.00
(3 rows)
```

If a subquery used with =, >, < etc. ever returns **more than one row**, PostgreSQL raises an error (`more than one row returned by a subquery used as an expression`) - such comparisons require the subquery to resolve to exactly one value.

13.4 Multi-Row Subqueries with `IN`

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE status =
'cancelled');

     name
---------------
 Ngozi Adichie
(1 row)
```

`IN` is designed for subqueries that return multiple rows (a list of values) - unlike =, it does not require exactly one row.

13.5 Correlated Subqueries

A **correlated subquery** references a column from the *outer* query, meaning it must conceptually run once **per row** of the outer query, rather than just once overall.

```
SELECT p.product_name, p.category, p.price
FROM products p
WHERE p.price > (SELECT AVG(p2.price) FROM products p2 WHERE
p2.category = p.category)
ORDER BY p.category;

        product_name         |  category   | price
-----------------------------+-------------+--------
 SQL Cookbook                | Books       |  42.00
 Denim Jeans                 | Clothing    |  39.99
 Mechanical Keyboard         | Electronics |  79.99
 Noise Cancelling Headphones | Electronics | 129.99
 Handwoven Basket            | Home        |  34.99
(5 rows)
```

Notice `p2.category = p.category` reaches back into the outer query's current row (`p`) - so the "average" being compared against is a **different** number for each category, computed fresh for every row of the outer query. This finds each product priced above its **own category's** average, a genuinely different (and more useful) question than Section 13.3's single, table-wide average.

13.6 `EXISTS` and `NOT EXISTS`

`EXISTS(subquery)` returns TRUE if the subquery returns **at least one row** - it never actually looks at the subquery's column values, only whether any row exists, which lets the database stop searching the instant a single match is found.

```
SELECT name FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id AND
o.status = 'cancelled'
);

     name
---------------
 Ngozi Adichie
(1 row)

SELECT name FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

    name
------------
```

```
 David Chen
(1 row)
```

`NOT EXISTS` correctly finds `David Chen`, the one customer with zero orders - this is the **safe**, recommended way to answer "which rows have no match," in contrast to `NOT IN`, covered next.

13.7 The `NOT IN` + `NULL` Trap

```
SELECT name FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);

    name
------------
 David Chen
(1 row)
```

This looks identical to the `NOT EXISTS` result above - and it is, **as long as the subquery's result contains no `NULL` values**. But watch what happens the moment even one `NULL` sneaks into that list:

```
SELECT name FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders UNION SELECT
NULL);

 name
------
(0 rows)
```

Zero rows - even though `David Chen` still, genuinely, has no orders. This happens because of three-valued logic (Part 8.6): `NOT IN (a, b, NULL)` is really `<> a AND <> b AND <> NULL`, and comparing anything to `NULL` produces `NULL`, not `TRUE` - so the entire `AND` chain becomes `NULL` (not `TRUE`) for every single row, and `WHERE` discards every row where the condition isn't `TRUE`.

The rule this teaches, permanently: prefer `NOT EXISTS` over `NOT IN` whenever the subquery's column could possibly contain a `NULL` - it sidesteps this trap entirely, since `NOT EXISTS` never compares values at all, only checks for the presence or absence of matching rows.

13.8 Subqueries vs Joins - When Each Is Appropriate

Situation	Prefer
You need columns from **both** tables in the final result	`JOIN`
You only need to filter one table based on a condition in another, without needing the other table's columns in the output	Subquery (`IN`, `EXISTS`)
You need to check for the **existence** (or absence) of related rows	`EXISTS` / `NOT EXISTS`

Situation	Prefer
You need a single summary value (an average, a max) to compare against	Scalar subquery
The subquery's list could contain `NULL`	`EXISTS/NOT EXISTS` instead of `IN/NOT IN`

In practice, PostgreSQL's query planner (Part 19) often rewrites simple `IN/EXISTS` subqueries into an equivalent join-like plan internally - so the choice is frequently more about **readability and correctness** (especially the `NULL` trap above) than raw performance, though this is not guaranteed for every query shape.

Exercises - Section 13

Beginner 1. Write a scalar subquery showing each employee's salary alongside the company-wide average salary.

Intermediate 2. Write a correlated subquery finding every employee who earns more than their own department's average salary.

Advanced 3. Reproduce Section 13.7's `NOT IN` + `NULL` trap using a different pair of tables from this course's schema, and rewrite the query safely using `NOT EXISTS`. # 14. Common Table Expressions (CTEs)

14.1 What `WITH` Does

A **Common Table Expression (CTE)**, introduced with `WITH`, defines a named, temporary result set that can be referenced later in the same query - like giving a subquery a name and using it as if it were a real table.

```
WITH high_value_orders AS (
    SELECT order_id, customer_id FROM payments p JOIN orders o USING
(order_id) WHERE p.amount > 100
)
SELECT c.name, h.order_id
FROM high_value_orders h
JOIN customers c ON c.customer_id = h.customer_id;

     name     | order_id
--------------+----------
 John Smith   |        6
 Ada Lovelace |       12
 John Smith   |       15
(3 rows)
```

(`USING (order_id)` is a shorthand join form, equivalent to `ON o.order_id = p.order_id` when both tables share an identically-named join column - covered further in Part 38.)

14.2 Why Use a CTE Instead of a Subquery?

The exact same result could be written as a nested subquery, but the CTE version is easier to read top-to-bottom, and the name `high_value_orders` documents *what the intermediate result represents* - a meaningful readability advantage that grows quickly as queries become more complex.

14.3 Multiple CTEs in One Query

A single `WITH` can define several CTEs, comma-separated, and later CTEs can reference earlier ones.

```
WITH order_totals AS (
    SELECT order_id, SUM(quantity * unit_price) AS total FROM
order_items GROUP BY order_id
),
customer_orders AS (
    SELECT o.customer_id, ot.total FROM orders o JOIN order_totals ot
ON ot.order_id = o.order_id
)
SELECT c.name, SUM(co.total) AS lifetime_value
FROM customer_orders co
JOIN customers c ON c.customer_id = co.customer_id
GROUP BY c.name
ORDER BY lifetime_value DESC
LIMIT 5;
```

```
     name     | lifetime_value
--------------+----------------
 John Smith   |         234.97
 Ada Lovelace |         215.45
 Grace Hopper |         175.97
 Chidi Okafor |         109.97
 Emily Davis  |          79.98
(5 rows)
```

Breaking a complex calculation into named stages (`order_totals`, then `customer_orders`) mirrors how a person would naturally explain the calculation out loud, step by step.

14.4 CTE Chains

CTEs can be chained, each one building on the previous, forming a readable pipeline.

```
WITH step1 AS (
    SELECT * FROM products WHERE category = 'Electronics'
),
step2 AS (
    SELECT product_name, price FROM step1 WHERE price > 20
)
SELECT * FROM step2 ORDER BY price;
```

```
        product_name         | price
-----------------------------+--------
 USB-C Hub                   |  24.99
 Mechanical Keyboard         |  79.99
 Noise Cancelling Headphones | 129.99
(3 rows)
```

14.5 Recursive CTEs

`WITH RECURSIVE` lets a CTE reference **itself**, essential for hierarchical or graph-like data - such as an organizational chart, where employees report to managers who themselves report to other managers.

```sql
WITH RECURSIVE org_chart AS (
    -- Base case: the top of the hierarchy (no manager)
    SELECT employee_id, full_name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive case: find employees managed by anyone already in
org_chart
    SELECT e.employee_id, e.full_name, e.manager_id, oc.level + 1
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.employee_id
)
SELECT level, full_name FROM org_chart ORDER BY level, full_name;
```

```
 level |   full_name
-------+----------------
     1 | Efe Johnson
     2 | Adaeze Obi
     2 | Grace Mensah
     2 | Ifeoma Nwosu
     2 | Samuel Kariuki
     3 | Chinwe Eze
     3 | Fatima Bello
     3 | Kwame Asante
     3 | Tunde Bakare
     3 | Zainab Yusuf
(10 rows)
```

Part	Explanation
Base case	`SELECT ... WHERE manager_id IS NULL` - the starting point, employees with no manager (level 1)
`UNION ALL`	Combines the base case with the recursive case's results (see Part 16 for `UNION` vs

Part	Explanation
	`UNION ALL`)
Recursive case	Joins `employees` back to `org_chart` **itself**, finding the next level down each time it runs
Termination	PostgreSQL automatically stops once the recursive case produces no new rows - here, once no employee reports to anyone at the current deepest level

Recursive CTEs are the standard SQL way to traverse trees and graphs: org charts, category hierarchies (a category containing subcategories containing sub-subcategories), or bill-of-materials structures, all follow this same base-case-plus-recursive-case pattern.

14.6 CTEs for Data Cleaning

CTEs are also commonly used purely for readability during multi-step data cleaning, even without any recursion:

```
WITH normalized_customers AS (
    SELECT
        customer_id,
        TRIM(name) AS name,
        LOWER(TRIM(email)) AS email,
        COALESCE(country, 'Unknown') AS country
    FROM customers
)
SELECT country, COUNT(*) FROM normalized_customers GROUP BY country
ORDER BY country;
```

```
     country     | count
-----------------+-------
 Canada          |     1
 Ghana           |     2
 Kenya           |     2
 Nigeria         |     3
 United Kingdom  |     2
 United States   |     1
(6 rows)
```

14.7 Readability and Performance Considerations

- A CTE that is referenced only **once** is, by default in modern PostgreSQL (12+), typically inlined by the planner exactly like an equivalent subquery would be - so there is usually no inherent performance penalty for using a CTE purely for readability.
- A CTE referenced **multiple times** in the same query may be computed once and reused (an optimization fence), or inlined multiple times, depending on the query and PostgreSQL version - if performance matters, check the actual behavior with `EXPLAIN` (Part 18-19) rather than assuming.

- CTEs dramatically improve readability for multi-step analytical logic - almost every advanced query in Part 33 and Part 37 relies on them to break a complex calculation into clear, named stages.

Exercises - Section 14

Beginner 1. Write a CTE named `expensive_products` selecting all products priced above $50, then select from it with an `ORDER BY`.

Intermediate 2. Write two chained CTEs: the first computing each customer's total number of orders, the second filtering to only customers with more than one order.

Advanced 3. Write a recursive CTE that, starting from `Efe Johnson`, lists every employee in their entire management chain downward (their direct reports, their reports' reports, etc.), including the recursion depth. # 15. Window Functions

15.1 What a Window Function Is - And How It Differs from `GROUP BY`

A **window function** computes a value across a set of related rows (its "window"), **without collapsing them into a single row** the way `GROUP BY` does - every original row survives in the output, each annotated with a value computed from its window.

```
-- GROUP BY collapses rows: 12 products become 5 category rows
SELECT category, AVG(price) FROM products GROUP BY category ORDER BY
category;

  category   |         avg
-------------+---------------------
 Accessories | 29.9900000000000000
 Books       | 38.4750000000000000
 Clothing    | 26.2450000000000000
 Electronics | 62.7400000000000000
 Home        | 25.4933333333333333
(5 rows)

-- A window function keeps all 12 product rows, each annotated with
its category average
SELECT product_name, category, price, AVG(price) OVER (PARTITION BY
category) AS category_avg
FROM products
ORDER BY category, product_name;

        product_name         |  category   | price  |    category_avg
-----------------------------+-------------+--------
+---------------------
 Leather Wallet              | Accessories |  29.99 |
29.9900000000000000
 Python Programming Book     | Books       |  34.95 |
38.4750000000000000
 SQL Cookbook                | Books       |  42.00 |
38.4750000000000000
```

```
 Cotton T-Shirt               | Clothing    |  12.50 |
26.2450000000000000
 Denim Jeans                  | Clothing    |  39.99 |
26.2450000000000000
 Mechanical Keyboard          | Electronics |  79.99 |
62.7400000000000000
 Noise Cancelling Headphones | Electronics | 129.99 |
62.7400000000000000
 USB-C Hub                    | Electronics |  24.99 |
62.7400000000000000
 Wireless Mouse               | Electronics |  15.99 |
62.7400000000000000
 Ceramic Mug Set              | Home        |  18.99 |
25.4933333333333333
 Desk Lamp                    | Home        |  22.50 |
25.4933333333333333
 Handwoven Basket             | Home        |  34.99 |
25.4933333333333333
(12 rows)
```

This is the essential distinction: `GROUP BY` answers "what is the average, per category?" (5 rows out); a window function answers "what is my category's average, next to my own row?" (12 rows out, each still showing its own product).

15.2 `OVER()`, `PARTITION BY`, and `ORDER BY` Inside a Window

```
function_name() OVER (
    PARTITION BY column    -- optional: divides rows into independent
groups
    ORDER BY column          -- optional: defines order within each
partition
    frame_clause               -- optional: which rows within the
partition to include (Section 15.6)
)
```

- `PARTITION BY` restarts the window's calculation for each distinct value - exactly like `GROUP BY`, but without collapsing rows.
- `ORDER BY` inside `OVER(...)` determines the order used for ranking, running totals, `LAG`/`LEAD`, etc. - and is completely independent of the query's own outer `ORDER BY`.
- Omitting `PARTITION BY` entirely treats the whole result set as one single window.

15.3 Ranking Functions: `ROW_NUMBER`, `RANK`, `DENSE_RANK`

```
SELECT product_name, category, price,
       ROW_NUMBER() OVER (PARTITION BY category ORDER BY price DESC)
AS rn,
       RANK()       OVER (PARTITION BY category ORDER BY price DESC)
AS rnk,
       DENSE_RANK() OVER (PARTITION BY category ORDER BY price DESC)
AS drnk
```

```
FROM products
ORDER BY category, price DESC;
```

```
        product_name         |  category   | price  | rn | rnk | drnk
-----------------------------+-------------+--------+----+-----+------
 Leather Wallet              | Accessories |  29.99 |  1 |   1 |    1
 SQL Cookbook                | Books       |  42.00 |  1 |   1 |    1
 Python Programming Book     | Books       |  34.95 |  2 |   2 |    2
 Denim Jeans                 | Clothing    |  39.99 |  1 |   1 |    1
 Cotton T-Shirt              | Clothing    |  12.50 |  2 |   2 |    2
 Noise Cancelling Headphones | Electronics | 129.99 |  1 |   1 |    1
 Mechanical Keyboard         | Electronics |  79.99 |  2 |   2 |    2
 USB-C Hub                   | Electronics |  24.99 |  3 |   3 |    3
 Wireless Mouse              | Electronics |  15.99 |  4 |   4 |    4
 Handwoven Basket            | Home        |  34.99 |  1 |   1 |    1
 Desk Lamp                   | Home        |  22.50 |  2 |   2 |    2
 Ceramic Mug Set             | Home        |  18.99 |  3 |   3 |    3
(12 rows)
```

This dataset has no price ties within a category, so all three ranking functions agree here. **Their difference only appears with tied values:**

Function	Behavior with Ties
`ROW_NUMBER()`	Always assigns a unique, sequential number - arbitrarily breaks ties
`RANK()`	Tied rows get the same rank; the **next** rank then skips (e.g., `1, 2, 2, 4`)
`DENSE_RANK()`	Tied rows get the same rank; the next rank does **not** skip (e.g., `1, 2, 2, 3`)

15.4 `NTILE` - Dividing into Buckets

```
SELECT product_name, price, NTILE(4) OVER (ORDER BY price) AS quartile
FROM products ORDER BY price;
```

```
        product_name         | price  | quartile
-----------------------------+--------+----------
 Cotton T-Shirt              |  12.50 |        1
 Wireless Mouse              |  15.99 |        1
 Ceramic Mug Set             |  18.99 |        1
 Desk Lamp                   |  22.50 |        2
 USB-C Hub                   |  24.99 |        2
 Leather Wallet              |  29.99 |        2
 Python Programming Book     |  34.95 |        3
 Handwoven Basket            |  34.99 |        3
 Denim Jeans                 |  39.99 |        3
 SQL Cookbook                |  42.00 |        4
 Mechanical Keyboard         |  79.99 |        4
 Noise Cancelling Headphones | 129.99 |        4
(12 rows)
```

`NTILE(4)` divides the ordered rows into 4 roughly equal-sized buckets ("quartiles") - a common technique for pricing tiers, customer segmentation (Part 33), or performance percentile grouping.

15.5 Navigation Functions: `LAG` and `LEAD`

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', payment_date)::date AS month,
SUM(amount) AS revenue
    FROM payments GROUP BY DATE_TRUNC('month', payment_date)::date
)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month,
       LEAD(revenue) OVER (ORDER BY month) AS next_month
FROM monthly ORDER BY month;
```

```
   month    | revenue | prev_month | next_month
------------+---------+------------+------------
 2025-01-01 |  192.44 |            |     212.47
 2025-02-01 |  212.47 |     192.44 |     429.34
 2025-03-01 |  429.34 |     212.47 |     153.96
 2025-04-01 |  153.96 |     429.34 |
(4 rows)
```

`LAG(x)` looks at the **previous** row's value (within the same window); `LEAD(x)` looks at the **next** row's value - both return `NULL` at the boundaries where no such row exists. This is the foundation of month-over-month growth calculations (Part 33.11).

15.6 `FIRST_VALUE` and `LAST_VALUE`, and Window Frames

```
SELECT product_name, category, price,
       FIRST_VALUE(product_name) OVER (PARTITION BY category ORDER BY
price DESC) AS most_expensive,
       LAST_VALUE(product_name) OVER (
           PARTITION BY category ORDER BY price DESC
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS least_expensive
FROM products
ORDER BY category, price DESC;
```

```
        product_name         |  category   | price  |
most_expensive        |     least_expensive
-----------------------------+-------------+--------
+-----------------------------+-------------------------
 Leather Wallet              | Accessories |  29.99 | Leather Wallet
| Leather Wallet
 SQL Cookbook                | Books       |  42.00 | SQL Cookbook
| Python Programming Book
 Python Programming Book     | Books       |  34.95 | SQL Cookbook
| Python Programming Book
 Denim Jeans                 | Clothing    |  39.99 | Denim Jeans
| Cotton T-Shirt
```

```
 Cotton T-Shirt               | Clothing    |  12.50 | Denim Jeans
| Cotton T-Shirt
 Noise Cancelling Headphones | Electronics | 129.99 | Noise Cancelling
Headphones | Wireless Mouse
 Mechanical Keyboard         | Electronics |  79.99 | Noise Cancelling
Headphones | Wireless Mouse
 USB-C Hub                   | Electronics |  24.99 | Noise Cancelling
Headphones | Wireless Mouse
 Wireless Mouse              | Electronics |  15.99 | Noise Cancelling
Headphones | Wireless Mouse
 Handwoven Basket            | Home        |  34.99 | Handwoven Basket
| Ceramic Mug Set
 Desk Lamp                   | Home        |  22.50 | Handwoven Basket
| Ceramic Mug Set
 Ceramic Mug Set             | Home        |  18.99 | Handwoven Basket
| Ceramic Mug Set
(12 rows)
```

Critical, commonly-missed detail about `LAST_VALUE`: by default, a window's **frame** (the subset of rows within the partition actually considered) is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` - meaning, without an explicit frame clause, `LAST_VALUE` would simply return the *current* row's own value every time (since "the last row up to and including the current one" is just the current one). The explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` frame above is what makes `LAST_VALUE` correctly consider the **entire** partition, giving the true least expensive product per category.

15.7 Aggregates as Window Functions: Running Totals and Moving Averages

Any aggregate function (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX`) can be used as a window function simply by adding `OVER(...)`.

Running total:

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', payment_date)::date AS month,
SUM(amount) AS revenue
    FROM payments GROUP BY DATE_TRUNC('month', payment_date)::date
)
SELECT month, revenue, SUM(revenue) OVER (ORDER BY month) AS
running_total
FROM monthly ORDER BY month;
```

```
   month    | revenue | running_total
------------+---------+---------------
 2025-01-01 |  192.44 |        192.44
 2025-02-01 |  212.47 |        404.91
 2025-03-01 |  429.34 |        834.25
 2025-04-01 |  153.96 |        988.21
(4 rows)
```

With `ORDER BY` but no explicit frame, `SUM() OVER (ORDER BY month)` defaults to "every row from the start of the partition up to and including the current row" - exactly what a running total needs.

Moving average (trailing 2 months):

```
SELECT month, revenue,
       ROUND(AVG(revenue) OVER (ORDER BY month ROWS BETWEEN 1
PRECEDING AND CURRENT ROW), 2) AS moving_avg_2mo
FROM monthly ORDER BY month;

   month    | revenue | moving_avg_2mo
------------+---------+----------------
 2025-01-01 |  192.44 |         192.44
 2025-02-01 |  212.47 |         202.46
 2025-03-01 |  429.34 |         320.91
 2025-04-01 |  153.96 |         291.65
(4 rows)
```

`ROWS BETWEEN 1 PRECEDING AND CURRENT ROW` explicitly limits the frame to "the previous row and the current row" - a genuine 2-month trailing window, rather than the running-total default.

Cumulative percentage of total:

```
SELECT month, revenue,
       ROUND(100.0 * SUM(revenue) OVER (ORDER BY month) / SUM(revenue)
OVER (), 1) AS cumulative_pct
FROM monthly ORDER BY month;

   month    | revenue | cumulative_pct
------------+---------+----------------
 2025-01-01 |  192.44 |           19.5
 2025-02-01 |  212.47 |           41.0
 2025-03-01 |  429.34 |           84.4
 2025-04-01 |  153.96 |          100.0
(4 rows)
```

`SUM(revenue) OVER ()` (no `ORDER BY`, no `PARTITION BY`) computes the grand total across every row - dividing the running total by this grand total, at every row, produces a running cumulative percentage.

15.8 Summary Table

Category	Functions
Ranking	`ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `NTILE(n)`
Navigation	`LAG(col, offset)`, `LEAD(col, offset)`, `FIRST_VALUE(col)`, `LAST_VALUE(col)`
Aggregates as windows	`SUM() OVER (...)`, `AVG() OVER (...)`,

Category	Functions
	`COUNT() OVER (...)`, `MIN()/MAX() OVER (...)`
Key clauses	`PARTITION BY` (independent groups), `ORDER BY` (defines sequence), frame clause (`ROWS/RANGE BETWEEN ...`)

Exercises - Section 15

Beginner 1. Rank all products by price (highest first) across the entire catalog (no `PARTITION BY`) using `RANK()`.

Intermediate 2. Compute each customer's running total spend over time, ordered by `order_date`, using a window function over `payments` joined to `orders`.

Advanced 3. Write a query showing, for each month, that month's revenue alongside its percentage growth compared to the previous month, using `LAG()` (hint: `(current - previous) / previous * 100`). # 16. Set Operations

16.1 `UNION` - Combining Results, Removing Duplicates

`UNION` stacks the results of two queries on top of each other, automatically removing duplicate rows.

```
SELECT city FROM customers WHERE country = 'Nigeria'
UNION
SELECT city FROM customers WHERE country = 'Kenya';

  city
---------
 Lagos
 Abuja
 Nairobi
(3 rows)
```

Even though two Nigerian customers share the city `'Lagos'`, it appears only **once** - `UNION` deduplicates the combined result set, exactly like `DISTINCT` (Part 7.5).

16.2 `UNION ALL` - Combining Results, Keeping Duplicates

```
SELECT country FROM customers WHERE country = 'Nigeria'
UNION ALL
SELECT country FROM customers WHERE country = 'Nigeria';

 country
---------
 Nigeria
 Nigeria
 Nigeria
 Nigeria
 Nigeria
```

```
 Nigeria
(6 rows)
```

`UNION ALL` keeps every row from both queries, including duplicates - with 3 Nigerian customers in each half, the combined result has all 6.

Performance note: `UNION ALL` is always faster than `UNION`, since it skips the deduplication step entirely. Always prefer `UNION ALL` unless removing duplicates is a genuine requirement of the query.

16.3 `INTERSECT` - Rows Present in Both Queries

```
SELECT o.customer_id
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id JOIN
products p ON p.product_id = oi.product_id
WHERE p.category = 'Books'

INTERSECT

SELECT o.customer_id
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id JOIN
products p ON p.product_id = oi.product_id
WHERE p.category = 'Home';

 customer_id
-------------
           3
(1 row)
```

Only customer 3 (Chidi Okafor) appears in **both** sets - having bought something from the `Books` category *and* something from the `Home` category. `INTERSECT` is a concise way to express "customers who did both X and Y," which would otherwise require a more verbose self-join or double `EXISTS`.

16.4 `EXCEPT` - Rows in the First Query, But Not the Second

```
SELECT customer_id FROM customers
EXCEPT
SELECT customer_id FROM orders;

 customer_id
-------------
          11
(1 row)
```

This returns every customer ID that exists in `customers` but does **not** appear anywhere in `orders` - customer 11 (David Chen), the same customer identified with `LEFT JOIN ... IS NULL` (Part 12.4) and `NOT EXISTS` (Part 13.6). `EXCEPT` is a fourth, equally valid way to express "find what's missing," often the most concise when comparing two simple column sets directly.

16.5 Column Compatibility

Every query combined with a set operation must return the **same number of columns**, with **compatible data types** in the same positions - column *names* don't need to match (the first query's column names are used for the combined result's labels).

```
SELECT name, 'customer' AS type FROM customers
UNION ALL
SELECT product_name, 'product' AS type FROM products
LIMIT 3;
```

```
     name     |   type
--------------+----------
 Ada Lovelace | customer
 Grace Hopper | customer
 Chidi Okafor | customer
(3 rows)
```

Both halves return exactly two columns, and both first columns are text - even though they come from entirely different tables (`customers.name` and `products.product_name`), which is perfectly valid.

16.6 Duplicate Handling and Performance Considerations

Operation	Removes Duplicates?	Relative Cost
`UNION`	Yes	Higher - requires sorting/hashing to deduplicate
`UNION ALL`	No	Lower - simple concatenation
`INTERSECT`	Yes (result is always a distinct set)	Similar to `UNION`
`EXCEPT`	Yes	Similar to `UNION`

For large datasets, always default to `UNION ALL` unless duplicates genuinely need removing - the deduplication step in `UNION/INTERSECT/EXCEPT` requires comparing every row against every other row (via sorting or hashing), which can be a meaningful cost on large result sets.

Exercises - Section 16

Beginner 1. Use `UNION` to combine the list of distinct product categories with the list of distinct payment methods into one column.

Intermediate 2. Use `EXCEPT` to find every product that has never appeared in any `order_items` row.

Advanced 3. Use `INTERSECT` to find every customer who has placed an order at both the `'Online Store'` and one physical (non-`NULL`-city) store, and explain why this dataset currently returns zero such customers. # 17. Views

17.1 What a View Is

A **view** is a named, saved SELECT query - queried exactly like a regular table, but with no data of its own. Every time a view is queried, PostgreSQL runs its underlying SELECT fresh against the current, live data.

17.2 Creating a View

```
CREATE VIEW customer_order_summary AS
SELECT c.customer_id, c.name, COUNT(o.order_id) AS total_orders,
COALESCE(SUM(p.amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN payments p ON p.order_id = o.order_id
GROUP BY c.customer_id, c.name;

CREATE VIEW

SELECT * FROM customer_order_summary ORDER BY total_spent DESC LIMIT
5;

 customer_id |     name      | total_orders | total_spent
-------------+---------------+--------------+-------------
           5 | John Smith    |            2 |      234.97
           1 | Ada Lovelace  |            3 |      215.45
           2 | Grace Hopper  |            2 |      175.97
           9 | Emily Davis   |            1 |       79.98
           7 | Kwabena Owusu |            1 |       69.90
(5 rows)
```

Once created, the view can be queried exactly like a table - filtered, sorted, joined, or aggregated further.

```
SELECT name FROM customer_order_summary WHERE total_orders = 0;

    name
------------
 David Chen
(1 row)
```

This is precisely the same underlying computation seen throughout Parts 12-16 (finding customers with no orders), now packaged behind a simple, memorable name.

17.3 Why Use a View

- **Encapsulation**: complex, multi-join, multi-CTE logic is written once, then reused everywhere via a simple name.
- **Consistency**: every report or application querying `customer_order_summary` gets the identical, correct definition of "total spent" - no risk of two teams computing it slightly differently.
- **Security** (Part 27): a view can expose only specific columns/rows of underlying tables, hiding sensitive data from users who only have access to the view.

17.4 Updating a View

`CREATE OR REPLACE VIEW` redefines a view's query without needing to drop it first (avoiding the need to redefine anything depending on it, as long as the output column set is compatible):

```
CREATE OR REPLACE VIEW customer_order_summary AS
SELECT c.customer_id, c.name, c.country, COUNT(o.order_id) AS
total_orders, COALESCE(SUM(p.amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN payments p ON p.order_id = o.order_id
GROUP BY c.customer_id, c.name, c.country;
```

Restriction: `CREATE OR REPLACE VIEW` can add new columns at the end, or change the query's logic, but cannot remove or reorder existing columns - doing so requires `DROP VIEW` followed by a fresh `CREATE VIEW`.

17.5 Dropping a View

```
DROP VIEW customer_order_summary;

DROP VIEW
```

17.6 Materialized Views

A **materialized view** is like a regular view, except its result is **physically stored** on disk at creation time, rather than recomputed on every query - trading freshness for speed.

```
CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT DATE_TRUNC('month', payment_date)::date AS month, SUM(amount)
AS revenue
FROM payments GROUP BY DATE_TRUNC('month', payment_date)::date;

SELECT * FROM monthly_revenue ORDER BY month;

   month    | revenue
------------+---------
 2025-01-01 |  192.44
 2025-02-01 |  212.47
 2025-03-01 |  429.34
 2025-04-01 |  153.96
(4 rows)
```

This data is now a physical snapshot - if new payments are added later, `monthly_revenue` will **not** automatically reflect them until explicitly refreshed:

```
REFRESH MATERIALIZED VIEW monthly_revenue;

REFRESH MATERIALIZED VIEW

DROP MATERIALIZED VIEW monthly_revenue;
```

```
DROP MATERIALIZED VIEW
```

17.7 Views vs Tables vs CTEs vs Materialized Views

Aspect	Table	View	CTE (`WITH`)	Materialized View
Stores data physically?	Yes	No - recomputed each query	No - exists only during one query	Yes - a physical snapshot
Always up to date?	N/A (it *is* the data)	Yes, always reflects live data	Yes, always reflects live data	No - only as of the last `REFRESH`
Reusable across queries?	Yes	Yes, by name, anywhere	No - scoped to a single query	Yes, by name, anywhere
Typical use	Storing real, primary data	Reusable, always-fresh logic	Readability within one complex query	Expensive aggregations that don't need real-time freshness

17.8 Analytics Use Cases for Views

- A `sales_dashboard_view` presenting pre-joined, pre-aggregated data exactly as a BI tool or dashboard needs it, hiding the underlying join complexity entirely from analysts.
- A `materialized view` refreshed nightly (via a scheduled job) for an expensive, company-wide revenue rollup that doesn't need to reflect the last few minutes of activity, dramatically speeding up dashboard load times.
- A restricted view exposing only non-sensitive columns of an `employees` table (e.g., `full_name`, `department_id`, but never `salary`) to a broader audience than the full table should be visible to (expanded in Part 27).

Exercises - Section 17

Beginner 1. Create a view `product_catalog` showing `product_name`, `category`, and `price` for all products priced under $50.

Intermediate 2. Create a materialized view summarizing total units sold per product, then insert a new `order_items` row and confirm the materialized view does *not* reflect it until `REFRESH MATERIALIZED VIEW` is run.

Advanced 3. Design a view that would be genuinely useful as a company dashboard's primary data source (combining at least three tables), and explain what makes it a good candidate for a materialized view versus a regular view, considering how often the dashboard is viewed versus how often the underlying data changes. # 18. Indexes

18.1 What an Index Is

An **index** is a separate, ordered data structure that lets PostgreSQL find rows matching a condition **without scanning every row in the table** - conceptually similar to a book's index, which lets you jump directly to a page instead of reading the entire book to find a topic.

18.2 Why Indexes Improve Queries

Without an index, PostgreSQL must perform a **sequential scan** - reading every single row and checking whether it matches. With a suitable index, PostgreSQL can instead perform an **index scan**, jumping almost directly to the matching row(s), which becomes dramatically faster as a table grows into the millions of rows.

18.3 Index Trade-offs

Indexes are not free: - They consume additional disk space. - They must be updated on every `INSERT/UPDATE/DELETE` affecting the indexed column(s), slowing down write operations. - They only help queries that actually filter, join, or sort on the indexed column(s).

The core trade-off: indexes trade slower writes and extra storage for faster reads on specific query patterns - which is exactly why not every column should be indexed, only the ones genuinely used in `WHERE`, `JOIN`, and `ORDER BY` clauses of important, frequent queries.

18.4 B-tree Indexes - The Default

```
CREATE INDEX idx_customers_email ON customers(email);

CREATE INDEX
```

B-tree is PostgreSQL's default index type, and the right choice for the overwhelming majority of cases: equality (`=`), ranges (`<`, `>`, `BETWEEN`), and sorting (`ORDER BY`) all use B-tree indexes efficiently.

18.5 A Crucial, Honest Lesson: Indexes Don't Always Get Used

```
EXPLAIN SELECT * FROM customers WHERE email = 'ada@example.com';

                        QUERY PLAN
-----------------------------------------------------------
 Seq Scan on customers  (cost=0.00..1.14 rows=1 width=136)
   Filter: (email = 'ada@example.com'::text)
(2 rows)
```

Even with `idx_customers_email` in place, PostgreSQL's planner chooses a **sequential scan** here - and it is genuinely correct to do so. With only 11 rows in `customers`, the entire table fits in a single disk page; reading it sequentially is just as fast as, or faster than, consulting an index and then fetching the matching row separately. **This is one of the most common points of confusion for learners: creating an index does not guarantee PostgreSQL will use it.** The planner (Part 19) always chooses whatever plan it estimates will be cheapest, and on small tables, that's almost always a sequential scan.

To see what an index scan actually looks like, forcing the planner's hand purely for demonstration:

```
SET enable_seqscan = off;
EXPLAIN SELECT * FROM customers WHERE email = 'ada@example.com';

                                        QUERY PLAN
-----------------------------------------------------------------------------
-------------------
 Index Scan using idx_customers_email on customers  (cost=0.14..8.15
rows=1 width=136)
   Index Cond: (email = 'ada@example.com'::text)
(2 rows)

RESET enable_seqscan;
```

On a real, production-sized table with thousands or millions of rows, PostgreSQL's planner would naturally choose this same index scan plan on its own, without any forcing - the trade-off tips decisively in the index's favor once sequentially scanning the whole table becomes genuinely expensive.

18.6 Hash Indexes

```
CREATE INDEX idx_orders_status_hash ON orders USING HASH (status);
```

Hash indexes support only equality (=) comparisons - not ranges or sorting - and are rarely chosen over B-tree in practice, since B-tree already handles equality efficiently *and* supports ranges/sorting too. Hash indexes are a narrow optimization, useful mainly for very specific high-volume equality-only lookup patterns.

18.7 GIN Indexes

GIN (Generalized Inverted Index) indexes are designed for columns containing **multiple values per row** - arrays, JSONB, and full-text search vectors.

```
CREATE TABLE gin_demo (id SERIAL, data JSONB);
CREATE INDEX idx_gin_demo ON gin_demo USING GIN (data);

CREATE TABLE
CREATE INDEX
```

GIN indexes are covered in full depth alongside JSONB itself in Part 22.

18.8 GiST Indexes

GiST (Generalized Search Tree) indexes support more exotic query types than simple equality/range - geometric data, full-text search, and range types (Part 22) all commonly use GiST. PostGIS (geospatial data) relies heavily on GiST indexes.

18.9 BRIN Indexes

BRIN (Block Range Index) indexes are extremely compact, storing only summary information (like min/max values) for large *ranges* of physical table pages, rather than a precise entry per row. They are ideal for very large tables where a column's values correlate strongly with physical storage order - the classic example being a `created_at` timestamp column on a huge, append-only log table, where rows are naturally inserted in roughly chronological (and thus physical) order.

18.10 Composite Indexes

An index spanning **multiple columns**, useful when queries frequently filter on the same combination of columns together.

```
CREATE INDEX idx_orders_customer_status ON orders(customer_id,
status);

CREATE INDEX
```

Column order matters enormously in a composite index. This index efficiently serves `WHERE customer_id = ...` and `WHERE customer_id = ... AND status = ...`, but is far less useful for a query that filters *only* on `status`, since the index is fundamentally organized first by `customer_id`.

18.11 Partial Indexes

An index covering only a **subset** of rows, matching a `WHERE` condition specified at index-creation time - smaller and faster than a full index, when queries consistently target the same narrow subset.

```
CREATE INDEX idx_orders_pending ON orders(order_id) WHERE status =
'pending';

CREATE INDEX
```

This index only contains entries for `pending` orders - ideal for an application that frequently needs to quickly find "what's still pending" out of a much larger table dominated by already-`completed` orders.

18.12 Expression Indexes

An index built on the **result of an expression or function**, rather than directly on a raw column - necessary when queries filter on a transformed version of a column.

```
CREATE INDEX idx_customers_lower_email ON customers(LOWER(email));

CREATE INDEX
```

This index only helps queries that filter using the *exact same expression*, e.g. `WHERE LOWER(email) = 'ada@example.com'` - it does **not** help a plain `WHERE email = ...` query, since that's a different expression entirely from PostgreSQL's perspective.

18.13 Unique Indexes

```
CREATE UNIQUE INDEX idx_products_name_unique ON
products(product_name);

CREATE INDEX
```

A `UNIQUE` constraint (Part 5.4) is, in fact, implemented internally as exactly this: a unique index. Creating a unique index directly (rather than via `ALTER TABLE ... ADD CONSTRAINT UNIQUE`) achieves the identical enforcement, and is occasionally used when finer control over the index itself (like making it partial) is needed alongside uniqueness.

18.14 Index Selectivity

Selectivity measures how effectively a column narrows down rows - a column like `customer_id` (nearly unique per row) is highly selective and benefits enormously from an index; a column like `status` with only 2-3 possible values across millions of rows is poorly selective, and an index on it alone often provides little benefit (though it can still help as part of a composite index, or with a partial index targeting one specific, rare value).

18.15 `EXPLAIN` and `EXPLAIN ANALYZE`

```
EXPLAIN ANALYZE SELECT * FROM customers WHERE email =
'ada@example.com';

                                                    QUERY PLAN
------------------------------------------------------------------------------
-------------------------------
 Seq Scan on customers  (cost=0.00..1.14 rows=1 width=136) (actual
time=0.007..0.009 rows=1 loops=1)
   Filter: (email = 'ada@example.com'::text)
   Rows Removed by Filter: 10
 Planning Time: 0.310 ms
 Execution Time: 0.029 ms
(5 rows)
```

Difference	`EXPLAIN`	`EXPLAIN ANALYZE`
Runs the query?	No - shows only the **estimated** plan	**Yes** - actually executes it, showing real timing
Shows actual row counts?	No, only estimates	Yes (`actual ... rows=1 loops=1`)
Safe on any query?	Yes, always safe	Use caution with `INSERT`/`UPDATE`/`DELETE` - it genuinely runs them (wrap in a transaction and `ROLLBACK` if just testing, per Part 20)

`cost=0.00..1.14` represents the planner's estimated cost (start-up cost .. total cost, in arbitrary planner units, not seconds) for this plan - used to compare *alternative* plans against each other, not as an absolute performance number.

Exercises - Section 18

Beginner 1. Create an index on `orders(order_date)` and use `EXPLAIN` to check whether the planner uses it for a date-range query on this course's small dataset.

Intermediate 2. Create a composite index on `order_items(order_id, product_id)` and explain, in your own words, which query shapes it would and wouldn't help.

Advanced 3. Design a partial index appropriate for a hypothetical `orders` table with 10 million rows, 95% of which are `'completed'`, where the application frequently needs to find `'pending'` and `'cancelled'` orders quickly - justify your choice using the selectivity concept from Section 18.14. # 19. Query Optimization

19.1 The Query Planner and Optimizer

Every SQL query PostgreSQL receives is first handed to the **query planner**, which considers multiple possible ways to execute it (which scan method per table, which join algorithm, which order to join tables in) and estimates the **cost** of each candidate plan, ultimately choosing the cheapest one it finds - this chosen plan is called the **execution plan**.

19.2 Reading `EXPLAIN` Output

```
EXPLAIN SELECT * FROM orders o JOIN customers c ON o.customer_id =
c.customer_id;

                                QUERY PLAN
-------------------------------------------------------------------------
----
 Hash Join  (cost=1.25..2.45 rows=15 width=188)
   Hash Cond: (o.customer_id = c.customer_id)
   ->  Seq Scan on orders o  (cost=0.00..1.15 rows=15 width=52)
   ->  Hash  (cost=1.11..1.11 rows=11 width=136)
         ->  Seq Scan on customers c  (cost=0.00..1.11 rows=11
width=136)
(5 rows)
```

Read from the **innermost, most-indented lines outward**: PostgreSQL first sequentially scans `orders` and `customers`, builds an in-memory hash table from the smaller side (`customers`), then probes it while scanning `orders` - this is a **Hash Join**, one of three join algorithms PostgreSQL can choose between.

19.3 Scan Types

Scan Type	What It Does	When the Planner Chooses It
Sequential Scan (`Seq Scan`)	Reads every row in the table, checking each against the filter	Small tables, or queries expected to match a large fraction of rows

Scan Type	What It Does	When the Planner Chooses It
Index Scan	Uses an index to locate matching rows directly, then fetches each from the table	A selective condition on an indexed column, on a large enough table
Bitmap Scan (`Bitmap Heap Scan` / `Bitmap Index Scan`)	Uses an index to build an in-memory "map" of matching page locations, then reads those pages efficiently in physical order	Matching a moderate number of scattered rows - a middle ground between a full index scan and a sequential scan

19.4 Join Algorithms

Join Type	How It Works	Best Suited For
Nested Loop	For each row in the outer table, scans (or index-probes) the inner table for matches	Small tables, or when one side is already filtered down to very few rows
Hash Join	Builds an in-memory hash table from one side, probes it while scanning the other	Larger tables with an equality join condition, as seen in Section 19.2
Merge Join	Both sides are sorted on the join key, then merged together in one pass	Both inputs are already sorted (e.g., via an index), or a sort is cheap to perform

19.5 Diagnosing Slow Queries - A Real, Verified Example

A very common, real-world performance mistake: applying a function to an indexed column inside `WHERE`, which prevents PostgreSQL from using the index at all.

Before optimization - filtering by casting a timestamp column to a date:

```
CREATE INDEX idx_orders_order_date ON orders(order_date);
SET enable_seqscan = off;   -- forces the planner's hand, purely to
prove no index plan exists
EXPLAIN SELECT * FROM orders WHERE order_date::date = '2025-03-01';

                                  QUERY PLAN
------------------------------------------------------------------------
---------
 Seq Scan on orders  (cost=10000000000.00..10000000001.23 rows=1
width=52)
   Filter: ((order_date)::date = '2025-03-01'::date)
```

Even with sequential scans explicitly disabled (note the artificially huge cost, `10000000000` - the planner's way of saying "I have no better option"), PostgreSQL is **still forced back to a sequential scan** - `order_date::date` is a different expression from the indexed `order_date` column itself, so `idx_orders_order_date` cannot help at all here.

After optimization - rewriting the same logical condition as a direct range comparison on the raw, indexed column:

```
EXPLAIN SELECT * FROM orders WHERE order_date >= '2025-03-01' AND
order_date < '2025-03-02';


QUERY PLAN
------------------------------------------------------------------------
------------------------------------------------------------------------
----------------
 Index Scan using idx_orders_order_date on orders  (cost=0.14..8.15
rows=1 width=52)
   Index Cond: ((order_date >= '2025-03-01 00:00:00+00'::timestamp
with time zone) AND (order_date < '2025-03-02 00:00:00+00'::timestamp
with time zone))

RESET enable_seqscan;
```

Same logical result (all orders placed on March 1st, 2025), but rewritten so the comparison is directly against the raw, indexed column - the planner now correctly finds and uses `idx_orders_order_date`. **The general lesson: avoid wrapping indexed columns in functions/casts within `WHERE` clauses; rewrite the condition so the raw column is compared directly wherever possible.**

19.6 A General Diagnostic Process for Slow Queries

1. Run `EXPLAIN ANALYZE` on the slow query (Part 18.15) - compare the planner's **estimated** row counts against the **actual** row counts; large discrepancies often indicate stale statistics (Part 28's `ANALYZE`).
2. Look for `Seq Scan` on large tables where an `Index Scan` would be expected - check whether a suitable index exists, and whether the query's `WHERE` clause is written in a way that can actually use it (Section 19.5).
3. Check whether `JOIN` conditions are on indexed columns on both sides.
4. Consider whether the query is asking for more data than it actually needs (unnecessary `SELECT *`, missing `LIMIT`).
5. Re-run `EXPLAIN ANALYZE` after each change, comparing costs and actual execution time, to confirm the change genuinely helped rather than just looking different.

Exercises - Section 19

Beginner 1. Run `EXPLAIN` on a simple `SELECT * FROM products;` and identify the scan type chosen.

Intermediate 2. Reproduce Section 19.5's "function on an indexed column" problem using a different column and function of your choice, and rewrite the query to avoid it.

Advanced 3. Using `EXPLAIN ANALYZE`, compare the estimated vs actual row counts for a query joining three of this course's tables, and explain what a large discrepancy between them would suggest about the table's statistics. # 20. Transactions

20.1 What a Transaction Is

A **transaction** groups multiple SQL statements into a single, all-or-nothing unit of work: either every statement in it succeeds and is made permanent together (`COMMIT`), or none of them are (`ROLLBACK`) - the database is never left in a state reflecting only *some* of the statements.

20.2 `BEGIN`, `COMMIT`, `ROLLBACK`

```
BEGIN;
UPDATE products SET stock_quantity = stock_quantity - 1 WHERE
product_id = 1;
SELECT stock_quantity FROM products WHERE product_id = 1;
COMMIT;

BEGIN
UPDATE 1
 stock_quantity
----------------
            199
(1 row)

COMMIT
```

The updated value (`199`) is visible *within* the transaction immediately, and becomes permanent once `COMMIT` runs.

```
BEGIN;
UPDATE products SET stock_quantity = 0 WHERE product_id = 1;
ROLLBACK;
SELECT stock_quantity FROM products WHERE product_id = 1;

BEGIN
UPDATE 1
ROLLBACK
 stock_quantity
----------------
            199
(1 row)
```

`ROLLBACK` completely undoes every change made since `BEGIN` - the stock quantity is still `199`, exactly as if the `UPDATE` had never happened. This is precisely the safety net referenced in Part 9.6 for safely testing risky `UPDATE`/`DELETE` statements before committing to them.

20.3 `SAVEPOINT` and `ROLLBACK TO`

A **savepoint** marks a point *within* a transaction that can be rolled back to, without discarding the entire transaction.

```
BEGIN;
UPDATE products SET stock_quantity = 100 WHERE product_id = 2;
SAVEPOINT sp1;
UPDATE products SET stock_quantity = -5 WHERE product_id = 3;
```

```
ROLLBACK TO sp1;
SELECT product_id, stock_quantity FROM products WHERE product_id IN
(2, 3);
COMMIT;
```

```
BEGIN
UPDATE 1
SAVEPOINT
UPDATE 1
ROLLBACK
 product_id | stock_quantity
------------+----------------
          3 |            150
          2 |            100
(2 rows)

COMMIT
```

The update to product 2 (`stock_quantity = 100`) survives and is committed; the update to product 3 (setting a nonsensical negative stock of `-5`) is undone by `ROLLBACK TO sp1`, while the transaction itself continues and is still ultimately committed. Savepoints are useful for handling a partial failure within a larger, multi-step transaction without discarding everything already done.

20.4 ACID - Precisely Defined

Property	Meaning
Atomicity	A transaction's statements happen entirely, or not at all - no partial completion
Consistency	A transaction moves the database from one valid state to another, never violating constraints (Part 5) along the way
Isolation	Concurrent transactions do not interfere with each other's intermediate, uncommitted state (Section 20.5)
Durability	Once committed, a transaction's changes survive even a subsequent crash - guaranteed by PostgreSQL's Write-Ahead Log (Part 28)

20.5 Transaction Isolation Levels

```
SHOW transaction_isolation;
```

```
 transaction_isolation
-----------------------
 read committed
(1 row)
```

`Read Committed` is PostgreSQL's default isolation level for every new transaction, unless specified otherwise.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
SHOW transaction_isolation;
COMMIT;

BEGIN
 transaction_isolation
-----------------------
 serializable
(1 row)

COMMIT
```

Isolation Level	Prevents	PostgreSQL Notes
Read Committed (default)	Dirty reads	Each statement sees only data committed before *that statement* began
Repeatable Read	Dirty reads, non-repeatable reads	The entire transaction sees a consistent snapshot from when it *itself* began
Serializable	Dirty reads, non-repeatable reads, phantom reads	Transactions behave as if executed one at a time, in some serial order - PostgreSQL enforces this by detecting and aborting conflicting transactions, requiring the application to retry

20.6 Dirty Reads, Non-Repeatable Reads, and Phantom Reads

Anomaly	Definition
Dirty read	Reading data written by another transaction that has **not yet committed** - if that other transaction rolls back, the data read was never actually real
Non-repeatable read	Reading the same row twice within one transaction and getting **different values**, because another transaction committed a change in between
Phantom read	Re-running the same query twice within one transaction and getting a **different set of rows**, because another transaction inserted or deleted matching rows in between

PostgreSQL, notably, **never permits dirty reads at any isolation level** - even its default `Read Committed` level already prevents them, which is stricter than the SQL standard technically requires at that level.

20.7 Deadlocks

A **deadlock** occurs when two transactions each hold a lock the other needs, and neither can proceed: Transaction A holds a lock on row 1 and waits for row 2; Transaction B holds a lock on row 2 and waits for row 1. PostgreSQL automatically **detects** deadlocks and resolves them by aborting one of the two transactions (returning an error to that transaction's client), rather than letting both hang forever.

20.8 Practical Guidance

- Keep transactions **short** - a long-running transaction holds locks longer, increasing contention with other concurrent work.
- Always handle the possibility that a transaction might fail (due to a constraint violation, a deadlock, or a serialization failure) and need to be retried by the application.
- Use explicit transactions (`BEGIN`/`COMMIT`) around any group of statements that must succeed or fail together - e.g., inserting an `order` and its `order_items` together, so a partial failure never leaves an order with no line items.

Exercises - Section 20

Beginner 1. Start a transaction, update a product's price, verify the change is visible within the transaction, then `ROLLBACK`, and confirm the original price is restored.

Intermediate 2. Write a transaction using two `SAVEPOINT`s, where the second savepoint's change is kept but the first is rolled back, and verify the final state.

Advanced 3. Explain, using Section 20.6's precise definitions, which specific anomaly (dirty read, non-repeatable read, or phantom read) would occur if Transaction A counts `orders WHERE status = 'pending'`, Transaction B inserts a new pending order and commits, and Transaction A then re-runs the identical count within the same transaction, at `Read Committed` isolation. # 21. Database Design

21.1 Entity Identification

Entities are the distinct "things" a database needs to track - in this course's schema, `Customer`, `Product`, `Order`, `Employee`, `Department`, `Supplier`, `Store` are all entities, each becoming a table. Identifying entities correctly is the first step of database design: ask "what are the nouns this business needs to keep records about?"

21.2 Attributes

Attributes are the properties of an entity - a `Customer` entity has attributes like name, email, city, country, signup date, each becoming a column.

21.3 Relationships and Cardinality

A **relationship** describes how two entities relate; **cardinality** describes *how many* of one entity relate to how many of another (Part 12.9):

Cardinality	Example	Modeled By
One-to-one	An employee has one optional parking-spot assignment	Foreign key with `UNIQUE` on the child
One-to-many	One customer places many orders	Foreign key on the "many" side
Many-to-many	Many orders contain many products	A junction table (`order_items`)

21.4 Primary and Foreign Keys (Recap)

Every entity needs a primary key (Part 5.2) to be uniquely identifiable; relationships between entities are expressed with foreign keys (Part 5.3) - this course's entire schema (Part 4) is a direct, worked example of applying both.

21.5 ER Diagrams (Entity-Relationship Diagrams)

An ER diagram visually represents entities, their attributes, and their relationships. A simplified text-based version of this course's schema:

```
DEPARTMENTS ---< EMPLOYEES >--- EMPLOYEES (self: manager_id)

CUSTOMERS ---< ORDERS >--- STORES
                  |
                  v
              ORDER_ITEMS >--- PRODUCTS >--- SUPPLIERS
                  |
ORDERS ---< PAYMENTS
```

(`---<` reads "one-to-many, one side to the many side.") Professional database design typically starts on a whiteboard or diagramming tool (e.g., a proper ER diagram with crow's-foot notation) before a single `CREATE TABLE` is written - Part 42's projects each begin with exactly this kind of design step.

21.6 Normalization - Why It Exists

Normalization is the systematic process of organizing tables to minimize data duplication and avoid update anomalies (where the same fact, stored in multiple places, can become inconsistent if only one copy is updated).

An Unnormalized Starting Point

```sql
CREATE TABLE unnormalized_orders (
    order_id INTEGER,
    customer_name TEXT,
    customer_email TEXT,
    product_names TEXT
);
INSERT INTO unnormalized_orders VALUES (1, 'Ada Lovelace',
```

```
'ada@example.com', 'Wireless Mouse, Cotton T-Shirt');
SELECT * FROM unnormalized_orders;

 order_id | customer_name | customer_email  |         product_names
----------+---------------+-----------------
+-------------------------------
        1 | Ada Lovelace  | ada@example.com | Wireless Mouse, Cotton
T-Shirt
```

This design has serious problems: `customer_name/customer_email` are duplicated on every order the same customer ever places (update Ada's email in one row and forget the others - now she has two different emails on file); `product_names` crams multiple values into one text field, making it essentially impossible to reliably query "how many orders included the Wireless Mouse?"

21.7 First Normal Form (1NF)

Rule: every column must hold a single, atomic value (no comma-separated lists, no repeating groups); every row must be uniquely identifiable.

Fix: split `product_names` into separate rows (this course's `order_items` table does exactly this - one row per product per order, rather than one crammed cell).

21.8 Second Normal Form (2NF)

Rule: must already be in 1NF, **and** every non-key column must depend on the **entire** primary key, not just part of it (only relevant when the primary key has multiple columns - a "composite key").

Example violation: a table with primary key `(order_id, product_id)` that also stores `customer_email` - `customer_email` depends only on `order_id` (via the order's customer), not on `product_id` at all, violating 2NF. **Fix:** move `customer_email` out to the `customers` table, exactly as this course's schema already does.

21.9 Third Normal Form (3NF)

Rule: must already be in 2NF, **and** no non-key column may depend on another **non-key** column (only on the primary key itself).

Example violation: an `orders` table storing both `customer_id` and `customer_country` - `customer_country` depends on `customer_id` (a non-key column in this table), not on `order_id` (the actual primary key) at all. **Fix:** `customer_country` belongs in `customers`, looked up via a join whenever needed - exactly this course's design, where `country` lives only in `customers`, never duplicated into `orders`.

21.10 Boyce-Codd Normal Form (BCNF)

A stricter version of 3NF: **every** determinant (a column, or set of columns, that determines another column's value) must itself be a candidate key. BCNF resolves rare edge cases 3NF can miss, typically involving tables with multiple overlapping candidate keys - encountered

occasionally in advanced design, but 3NF is sufficient for the vast majority of real-world schemas, including every table in this course.

21.11 Denormalization - When It May Be Useful

Denormalization deliberately reintroduces some duplication or precomputed data, trading normalization's data-integrity guarantees for query performance or simplicity.

Scenario	Why Denormalize
A materialized view (Part 17.6) storing precomputed monthly revenue	Avoids recomputing an expensive aggregation on every dashboard load
Storing a denormalized `order_total` column directly on `orders`	Avoids recalculating `SUM(order_items)` on every single read, at the cost of needing to keep it in sync (often via a trigger, Part 25)
A reporting/analytics warehouse (Part 35) using wide, denormalized fact tables	Analytical queries favor fewer joins over strict normalization

The rule of thumb: normalize first, by default - only denormalize deliberately, for a specific, measured performance reason, and with a clear plan for keeping the duplicated data consistent.

21.12 A Complete Database Design Project - Walkthrough

Business requirement: design a database for a small veterinary clinic tracking pet owners, their pets, appointments, and veterinarians.

Step 1 - Identify entities: `Owner`, `Pet`, `Veterinarian`, `Appointment`.

Step 2 - Identify relationships: one `Owner` has many `Pets` (one-to-many); one `Pet` has many `Appointments` (one-to-many); one `Veterinarian` has many `Appointments` (one-to-many).

Step 3 - Design tables (already normalized to 3NF):

```
CREATE TABLE owners (
    owner_id   SERIAL PRIMARY KEY,
    full_name  TEXT NOT NULL,
    phone      TEXT
);

CREATE TABLE veterinarians (
    vet_id     SERIAL PRIMARY KEY,
    full_name  TEXT NOT NULL,
    specialty  TEXT
);

CREATE TABLE pets (
    pet_id     SERIAL PRIMARY KEY,
    owner_id   INTEGER NOT NULL REFERENCES owners(owner_id),
    name       TEXT NOT NULL,
    species    TEXT NOT NULL,
```

```
    birth_date DATE
);

CREATE TABLE appointments (
    appointment_id SERIAL PRIMARY KEY,
    pet_id         INTEGER NOT NULL REFERENCES pets(pet_id),
    vet_id         INTEGER NOT NULL REFERENCES veterinarians(vet_id),
    scheduled_at   TIMESTAMPTZ NOT NULL,
    reason         TEXT
);

CREATE TABLE  (x4)
```

Every table has a clear primary key; every relationship is a straightforward foreign key; no column is duplicated across tables; `pets.owner_id` depends on the whole (single-column) key correctly, satisfying 1NF-3NF throughout.

Exercises - Section 21

Beginner 1. Identify the entities and relationships in a simple "school" domain (students, courses, enrollments), and sketch the tables.

Intermediate 2. Given a single denormalized table `student_id, student_name, course_name, course_credits, instructor_name`, identify which normal form it violates and redesign it into properly normalized tables.

Advanced 3. Design a complete, normalized schema (at least 4 tables) for a gym membership system tracking members, membership plans, class schedules, and class attendance, and justify one deliberate denormalization decision you might make for a reporting dashboard. # 22. Advanced PostgreSQL Data Types

22.1 `JSON` vs `JSONB`

Both store JSON documents; `JSON` stores the exact original text (preserving whitespace and key order, but re-parsing on every access), while `JSONB` stores an already-parsed, efficient **binary** representation - faster to query, and the only one of the two that can be indexed. **In almost all real applications, `JSONB` is the correct choice.**

22.2 Storing Semi-Structured Customer Profiles

```
CREATE TABLE customer_profiles (
    customer_id INTEGER PRIMARY KEY REFERENCES customers(customer_id),
    profile JSONB
);

INSERT INTO customer_profiles VALUES
    (1, '{"preferences": {"newsletter": true, "language": "en"},
"tags": ["vip", "early_adopter"]}'),
    (2, '{"preferences": {"newsletter": false, "language": "fr"},
"tags": ["new"]}');
```

```
CREATE TABLE
INSERT 0 2
```

22.3 JSONB Operators

```
SELECT customer_id, profile -> 'preferences' ->> 'language' AS lang
FROM customer_profiles;

 customer_id | lang
-------------+------
           1 | en
           2 | fr
(2 rows)
```

Operator	Returns	Example
`->`	A JSON value (still JSONB)	`profile -> 'preferences'`
`->>`	A JSON value **as text**	`profile ->> 'preferences'`
`#>`	A nested value by path (array of keys)	`profile #> '{preferences,language}'`
`@>`	"Contains" - does the left JSONB contain the right as a subset?	See below
`?`	Does a top-level key/array element exist?	See below

```
SELECT customer_id FROM customer_profiles WHERE profile @>
'{"preferences": {"newsletter": true}}';

 customer_id
-------------
           1
(1 row)

SELECT customer_id FROM customer_profiles WHERE profile -> 'tags' ?
'vip';

 customer_id
-------------
           1
(1 row)
```

22.4 Indexing JSONB

```
CREATE INDEX idx_profile_gin ON customer_profiles USING GIN (profile);

CREATE INDEX
```

A GIN index (Part 18.7) on a `JSONB` column accelerates `@>` (containment) and `?` (key existence) queries - exactly the operators shown above - making them efficient even on large tables full of semi-structured documents.

22.5 Arrays

```
CREATE TABLE product_tags (
    product_id INTEGER PRIMARY KEY REFERENCES products(product_id),
    tags TEXT[]
);

INSERT INTO product_tags VALUES (1, ARRAY['wireless', 'ergonomic']),
(2, ARRAY['mechanical', 'rgb']);

SELECT product_id, tags FROM product_tags;

 product_id |         tags
------------+----------------------
          1 | {wireless,ergonomic}
          2 | {mechanical,rgb}
(2 rows)

SELECT product_id FROM product_tags WHERE 'wireless' = ANY(tags);

 product_id
------------
          1
(1 row)
```

`= ANY(array_column)` is the idiomatic way to test "does this array contain this value?" - conceptually similar to `IN`, but for a single row's array column rather than a subquery's result set.

```
SELECT product_id, array_length(tags, 1) FROM product_tags;

 product_id | array_length
------------+--------------
          1 |            2
          2 |            2
(2 rows)
```

22.6 UUID

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
SELECT uuid_generate_v4() AS sample_uuid;

             sample_uuid
--------------------------------------
 dc635970-450e-4deb-a2bd-3ec751875a35
(1 row)
```

(Every generated UUID is different - this exact value will never repeat.)

```
CREATE TABLE sessions (
    session_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    customer_id INTEGER
);
INSERT INTO sessions (customer_id) VALUES (1) RETURNING session_id;

              session_id
--------------------------------------
 e015290f-ed6c-4e53-a877-14708707903b
(1 row)
```

UUID primary keys are especially useful in distributed systems, where multiple servers need to generate globally-unique IDs **independently**, without coordinating with each other (unlike a SERIAL/sequence-based ID, which requires a single, central counter).

22.7 ENUM

```
CREATE TYPE order_status AS ENUM ('pending', 'completed', 'cancelled',
'refunded');
CREATE TABLE order_status_demo (id SERIAL, status order_status);
INSERT INTO order_status_demo (status) VALUES ('pending'),
('completed');
SELECT * FROM order_status_demo;

 id |  status
----+-----------
  1 | pending
  2 | completed
(2 rows)
```

ENUM restricts a column to a fixed, named set of values, enforced at the database level - stricter and more self-documenting than a plain TEXT column with an informal convention, though it requires an ALTER TYPE to add new values later, making it best suited for genuinely stable, rarely-changing value sets.

22.8 Range Types

```
SELECT '[2025-01-01,2025-01-31]'::daterange AS january;

         january
-------------------------
 [2025-01-01,2025-02-01)
```

Note PostgreSQL automatically normalizes the inclusive upper bound `2025-01-31` (closed, `]`) into the equivalent canonical form using an exclusive upper bound `2025-02-01` (open, `)`) - `daterange` internally always represents its upper bound as exclusive.

```
SELECT '[2025-01-15,2025-02-15]'::daterange && '[2025-01-01,2025-01-
31]'::daterange AS overlaps;

 overlaps
----------
```

```
 t
(1 row)
```

&& tests whether two ranges overlap - directly useful for scheduling/booking systems checking for conflicting date ranges, without needing manually written `start1 <= end2 AND start2 <= end1`-style comparisons.

22.9 Network Address Types

```
SELECT '192.168.1.1'::inet AS ip, '192.168.1.0/24'::cidr AS subnet;
```

```
     ip      |     subnet
-------------+----------------
 192.168.1.1 | 192.168.1.0/24
(1 row)
```

`INET` stores a single IP address (optionally with a subnet); `CIDR` stores a network specification strictly (rejecting any address bits set beyond its declared subnet). These are purpose-built for network/security-related applications (e.g., logging request IP addresses, storing firewall rules), supporting natural operators like `<<` ("is contained within this subnet").

22.10 Real-World Example: API Response Storage

```
CREATE TABLE api_event_log (
    event_id   SERIAL PRIMARY KEY,
    event_type TEXT NOT NULL,
    payload    JSONB NOT NULL,
    received_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO api_event_log (event_type, payload) VALUES
    ('order.created', '{"order_id": 12, "total": 145.98, "items":
2}');

SELECT event_type, payload ->> 'order_id' AS order_id, (payload ->>
'total')::numeric AS total
FROM api_event_log;
```

```
  event_type   | order_id | total
---------------+----------+--------
 order.created | 12       | 145.98
(1 row)
```

Storing raw, variably-structured API payloads or event data as `JSONB` (rather than forcing every possible field into its own rigid column upfront) is a very common, pragmatic pattern for logging, webhooks, and event-driven systems - covered further from an AI-application perspective in Part 36.

Exercises - Section 22

Beginner 1. Create a table with a `JSONB` column storing a simple product's specifications, and query one nested field from it using `->>`.

Intermediate 2. Create a table with a `TEXT[]` column storing multiple email addresses per customer, and write a query finding all customers with more than one email using `array_length`.

Advanced 3. Design a schema using an `ENUM` for a limited, stable set of statuses and a `JSONB` column for flexible, evolving metadata on the same table, and explain why each column was given its specific data type. # 23. Advanced SQL Expressions

This section builds on `CASE`, `COALESCE`, and `NULLIF` already introduced in Part 10, focusing on more advanced analytical expressions - especially PostgreSQL's own extensions that go beyond the SQL standard.

23.1 Conditional Aggregation with `CASE`

Before PostgreSQL's `FILTER` clause existed (and still, for portability to other databases), conditional counts/sums are commonly expressed with `CASE` inside an aggregate:

```
SELECT
    SUM(CASE WHEN status = 'completed' THEN 1 ELSE 0 END) AS
completed_count,
    SUM(CASE WHEN status = 'pending' THEN 1 ELSE 0 END) AS
pending_count
FROM orders;

 completed_count | pending_count
-----------------+---------------
              13 |             1
(1 row)
```

23.2 The `FILTER` Clause - PostgreSQL's Cleaner Alternative

```
SELECT
    COUNT(*) FILTER (WHERE status = 'completed') AS completed_count,
    COUNT(*) FILTER (WHERE status = 'pending') AS pending_count,
    COUNT(*) FILTER (WHERE status = 'cancelled') AS cancelled_count
FROM orders;

 completed_count | pending_count | cancelled_count
-----------------+---------------+-----------------
              13 |             1 |               1
(1 row)
```

`FILTER (WHERE ...)` attaches a condition directly to a **specific aggregate function call**, applying only to that one aggregate - producing the identical result as the `CASE`-based version above, but considerably more readable, especially with several conditional aggregates in one query. `FILTER` is a PostgreSQL-supported syntax from the SQL standard that many other databases still lack, making it one of the clearest wins for analytical query readability covered in this course.

23.3 `DISTINCT` Inside Aggregate Functions

```
SELECT COUNT(DISTINCT category) AS distinct_categories,
COUNT(category) AS total_rows FROM products;
```

```
 distinct_categories | total_rows
---------------------+------------
                   5 |         12
(1 row)
```

`COUNT(DISTINCT column)` counts unique values only; plain `COUNT(column)` counts every non-`NULL` row - already introduced in Part 11.1, restated here because `DISTINCT` works inside `SUM`, `AVG`, and `STRING_AGG` too, not just `COUNT`.

23.4 String Aggregation with `STRING_AGG`

```
SELECT o.order_id, STRING_AGG(p.product_name, ', ' ORDER BY
p.product_name) AS products
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
GROUP BY o.order_id
ORDER BY o.order_id
LIMIT 3;
```

```
 order_id |            products
----------+---------------------------------
        1 | Cotton T-Shirt, Wireless Mouse
        2 | Mechanical Keyboard
        3 | Ceramic Mug Set, Leather Wallet
(3 rows)
```

`STRING_AGG(expression, separator ORDER BY ...)` concatenates values from multiple rows within a group into a single string - the aggregate-function equivalent of `CONCAT_WS` (Part 10.1) applied across rows instead of columns. This is extremely common in reporting: turning "many rows of line items" into "one readable summary cell" per order.

23.5 `DISTINCT ON` - A PostgreSQL-Specific Extension

```
SELECT DISTINCT ON (customer_id) customer_id, order_id, order_date
FROM orders
ORDER BY customer_id, order_date DESC
LIMIT 5;
```

```
 customer_id | order_id |       order_date
-------------+----------+------------------------
           1 |       12 | 2025-03-22 09:35:00+00
           2 |        8 | 2025-03-01 15:05:00+00
           3 |       10 | 2025-03-15 17:25:00+00
           4 |        5 | 2025-02-14 11:20:00+00
           5 |       15 | 2025-04-10 16:20:00+00
(5 rows)
```

`DISTINCT ON (column)` keeps exactly **one row per distinct value** of `column` - specifically, the *first* row for each, according to `ORDER BY`. Here, it returns each customer's **most recent** order (`ORDER BY customer_id, order_date DESC` means, within each customer, the newest order comes first, and `DISTINCT ON` keeps only that first row). This is a PostgreSQL-only feature (not standard SQL); the equivalent in standard SQL would require a window function (`ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) = 1`, Part 15.3) - `DISTINCT ON` is simply more concise for this exact "latest/first per group" pattern.

Critical rule: the leftmost expressions in `ORDER BY` **must** match the `DISTINCT ON (...)` columns exactly, in the same order - PostgreSQL enforces this, since `DISTINCT ON` needs to know which row is "first" within each distinct group.

23.6 Combining These Techniques

```
SELECT
    c.name,
    COUNT(*) FILTER (WHERE o.status = 'completed') AS
completed_orders,
    COUNT(DISTINCT o.store_id) AS stores_used,
    STRING_AGG(DISTINCT o.status, ', ') AS statuses_seen
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.name
ORDER BY completed_orders DESC
LIMIT 3;
```

```
     name     | completed_orders | stores_used |  statuses_seen
--------------+------------------+-------------+-----------------
 Ada Lovelace |                3 |           1 | completed
 Chidi Okafor |                1 |           1 | completed, pending
 Grace Hopper |                2 |           1 | completed
(3 rows)
```

`FILTER`, `COUNT(DISTINCT ...)`, and `STRING_AGG(DISTINCT ...)` combine naturally in a single query - each aggregate independently conditioned or deduplicated exactly as needed, producing a rich, single-row-per-customer summary directly from row-level data.

Exercises - Section 23

Beginner 1. Use `FILTER` to count how many products fall into `'Electronics'` versus every other category, in one query.

Intermediate 2. Use `STRING_AGG` to produce a comma-separated list of all product categories supplied by each supplier.

Advanced 3. Use `DISTINCT ON` to find, for every product category, the single cheapest product in that category, and compare your solution's readability against an equivalent window-function-based approach from Part 15. # 24. Stored Procedures and Functions

24.1 Why Database-Side Logic Exists

A **user-defined function** packages reusable logic directly inside the database, callable from any SQL query - useful for calculations, validations, or transformations needed consistently across many queries and applications.

24.2 A Simple Function

```
CREATE FUNCTION calculate_tax(price NUMERIC) RETURNS NUMERIC AS $$
BEGIN
    RETURN price * 0.075;
END;
$$ LANGUAGE plpgsql;

SELECT calculate_tax(100);
```

```
 calculate_tax
---------------
         7.500
```

Part	Explanation
`calculate_tax(price NUMERIC)`	The function's name and its single parameter, with its type
`RETURNS NUMERIC`	The function returns a single numeric value
`AS $$ ... $$`	**Dollar quoting** - PostgreSQL's way of delimiting a block of code (here, PL/pgSQL) without needing to escape single quotes inside it
`LANGUAGE plpgsql`	**PL/pgSQL** is PostgreSQL's own procedural language extension to SQL, adding variables, loops, and conditionals

24.3 PL/pgSQL - Variables, Conditionals, and Exception Handling

```
CREATE FUNCTION classify_salary(emp_id INTEGER) RETURNS TEXT AS $$
DECLARE
    sal NUMERIC;
    result TEXT;
BEGIN
    SELECT salary INTO sal FROM employees WHERE employee_id = emp_id;
    IF sal IS NULL THEN
        RAISE EXCEPTION 'Employee % not found', emp_id;
    ELSIF sal > 80000 THEN
        result := 'Senior';
    ELSIF sal > 55000 THEN
        result := 'Mid-level';
    ELSE
        result := 'Junior';
    END IF;
    RETURN result;
```

```
END;
$$ LANGUAGE plpgsql;

SELECT classify_salary(1) AS emp1, classify_salary(3) AS emp3;

  emp1  |  emp3
--------+--------
 Senior | Junior

SELECT classify_salary(999);

ERROR:  Employee 999 not found
CONTEXT:  PL/pgSQL function classify_salary(integer) line 8 at RAISE
```

Element	Explanation
`DECLARE ... BEGIN ... END;`	The standard PL/pgSQL block structure - variables are declared first, then the executable logic follows
`SELECT salary INTO sal FROM ...`	Runs a query and stores its single result directly into a variable
`IF ... ELSIF ... ELSE ... END IF;`	Standard conditional branching
`RAISE EXCEPTION 'message %', value`	Raises a genuine PostgreSQL error, with `%` substituted by the given value - exactly as seen for the missing employee 999

24.4 Loops

```
CREATE FUNCTION sum_to(n INTEGER) RETURNS INTEGER AS $$
DECLARE
    total INTEGER := 0;
    i INTEGER;
BEGIN
    FOR i IN 1..n LOOP
        total := total + i;
    END LOOP;
    RETURN total;
END;
$$ LANGUAGE plpgsql;

SELECT sum_to(5);

 sum_to
--------
     15
```

`FOR i IN 1..n LOOP ... END LOOP;` is PL/pgSQL's counting-loop syntax; PL/pgSQL also supports `WHILE` loops and looping directly over query results (`FOR row IN SELECT ... LOOP`).

24.5 RETURNS TABLE - Functions That Return Multiple Rows

```
CREATE FUNCTION top_products(n INTEGER) RETURNS TABLE(product_name
TEXT, total_sold NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT p.product_name, SUM(oi.quantity)::NUMERIC
    FROM products p JOIN order_items oi ON oi.product_id =
p.product_id
    GROUP BY p.product_name
    ORDER BY SUM(oi.quantity) DESC
    LIMIT n;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM top_products(3);
```

```
  product_name  | total_sold
----------------+------------
 Wireless Mouse |          4
 Cotton T-Shirt |          4
 Denim Jeans    |          3
(3 rows)
```

`RETURNS TABLE(...)` declares the function's output shape as a full result set (a "table"), and `RETURN QUERY` runs a query, streaming its rows directly out as the function's result - callable with `SELECT * FROM function_name(...)`, exactly like querying a real table or view.

24.6 Procedures (vs Functions)

A **procedure**, unlike a function, does not return a value, and - critically - can manage its own transactions (`COMMIT`/`ROLLBACK`) internally, which a function cannot do.

```
CREATE PROCEDURE give_raise(emp_id INTEGER, pct NUMERIC) AS $$
BEGIN
    UPDATE employees SET salary = salary * (1 + pct/100) WHERE
employee_id = emp_id;
    COMMIT;
END;
$$ LANGUAGE plpgsql;

CALL give_raise(9, 5);
SELECT employee_id, salary FROM employees WHERE employee_id = 9;
```

```
CALL
 employee_id | salary
-------------+---------
           9 | 50400.00
(1 row)
```

Procedures are invoked with `CALL`, not `SELECT` - and are the right tool specifically when a routine needs to control its own transaction boundaries (e.g., committing partial progress through a long multi-step batch job), which a function is not permitted to do.

24.7 When Database-Side Logic Should - and Should Not - Be Used

Use Database-Side Logic When	Avoid It When
The same calculation/validation must be enforced **consistently**, regardless of which application or script touches the data	The logic is purely about presentation/formatting, better suited to the application layer
The logic benefits from being co-located with the data (avoiding round-trips for a multi-step calculation)	The logic changes frequently - database code is typically harder to version, test, and deploy than application code
Enforcing a genuine data-integrity rule beyond what constraints (Part 5) can express	The team lacks strong SQL/PL-pgSQL expertise, risking hard-to-maintain, opaque business logic buried in the database

Exercises - Section 24

Beginner 1. Write a function `full_name_upper(customer_id INTEGER)` returning a customer's name in uppercase.

Intermediate 2. Write a function `orders_for_customer(cust_id INTEGER) RETURNS TABLE(order_id INTEGER, order_date TIMESTAMPTZ)` returning all orders for a given customer.

Advanced 3. Write a procedure `process_refund(order_id INTEGER)` that validates the order exists and is `'completed'` (raising an exception otherwise), then updates its status to `'refunded'` and commits - test both a successful and a failing call. # 25. Triggers

25.1 What Triggers Are

A **trigger** automatically executes a specified function whenever a given event (`INSERT`, `UPDATE`, `DELETE`) happens on a table - without any application code needing to explicitly request it. Triggers consist of two parts: a **trigger function** (written in PL/pgSQL, returning a special `TRIGGER` type) and the `CREATE TRIGGER` statement binding that function to a specific table and event.

25.2 `BEFORE` vs `AFTER` Triggers

Timing	Runs	Typical Use
`BEFORE`	Before the triggering operation is applied	Modifying the row about to be written (e.g., auto-setting a timestamp), or validating and aborting the operation entirely
`AFTER`	After the triggering operation has been applied	Side effects that shouldn't block the original operation -

Timing	Runs	Typical Use
		logging, notifications, cascading updates to other tables

25.3 Practical Example: Automatically Updating Timestamps

```
ALTER TABLE products ADD COLUMN updated_at TIMESTAMPTZ DEFAULT NOW();

CREATE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_products_updated_at
BEFORE UPDATE ON products
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

```
ALTER TABLE
CREATE FUNCTION
CREATE TRIGGER
```

```
UPDATE products SET price = 16.99 WHERE product_id = 1;
SELECT product_id, price, updated_at FROM products WHERE product_id =
1;
```

```
 product_id | price |          updated_at
------------+-------+-------------------------------
          1 | 16.99 | 2026-09-08 19:20:40.489883+00
(1 row)
```

Element	Explanation
`NEW`	A special variable, available inside row-level `BEFORE/AFTER INSERT/UPDATE` triggers, representing the row **as it will be written**
`OLD`	The row's values **before** the change - available in `UPDATE/DELETE` triggers
`NEW.updated_at = NOW();`	Because this runs `BEFORE UPDATE`, modifying `NEW` here changes what actually gets written to the table - no application code needed to remember to set this field itself
`FOR EACH ROW`	Runs the trigger function once per affected row (as opposed to `FOR EACH STATEMENT`, which runs it once total, regardless of how many rows were affected)

25.4 Practical Example: Audit Logging

```
CREATE TABLE audit_log (
    log_id SERIAL PRIMARY KEY,
    table_name TEXT,
    action TEXT,
    row_data JSONB,
    changed_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE FUNCTION audit_products() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log (table_name, action, row_data) VALUES
('products', 'DELETE', to_jsonb(OLD));
        RETURN OLD;
    ELSE
        INSERT INTO audit_log (table_name, action, row_data) VALUES
('products', TG_OP, to_jsonb(NEW));
        RETURN NEW;
    END IF;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_products
AFTER INSERT OR UPDATE OR DELETE ON products
FOR EACH ROW EXECUTE FUNCTION audit_products();

CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER

UPDATE products SET stock_quantity = 195 WHERE product_id = 1;
SELECT table_name, action, row_data ->> 'product_name' AS product FROM
audit_log;

UPDATE 1
 table_name | action |    product
------------+--------+----------------
 products   | UPDATE | Wireless Mouse
(1 row)
```

Element	Explanation
`TG_OP`	A special variable available inside every trigger function, holding the operation's name (`'INSERT'`, `'UPDATE'`, or `'DELETE'`) as text
`to_jsonb(OLD)` / `to_jsonb(NEW)`	Converts the entire row into a `JSONB` value (Part 22) - a convenient, schema-flexible way to store a full snapshot in the audit log,

Element	Explanation
	regardless of which specific columns changed
`AFTER INSERT OR UPDATE OR DELETE`	This single trigger fires for all three operations - `TG_OP` inside the function distinguishes which one actually happened

This audit trigger runs automatically for **every** change to `products`, from any application or script, with no possibility of forgetting to log a change - a key advantage over relying on application code to remember to write its own audit entries.

25.5 Validation with Triggers

```
CREATE FUNCTION prevent_negative_stock() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.stock_quantity < 0 THEN
        RAISE EXCEPTION 'Stock quantity cannot be negative for product
%', NEW.product_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

A trigger-based validation like this can express business rules too complex for a simple `CHECK` constraint (Part 5.6) - for example, rules that need to look up values in *other* tables, which `CHECK` constraints cannot do.

25.6 Change Tracking

The audit-log pattern (Section 25.4) is the standard foundation for **change tracking** - knowing not just the current state of data, but its complete history: who changed what, and when. This is essential for compliance-sensitive systems (financial records, healthcare data) where "we don't actually know what the value used to be" is not an acceptable answer.

25.7 The Risks of Excessive Trigger Usage

- **Hidden behavior**: a developer looking only at application code has no visibility into logic silently running inside database triggers - this can make debugging surprising, hard-to-trace behavior significantly harder.
- **Performance cost**: every trigger adds overhead to every affected `INSERT`/`UPDATE`/`DELETE`, particularly costly if a trigger performs expensive operations (extra joins, external calls) on every single row.
- **Cascading complexity**: a trigger that itself modifies another table can fire *that* table's own triggers, creating a chain of side effects that becomes very difficult to reason about as a system grows.
- **Best practice**: use triggers deliberately, for genuinely cross-cutting concerns (audit logging, timestamp maintenance, enforcing integrity that constraints can't express) - not as a general-purpose place to bury core business logic that would be clearer and more testable in application code.

Exercises - Section 25

Beginner 1. Add an `updated_at` column and trigger to the `customers` table, following Section 25.3's pattern.

Intermediate 2. Extend the audit log trigger (Section 25.4) to also track changes on the `orders` table, and verify both tables' changes appear correctly labeled in `audit_log`.

Advanced 3. Write a trigger that prevents an order's status from being changed directly from `'pending'` to `'refunded'` (skipping `'completed'`), raising a clear exception if attempted, and test both a valid and an invalid status transition. # 26. Temporary Tables

26.1 What Temporary Tables Are

A **temporary table** behaves exactly like a regular table (supports indexes, constraints, and full SQL) but is automatically dropped at the end of the session (or, optionally, at the end of the current transaction) that created it - and is completely invisible to every other session.

26.2 Creating and Using a Temporary Table

```
CREATE TEMPORARY TABLE temp_high_value_customers AS
SELECT c.customer_id, c.name, SUM(p.amount) AS total_spent
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN payments p ON p.order_id = o.order_id
GROUP BY c.customer_id, c.name
HAVING SUM(p.amount) > 100;

SELECT * FROM temp_high_value_customers ORDER BY total_spent DESC;
```

```
 customer_id |     name     | total_spent
-------------+--------------+-------------
           5 | John Smith   |      234.97
           1 | Ada Lovelace |      215.45
           2 | Grace Hopper |      175.97
(3 rows)
```

26.3 Temporary Table Lifecycle

```
-- In a brand-new session/connection:
SELECT * FROM temp_high_value_customers;

ERROR:  relation "temp_high_value_customers" does not exist
LINE 1: SELECT * FROM temp_high_value_customers;
```

The temporary table created in one session is completely gone (and was never even visible) from any other session - confirming its defining characteristic: session-scoped, private, automatically cleaned up.

Lifecycle Option	Behavior
`CREATE TEMPORARY TABLE ...` (default)	Dropped automatically at the end of the current session

Lifecycle Option	Behavior
`CREATE TEMPORARY TABLE ... ON COMMIT DROP`	Dropped automatically at the end of the current transaction, even if the session continues
`CREATE TEMPORARY TABLE ... ON COMMIT DELETE ROWS`	Table definition persists for the session, but its rows are cleared after every transaction commit

26.4 Temporary Tables vs CTEs

Aspect	Temporary Table	CTE (`WITH`)
Scope	Entire session (or transaction) - reusable across many separate queries	A single query only
Can be indexed?	Yes	No
Materialized (computed once, stored)?	Yes, always	Depends on the planner (Part 14.7)
Best suited for	Multi-step analysis needing the same intermediate result repeatedly, or a result large/expensive enough to benefit from its own index	A single query's internal readability/structure

26.5 Temporary Tables vs Permanent Tables

Aspect	Temporary Table	Permanent Table
Visibility	Only the creating session	Every session with appropriate permissions (Part 27)
Persistence	Dropped automatically (Section 26.3)	Persists until explicitly dropped
Typical use	Scratch space for a multi-step analysis or ETL process (Part 35)	Storing real, durable business data

26.6 Practical Analytics Use Cases

- **Staging intermediate results** in a multi-step analytical script - computing an expensive intermediate aggregation once into a temp table, then running several different follow-up queries against it without recomputing the aggregation each time.
- **ETL/data pipeline staging** (Part 35) - loading raw data into a temporary table first, validating and transforming it, and only inserting the cleaned result into a permanent table.
- **Ad-hoc exploratory analysis** in a single analyst's session, without cluttering the shared database with permanent scratch tables other users would see.

```sql
-- Example: a multi-step analysis session using a temp table as
scratch space
CREATE TEMPORARY TABLE temp_monthly_stats AS
SELECT DATE_TRUNC('month', payment_date)::date AS month, SUM(amount)
AS revenue, COUNT(*) AS payment_count
FROM payments
GROUP BY DATE_TRUNC('month', payment_date)::date;

-- Now run several different follow-up analyses against the SAME
precomputed result
SELECT * FROM temp_monthly_stats ORDER BY revenue DESC LIMIT 1;   --
best month
SELECT ROUND(AVG(revenue), 2) FROM temp_monthly_stats;              --
average monthly revenue
```

Exercises - Section 26

Beginner 1. Create a temporary table summarizing total stock quantity per product category, then query it twice with two different `ORDER BY` clauses.

Intermediate 2. Explain, in your own words, a scenario from your own analysis work where a temporary table would be preferable to repeating the same expensive subquery/CTE multiple times.

Advanced 3. Create a temporary table with `ON COMMIT DROP` inside an explicit transaction, and verify it is gone immediately after `COMMIT` - even within the *same* session, unlike the default behavior in Section 26.3. # 27. Security and Users

27.1 Users, Roles, and Groups - Precisely Defined

In PostgreSQL, **there is no separate concept of "users" versus "roles" internally** - `CREATE USER` is simply shorthand for `CREATE ROLE ... WITH LOGIN`. A **role** can represent an individual person (able to log in) or a **group** (typically without login, used purely to bundle permissions that are then granted to individual login roles).

```sql
CREATE ROLE analyst LOGIN PASSWORD 'secure_pass_123';
CREATE ROLE app_readonly;
GRANT app_readonly TO analyst;
```

```
CREATE ROLE
CREATE ROLE
GRANT ROLE
```

`analyst` is a genuine login role (a "user," in older terminology); `app_readonly` is a group role with no login capability of its own, existing purely as a reusable bundle of permissions - `GRANT app_readonly TO analyst` makes `analyst` inherit everything granted to `app_readonly`.

27.2 GRANT and REVOKE - Table Privileges

```
GRANT SELECT ON customers, orders, products TO app_readonly;
GRANT SELECT, INSERT, UPDATE ON orders TO analyst;
REVOKE INSERT ON orders FROM analyst;

GRANT
GRANT
REVOKE
```

GRANT adds a specific privilege; REVOKE removes one - here, `analyst` ends up with SELECT (via `app_readonly`) plus SELECT/UPDATE directly on `orders` (the INSERT privilege granted, then explicitly revoked again).

Privilege	Allows
SELECT	Reading rows
INSERT	Adding new rows
UPDATE	Modifying existing rows
DELETE	Removing rows
TRUNCATE	Using TRUNCATE (Part 4.7)
REFERENCES	Creating foreign keys referencing this table
ALL PRIVILEGES	Every applicable privilege at once

27.3 Schema Permissions

```
GRANT USAGE ON SCHEMA public TO app_readonly;

GRANT
```

USAGE on a schema allows a role to "see into" it at all - without it, even a role with SELECT granted on a specific table inside that schema cannot actually reach it. Schema-level USAGE and table-level privileges (Section 27.2) work together, both required.

27.4 Column-Level Permissions

```
GRANT SELECT (employee_id, full_name, department_id) ON employees TO
app_readonly;

GRANT
```

This grants read access to **only** the listed columns - `employee_id`, `full_name`, `department_id` - explicitly excluding `salary` and `manager_id`. Verified directly:

```
-- Connected as 'analyst' (who inherits app_readonly's permissions):
SELECT salary FROM employees LIMIT 1;

ERROR:  permission denied for table employees

SELECT full_name FROM employees LIMIT 1;
```

```
  full_name
-------------
 Efe Johnson
(1 row)
```

Column-level privileges let sensitive columns (like `salary`) remain protected, even for roles that otherwise need broad read access to a table for legitimate reasons.

27.5 Verifying Table-Level Enforcement

```
-- Connected as 'analyst':
SELECT name FROM customers LIMIT 2;

     name
--------------
 Ada Lovelace
 Grace Hopper
(2 rows)

DELETE FROM orders WHERE order_id = 1;

ERROR:  permission denied for table orders
```

`analyst` can read `customers` (via the `app_readonly` group role) and read/update `orders` directly, but was never granted `DELETE` on anything - and PostgreSQL correctly, immediately blocks the attempt.

27.6 Least Privilege

The principle of least privilege: every role should have the **minimum** set of permissions genuinely required for its purpose - nothing more. `analyst` above illustrates this precisely: read access broadly (via a shared group role), write access only where specifically needed (`orders`), and no access at all to sensitive columns (`employees.salary`) or destructive operations (`DELETE`) it has no legitimate need for.

27.7 Authentication vs Authorization

Concept	Question It Answers	PostgreSQL Mechanism
Authentication	"Are you who you claim to be?"	Password verification, `pg_hba.conf` connection rules, certificate-based auth
Authorization	"What are you allowed to do, now that we know who you are?"	`GRANT`/`REVOKE`, role memberships

27.8 Secure Database Access - Practical Guidance

- **Never share a single superuser account** across an application and multiple developers - create distinct roles per application/service, each with only the privileges it needs.

- **Use strong, unique passwords** for every role, and prefer certificate or identity-provider-based authentication over plain passwords in production where possible.
- **Grant privileges to group roles, not individual users directly**, then grant group roles to individuals - Section 27.1's pattern - so permission changes only need to happen in one place as team membership changes.
- **Regularly audit** which roles have which privileges (`\du` in `psql`, or querying `information_schema.role_table_grants`), since permissions tend to accumulate and rarely get cleaned up automatically.

Exercises - Section 27

Beginner 1. Create a role `report_viewer` with `SELECT`-only access to `products` and `orders`, and verify it cannot `INSERT` into either.

Intermediate 2. Create a group role `sales_team`, grant it appropriate privileges on `customers` and `orders`, then create two login roles and grant both membership in `sales_team`.

Advanced 3. Design a column-level permission scheme for the `employees` table appropriate for three different roles: an HR administrator (full access), a payroll processor (access to `salary` but not personal details), and a general employee directory viewer (name and department only, no salary). # 28. PostgreSQL Administration Concepts

This section introduces administration concepts at a level appropriate for a data/SQL professional working alongside a database, not a full database-administrator (DBA) training - understanding what these mechanisms do, and why they matter, is valuable even without configuring them directly.

28.1 PostgreSQL Server Processes

PostgreSQL runs as a **postmaster** process that listens for connections, spawning a separate **backend process** for each connected client - meaning every session (Part 26) genuinely runs as its own OS process, providing strong isolation between concurrent connections.

28.2 Configuration

PostgreSQL's behavior is controlled by settings in `postgresql.conf` (and connection rules in `pg_hba.conf`), covering everything from memory allocation to logging verbosity. Many settings can also be inspected (and some changed) at runtime:

```
SHOW max_connections;

 max_connections
-----------------
             100
(1 row)
```

28.3 Connections

Every connected client consumes one backend process and a slice of `max_connections`. Exhausting this limit causes new connection attempts to fail - a common production issue mitigated by **connection pooling** (introduced from the application side in Part 31.9, and via tools like PgBouncer at the infrastructure level).

```
SELECT count(*) FROM pg_stat_activity;
```

```
 count
-------
     6
(1 row)
```

`pg_stat_activity` is a system view showing every currently active connection/backend, useful for diagnosing connection exhaustion or identifying long-running queries.

28.4 Memory Concepts

```
SHOW shared_buffers;
SHOW work_mem;
```

```
 shared_buffers
----------------
 128MB

 work_mem
----------
 4MB
```

Setting	Purpose
`shared_buffers`	The main cache of data pages PostgreSQL keeps in memory, shared across all connections
`work_mem`	Memory available **per query operation** (a sort, a hash join) before spilling to disk - too low, and complex queries slow down significantly by spilling unnecessarily; too high, and many concurrent queries can exhaust server memory

28.5 Storage

PostgreSQL stores each table and index as one or more files on disk, organized into fixed-size **pages** (8KB by default) - the fundamental unit of I/O the server reads and writes.

```
SELECT pg_size_pretty(pg_database_size('company_db'));
```

```
 pg_size_pretty
----------------
 8327 kB
```

28.6 The Write-Ahead Log (WAL)

Every change to the database is first recorded in the **Write-Ahead Log** *before* being applied to the actual data files - this is what makes **durability** (Part 20.4) possible: if the server crashes mid-operation, the WAL can be replayed on restart to recover any committed changes that hadn't yet been fully written to the main data files.

```
SELECT pg_current_wal_lsn();

 pg_current_wal_lsn
---------------------
 0/353F108
```

This "Log Sequence Number" identifies the current position in the WAL stream - the same underlying mechanism that powers PostgreSQL's physical replication and point-in-time recovery capabilities (Part 29).

28.7 Checkpoints

A **checkpoint** is the point at which PostgreSQL guarantees all changes recorded in the WAL up to that point have also been safely written to the actual data files on disk - allowing older WAL segments to eventually be safely removed/recycled, and bounding how much WAL would need replaying after a crash.

28.8 `VACUUM` and `Autovacuum`

PostgreSQL uses an MVCC (Multi-Version Concurrency Control) model: an `UPDATE` or `DELETE` doesn't immediately erase old row versions - it marks them as no longer current, but the physical space isn't reclaimed until `VACUUM` runs.

```
VACUUM ANALYZE products;
SELECT relname, n_live_tup, n_dead_tup, last_vacuum FROM
pg_stat_user_tables WHERE relname = 'products';

VACUUM
 relname  | n_live_tup | n_dead_tup |          last_vacuum
----------+------------+------------+-------------------------------
 products |         12 |          0 | 2026-09-09 11:34:33.717266+00
(1 row)
```

`n_dead_tup` (dead tuples - obsolete row versions awaiting cleanup) is `0` here since this table has had relatively few updates; a heavily-updated table without regular vacuuming can accumulate large numbers of dead tuples, wasting space and slowing down queries that must skip over them.

`Autovacuum` is a background process that runs `VACUUM` (and `ANALYZE`) automatically once a table's dead-tuple count crosses a configured threshold - in modern PostgreSQL, autovacuum is

enabled by default and handles the vast majority of routine vacuuming needs without any manual intervention.

28.9 `ANALYZE` and Statistics

`ANALYZE` (distinct from `EXPLAIN ANALYZE`, Part 18.15) collects statistics about a table's data distribution - how many distinct values a column has, how values are distributed - which the query planner (Part 19) relies on to estimate costs and choose good execution plans. Stale statistics (e.g., after a large bulk load) can cause the planner to make poor choices; running `ANALYZE` (or `VACUUM ANALYZE`, as above) refreshes them.

28.10 Database Maintenance - A Practical Summary

Task	Purpose	Typically Handled By
`VACUUM`	Reclaim space from dead tuples	Autovacuum (automatic, by default)
`ANALYZE`	Keep planner statistics current	Autovacuum (automatic, by default)
Monitoring `pg_stat_activity`	Spot long-running or stuck queries	Manual/periodic DBA review
Monitoring database/table size growth	Capacity planning	Manual/periodic DBA review
Backups (Part 29)	Disaster recovery	Scheduled jobs

Exercises - Section 28

Beginner 1. Run `SHOW` on three different PostgreSQL configuration settings of your choice and look up what each controls.

Intermediate 2. Query `pg_stat_user_tables` for every table in the course database and identify which has the highest `n_live_tup`.

Advanced 3. Explain, in your own words, why MVCC (Section 28.8) requires a background cleanup process like autovacuum at all - what would happen to a heavily-updated table if dead tuples were never reclaimed? # 29. Backup and Restore

29.1 Why Backups Are Critical

No database is immune to hardware failure, accidental mass deletion (Part 9.5), a bad migration, or human error. A backup strategy is what turns any of these from "catastrophic, unrecoverable data loss" into "an inconvenience with a known recovery procedure." Backups are not optional for any system holding data anyone actually cares about.

29.2 `pg_dump` - Logical Backups

`pg_dump` exports a database's schema and data as a portable file, either as plain SQL or a compact custom binary format.

```
pg_dump -h localhost -U postgres -d company_db -F c -f
company_db_backup.dump

ls -la company_db_backup.dump

-rw-r--r-- 1 root root 26002 Sep  9 11:35 company_db_backup.dump
```

Flag	Meaning
`-h`, `-U`, `-d`	Host, username, database (same as `psql`, Part 2.5)
`-F c`	Custom format - compressed, and required for `pg_restore`'s more advanced features (parallel restore, selective restore)
`-F p`	Plain SQL format - a human-readable `.sql` script, restorable with plain `psql`
`-f`	Output file path

29.3 Plain SQL Dumps

```
pg_dump -h localhost -U postgres -d company_db -F p -f
company_db_backup.sql
wc -l company_db_backup.sql

821 company_db_backup.sql
```

A plain SQL dump is simply a script of `CREATE TABLE`, `COPY`/`INSERT`, and constraint statements that reconstructs the database when run - inspectable in a text editor, and restorable with nothing more than `psql -f company_db_backup.sql`, at the cost of losing the custom format's compression and advanced restore options.

29.4 `pg_restore` - Restoring from a Custom-Format Dump

```
createdb -h localhost -U postgres company_db_restore_test
pg_restore -h localhost -U postgres -d company_db_restore_test
company_db_backup.dump

SELECT COUNT(*) FROM customers;

 count
-------
    11
(1 row)
```

The restored database contains the exact, complete dataset from Part 6 - confirming the dump/restore round trip preserves everything correctly.

29.5 Restoring a Plain SQL Dump

```
createdb -h localhost -U postgres another_restore_test
psql -h localhost -U postgres -d another_restore_test -f
company_db_backup.sql
```

Plain SQL dumps restore via ordinary `psql` execution, statement by statement - simpler conceptually, but without `pg_restore`'s ability to restore selectively (e.g., only specific tables) or in parallel.

29.6 Backup Strategies

Strategy	Description	Trade-off
Logical backup (`pg_dump`)	Exports data as portable SQL/custom-format files	Simple, portable across PostgreSQL versions; slower for very large databases
Physical backup (file-system-level, e.g., `pg_basebackup`)	Copies the actual data files directly	Faster for large databases; tied to the exact PostgreSQL version/architecture
Continuous archiving (WAL archiving)	Continuously archives the Write-Ahead Log (Part 28.6), enabling point-in-time recovery	Allows restoring to any specific moment, not just the last full backup

29.7 Restore Strategies

- **Full restore**: rebuilding an entire database from a single backup file - the simplest case, shown above.
- **Selective restore**: `pg_restore` (with a custom-format dump) can restore only specific tables (`--table=tablename`), useful when only one table needs recovering, not the whole database.
- **Point-in-time recovery**: combining a physical base backup with archived WAL segments to restore a database to its exact state at any specific past moment - essential for recovering from "someone ran a bad `DELETE` at 2:47pm" scenarios precisely, without losing all changes made after the last nightly backup.

29.8 Backup Best Practices

- **Automate backups** on a regular schedule - never rely on remembering to run them manually.
- **Test restores regularly** - a backup that has never actually been restored successfully is an unverified assumption, not a guarantee.
- **Store backups off-server** (a different machine, cloud storage) - a backup sitting on the same disk as the database it protects doesn't protect against hardware failure of that disk.
- **Keep multiple backup generations**, not just the most recent one - protects against a corrupted backup, or discovering a problem that occurred several backups ago.
- **Document the restore procedure** clearly, in advance - the middle of an actual incident is the worst time to be figuring out the correct `pg_restore` flags for the first time.

Exercises - Section 29

Beginner 1. Create a `pg_dump` custom-format backup of the course database and confirm the file was created successfully.

Intermediate 2. Restore your backup into a newly created, differently-named database, and verify the row counts match the original for at least three tables.

Advanced 3. Research (via PostgreSQL's documentation) how `pg_basebackup` and WAL archiving combine to enable point-in-time recovery, and summarize in your own words the general procedure for restoring a database to "5 minutes before an accidental mass delete." # 30. Importing and Exporting Data

30.1 `\copy` - Client-Side Data Transfer

`\copy` is a `psql` meta-command (Part 2.7) that transfers data between a table and a file **on the client machine** - the file never needs to be accessible to the PostgreSQL server itself, making it the right choice for almost all everyday import/export work.

Exporting to CSV

```
\copy (SELECT * FROM products ORDER BY product_id) TO
'products_export.csv' WITH (FORMAT csv, HEADER true)

COPY 12

product_id,product_name,category,price,stock_quantity,supplier_id
1,Wireless Mouse,Electronics,15.99,200,1
2,Mechanical Keyboard,Electronics,79.99,80,1
3,USB-C Hub,Electronics,24.99,150,2
4,Noise Cancelling Headphones,Electronics,129.99,60,1
```

Importing from CSV

```
new_suppliers.csv contents:
supplier_name,country
Nordic Traders,Sweden
Pacific Imports,Japan

\copy suppliers(supplier_name, country) FROM 'new_suppliers.csv' WITH
(FORMAT csv, HEADER true)

COPY 2

SELECT * FROM suppliers WHERE supplier_name IN ('Nordic Traders',
'Pacific Imports');

 supplier_id |  supplier_name  | country
-------------+-----------------+---------
           5 | Nordic Traders  | Sweden
           6 | Pacific Imports | Japan
(2 rows)
```

Specifying the column list (`suppliers(supplier_name, country)`) tells PostgreSQL exactly which columns the CSV's columns map to, in order - `supplier_id` is correctly left to its `SERIAL` default rather than being expected in the file at all.

30.2 `COPY` - Server-Side Data Transfer

Plain SQL `COPY` (no backslash, and usable from any client, not just `psql`) reads/writes a file that must be accessible **on the server's own filesystem** - typically requiring superuser or specifically-granted privileges, since it interacts directly with the server's disk.

```
COPY products TO '/tmp/products_server_side.csv' WITH (FORMAT csv,
HEADER true);

COPY 12
```

Aspect	`\copy` (psql meta-command)	`COPY` (SQL command)
File location	Client machine	Server machine
Requires elevated privileges?	No (works with normal `SELECT`/`INSERT` privileges)	Often yes (server filesystem access)
Usable from	`psql` only	Any SQL client/driver
Typical use	Everyday import/export from your own machine	Server-side automation, scripts running directly on the database host

30.3 Importing JSON

```
CREATE TEMPORARY TABLE json_staging (raw_data JSONB);
\copy json_staging FROM 'events.json' WITH (FORMAT text)
-- then parse/expand raw_data as needed using JSONB operators (Part
22.3)
```

For structured JSON arrays of records, a common pattern is staging the raw JSON into a single-column table first, then using `jsonb_array_elements` and JSONB operators to expand it into proper relational rows - covered further in the ETL context of Part 35.

30.4 Excel Integration Through External Tools

PostgreSQL has no native Excel format support - the standard workflow is: export from Excel to CSV first, then `\copy` the CSV in (Section 30.1); conversely, export query results to CSV and open that CSV in Excel. Tools like `pandas` (Part 32) can also read/write `.xlsx` directly, often making a Python script a more convenient bridge than manual CSV round-tripping for recurring workflows.

30.5 Bulk Loading Considerations

For loading very large volumes of data efficiently: - `COPY`/`\copy` is dramatically faster than many individual `INSERT` statements - it is the standard, correct tool for bulk loading in PostgreSQL. - Consider temporarily dropping non-essential indexes before a very large bulk load, then recreating them afterward - maintaining indexes during the load itself adds

significant overhead per row. - Wrap a large load in a single transaction where appropriate, so a failure partway through can be cleanly rolled back rather than leaving a partially-loaded table.

30.6 Data Migration Concepts

Data migration is the process of moving data from one system (an old database, a legacy application, a different RDBMS) into PostgreSQL. A typical migration pipeline: extract data from the source system (often to CSV or JSON as an intermediate format), transform it to match the target schema (data types, renamed columns, cleaned values), then load it via `COPY`/`\copy` - the same **ETL** pattern covered in full depth in Part 35.

Exercises - Section 30

Beginner 1. Export the `customers` table to a CSV file using `\copy`, then inspect the file's contents.

Intermediate 2. Create a CSV file with two new products and import it into the `products` table using `\copy`, specifying an explicit column list.

Advanced 3. Design a bulk-loading plan (in prose, referencing Section 30.5) for importing 10 million historical order records into this course's schema, including your reasoning about indexes and transaction boundaries. # 31. PostgreSQL + Python

31.1 `psycopg2` - Connecting to PostgreSQL from Python

`psycopg2` is the most widely used PostgreSQL driver for Python, implementing the Python DB-API 2.0 standard interface.

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    port=5432,
    dbname="company_db",
    user="postgres",
    password="postgres"
)
```

A `connection` object represents one live session to the database - conceptually identical to a `psql` session (Part 2.5), just opened from Python instead of the command line.

31.2 Cursors

A **cursor** is the object used to actually execute SQL and retrieve results, created from an open connection.

```
cur = conn.cursor()
```

31.3 Executing SQL and Fetching Data

```
cur.execute("SELECT name, email FROM customers WHERE country = %s
ORDER BY name LIMIT 3;", ("Nigeria",))
```

```
rows = cur.fetchall()
print("Query results:", rows)

Query results: [('Ada Lovelace', 'ada@example.com'), ('Chidi Okafor',
'chidi@example.com'), ('Ngozi Adichie', 'ngozi@example.com')]
```

Method	Purpose
`cur.execute(sql, params)`	Runs a SQL statement, optionally with parameters
`cur.fetchall()`	Returns every remaining result row as a list of tuples
`cur.fetchone()`	Returns just the next single row (or `None` if exhausted)
`cur.fetchmany(n)`	Returns up to `n` rows at a time - useful for processing large results in batches

31.4 Parameterized Queries - and Why They Matter

Never build SQL by directly inserting Python values into a string (e.g., f-strings or `%`-formatting the SQL itself) - always pass values as separate parameters, as shown above with `%s` placeholders. This is not just a style preference; it is the single most important defense against **SQL injection**.

```
malicious_input = "Nigeria'; DROP TABLE customers; --"
cur.execute("SELECT COUNT(*) FROM customers WHERE country = %s;",
(malicious_input,))
print("Safe query with malicious-looking input:", cur.fetchone())

cur.execute("SELECT to_regclass('customers');")
print("customers table still exists:", cur.fetchone())

Safe query with malicious-looking input: (0,)
customers table still exists: ('customers',)
```

Because `%s` is a genuine **parameter placeholder** (handled by `psycopg2` and PostgreSQL together, never by risky string concatenation), the entire malicious string - including the embedded `DROP TABLE` attempt - is treated as **literal data** to compare against `country`, not as executable SQL. The `customers` table is completely unaffected, confirmed by `to_regclass` still finding it.

What SQL injection actually is: a vulnerability where untrusted input is concatenated directly into a SQL string, allowing an attacker to inject their own SQL logic. The unsafe equivalent (**never do this**) would look like:

```
# DANGEROUS - NEVER DO THIS: directly formatting untrusted input into
SQL
query = f"SELECT COUNT(*) FROM customers WHERE country =
'{malicious_input}'"
# This would execute: ...WHERE country = 'Nigeria'; DROP TABLE
```

```
customers; --'
# potentially executing the attacker's injected DROP TABLE statement
```

Parameterized queries (`%s` placeholders, values passed separately) close this vulnerability entirely, by design - not through input validation or escaping tricks, but because the SQL structure and the data are transmitted to PostgreSQL as genuinely separate things.

31.5 Transactions and Error Handling

```
from psycopg2 import errors

try:
    cur.execute("UPDATE products SET stock_quantity = stock_quantity -
1 WHERE product_id = 1;")
    cur.execute("INSERT INTO order_items (order_id, product_id,
quantity, unit_price) VALUES (1, 999, 1, 10);")
    conn.commit()
except errors.ForeignKeyViolation as e:
    conn.rollback()
    print("Rolled back due to:", str(e).strip())

cur.execute("SELECT stock_quantity FROM products WHERE product_id =
1;")
print("Stock quantity after rollback:", cur.fetchone())
```

```
Rolled back due to: insert or update on table "order_items" violates
foreign key constraint "order_items_product_id_fkey"
DETAIL:  Key (product_id)=(999) is not present in table "products".
Stock quantity after rollback: (200,)
```

By default, `psycopg2` opens an implicit transaction with the connection's first statement; `conn.commit()` makes changes permanent, `conn.rollback()` discards them entirely - exactly mirroring Part 20's SQL-level `COMMIT`/`ROLLBACK`. Here, the intentionally-invalid `product_id = 999` insert raises `ForeignKeyViolation` (Part 5.3), and the `except` block correctly rolls back **both** statements together - the stock quantity decrement is undone too, confirming the transaction's atomicity (Part 20.4) held even though only the *second* statement actually failed.

31.6 Closing Connections and Cursors

```
cur.close()
conn.close()
```

Always close cursors and connections when finished (or use Python's `with` statement / a context manager, covered in the separate Context Managers reference, for guaranteed cleanup) - an unclosed connection holds a server-side backend process (Part 28.1) and a slot against `max_connections` (Part 28.3) indefinitely.

31.7 Connection Pooling

Opening a new database connection has real overhead (a full TCP handshake, authentication, backend process startup). **Connection pooling** maintains a set of already-open connections,

handed out to application code as needed and returned to the pool rather than closed - dramatically reducing overhead for applications making frequent, short-lived database interactions.

```
from psycopg2 import pool

connection_pool = psycopg2.pool.SimpleConnectionPool(
    minconn=1, maxconn=10,
    host="localhost", port=5432, dbname="company_db", user="postgres",
password="postgres"
)

conn = connection_pool.getconn()
try:
    cur = conn.cursor()
    cur.execute("SELECT COUNT(*) FROM customers;")
    print(cur.fetchone())
finally:
    connection_pool.putconn(conn)   # returns the connection to the
pool, rather than closing it
```

31.8 Summary Table

Concept	psycopg2 Mechanism
Connect	`psycopg2.connect(...)`
Execute SQL	`cursor.execute(sql, params)`
Fetch results	`cursor.fetchall()` / `fetchone()` / `fetchmany(n)`
Prevent SQL injection	Always use `%s` parameter placeholders, never string formatting
Commit/rollback	`connection.commit()` / `connection.rollback()`
Handle specific errors	`from psycopg2 import errors`, catch specific exception classes
Reduce connection overhead	`psycopg2.pool.SimpleConnectionPool`

Exercises - Section 31

Beginner 1. Write a Python script connecting to the course database and printing every product's name and price.

Intermediate 2. Write a parameterized query function `get_orders_for_customer(customer_id)` returning all of that customer's orders, and test it with both a valid and a nonexistent customer ID.

Advanced 3. Write a Python function that inserts a new order and its order_items inside a single transaction, rolling back cleanly if any line item references a nonexistent product, and demonstrate both the success and failure paths. # 32. PostgreSQL + Pandas

32.1 Reading SQL Data into a DataFrame

```
import pandas as pd
from sqlalchemy import create_engine

engine =
create_engine("postgresql+psycopg2://postgres:postgres@localhost:5432/
company_db")

df = pd.read_sql("SELECT * FROM products ORDER BY product_id;",
engine)
print(df.head(3))
```

```
   product_id         product_name  ... stock_quantity  supplier_id
0           1       Wireless Mouse  ...            200            1
1           2  Mechanical Keyboard  ...             80            1
2           3            USB-C Hub  ...            150            2

[3 rows x 6 columns]
```

`pd.read_sql(sql, connection)` runs any SQL query and returns the result directly as a pandas `DataFrame` - the standard bridge between PostgreSQL and the entire Python data-analysis ecosystem.

32.2 SQLAlchemy Integration

`pandas` does not talk to PostgreSQL directly - it goes through **SQLAlchemy**, a database toolkit that provides the `create_engine(...)` connection object `read_sql/to_sql` expect. The connection string format is `dialect+driver://user:password@host:port/dbname` - here, `postgresql+psycopg2` specifies PostgreSQL as the dialect and `psycopg2` (Part 31) as the underlying driver.

32.3 Parameterized Queries with Pandas

```
from sqlalchemy import text

df_params = pd.read_sql(
    text("SELECT * FROM customers WHERE country = :country"),
    engine,
    params={"country": "Nigeria"}
)
print(df_params[["name", "country"]])
```

```
            name  country
0   Ada Lovelace  Nigeria
1   Chidi Okafor  Nigeria
2  Ngozi Adichie  Nigeria
```

Exactly like `psycopg2` (Part 31.4), always use parameter placeholders (here, SQLAlchemy's named `:country` style) rather than formatting untrusted values directly into the SQL string - the SQL injection risk is identical regardless of which library sits between Python and PostgreSQL.

32.4 Data Type Mapping

```
print(df.dtypes)

product_id          int64
product_name          str
category              str
price             float64
stock_quantity      int64
supplier_id         int64
dtype: object
```

PostgreSQL Type	Typical pandas Type
`INTEGER`, `BIGINT`, `SMALLINT`	`int64`
`TEXT`, `VARCHAR`, `CHAR`	`str` (modern pandas string dtype) / `object` (older pandas versions)
`NUMERIC`, `REAL`, `DOUBLE PRECISION`	`float64`
`BOOLEAN`	`bool`
`DATE`, `TIMESTAMP`, `TIMESTAMPTZ`	`datetime64`

Important caution: `NUMERIC` (Part 4.3) is PostgreSQL's *exact* decimal type, specifically chosen for money to avoid floating-point rounding errors - but `pandas` converts it to `float64` by default on read, which **can** reintroduce those same rounding imprecisions once the data is in a DataFrame. For financial calculations that must remain exact throughout, keep the arithmetic in SQL (Part 10.2) or use Python's `Decimal` type explicitly rather than relying on the DataFrame's `float64` columns.

32.5 Writing DataFrames to PostgreSQL

```
new_data = pd.DataFrame({
    "supplier_name": ["Test Supplier A", "Test Supplier B"],
    "country": ["Testland", "Testland"]
})
new_data.to_sql("suppliers", engine, if_exists="append", index=False)

check = pd.read_sql("SELECT * FROM suppliers WHERE country =
'Testland';", engine)
print(check)

   supplier_id    supplier_name   country
0            7  Test Supplier A  Testland
1            8  Test Supplier B  Testland
```

`to_sql` Parameter	Purpose
`if_exists='append'`	Add rows to an existing table (most common for real data)
`if_exists='replace'`	Drop and recreate the table from the DataFrame - useful for scratch/staging tables, dangerous for real data
`if_exists='fail'` (default)	Raise an error if the table already exists
`index=False`	Don't write the DataFrame's pandas index as its own column (almost always what you want)

32.6 Chunking Large Datasets

For result sets too large to comfortably hold in memory at once, `read_sql`'s `chunksize` parameter returns an iterator of smaller DataFrames instead of one giant one.

```
total_rows = 0
for chunk in pd.read_sql("SELECT * FROM order_items;", engine,
chunksize=5):
    total_rows += len(chunk)
print("Total rows via chunking:", total_rows)
```

```
Total rows via chunking: 22
```

Each `chunk` here is processed as its own small DataFrame (5 rows at a time) - the same technique scales naturally to genuinely large tables, processing millions of rows in fixed-size batches without ever loading the entire result set into memory simultaneously.

32.7 A Complete Data Analytics Example

```
# Revenue by country, computed with SQL, finished with pandas for
presentation
revenue_by_country = pd.read_sql("""
    SELECT c.country, SUM(p.amount) AS revenue
    FROM customers c
    JOIN orders o ON o.customer_id = c.customer_id
    JOIN payments p ON p.order_id = o.order_id
    GROUP BY c.country
    ORDER BY revenue DESC
""", engine)

revenue_by_country["pct_of_total"] = (
    revenue_by_country["revenue"] /
revenue_by_country["revenue"].sum() * 100
).round(1)
print(revenue_by_country)
```

```
          country  revenue  pct_of_total
0         Nigeria   283.42          28.7
```

```
1  United Kingdom   255.95          25.9
2   United States   234.97          23.8
3           Ghana   127.39          12.9
4           Kenya    86.48           8.8
```

This is a common, effective pattern: let PostgreSQL do what it's best at (joining and aggregating relational data efficiently, Part 11-12), then hand the already-summarized result to pandas for any further Python-side calculation, visualization, or export - rather than pulling raw, unaggregated rows into Python and doing the aggregation there.

Exercises - Section 32

Beginner 1. Read the entire `employees` table into a DataFrame and print the average `salary` using pandas.

Intermediate 2. Write a DataFrame of three new products to the `products` table using `to_sql`, then read them back to confirm.

Advanced 3. Write a chunked-reading script that processes `order_items` in batches of 10, computing a running total of `quantity * unit_price` across all chunks without loading the entire table into memory at once. # 33. PostgreSQL for Data Analytics

This section applies concepts from Parts 10-16 to realistic business analytics questions, using this course's full dataset (Part 6). It assumes the current system date falls in 2026, well after the 2025 order data - so "recency" figures below reflect that genuine gap, exactly as they would in any real analysis run some time after the data was collected.

33.1 Key Performance Indicators (KPIs)

```sql
SELECT
    COUNT(DISTINCT o.order_id) AS total_orders,
    COUNT(DISTINCT o.customer_id) AS active_customers,
    SUM(p.amount) AS total_revenue,
    ROUND(AVG(p.amount), 2) AS avg_order_value
FROM orders o
JOIN payments p ON p.order_id = o.order_id;
```

```
 total_orders | active_customers | total_revenue | avg_order_value
--------------+------------------+---------------+-----------------
           13 |                9 |        988.21 |           76.02
(1 row)
```

A single query producing the handful of top-line numbers that typically anchor a business dashboard - already familiar individually from Part 11, combined here into one summary row.

33.2 Revenue Analysis by Country

```sql
SELECT c.country, SUM(p.amount) AS revenue
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN payments p ON p.order_id = o.order_id
```

```
GROUP BY c.country
ORDER BY revenue DESC;
```

```
    country     | revenue
----------------+---------
 Nigeria        |  283.42
 United Kingdom |  255.95
 United States  |  234.97
 Ghana          |  127.39
 Kenya          |   86.48
(5 rows)
```

(Repeated from Part 11.5, restated here as the anchor for this section's further analysis.)

33.3 Customer Segmentation

```
WITH spend AS (
    SELECT c.customer_id, c.name, COALESCE(SUM(p.amount), 0) AS
total_spent
    FROM customers c
    LEFT JOIN orders o ON o.customer_id = c.customer_id
    LEFT JOIN payments p ON p.order_id = o.order_id
    GROUP BY c.customer_id, c.name
)
SELECT name, total_spent, NTILE(3) OVER (ORDER BY total_spent DESC) AS
segment
FROM spend ORDER BY total_spent DESC;
```

```
     name      | total_spent | segment
---------------+-------------+---------
 John Smith    |      234.97 |       1
 Ada Lovelace  |      215.45 |       1
 Grace Hopper  |      175.97 |       1
 Emily Davis   |       79.98 |       1
 Kwabena Owusu |       69.90 |       2
 Chidi Okafor  |       67.97 |       2
 Amara Chukwu  |       57.49 |       2
 Tariq Hassan  |       48.98 |       2
 Wanjiru Kamau |       37.50 |       3
 Ngozi Adichie |           0 |       3
 David Chen    |           0 |       3
(11 rows)
```

`NTILE(3)` (Part 15.4) splits customers into three roughly equal-sized spending tiers - segment 1 ("high-value"), 2 ("mid-value"), 3 ("low-value/inactive") - a simple, immediately actionable segmentation directly from a single window function.

33.4 RFM Analysis (Recency, Frequency, Monetary)

```
WITH customer_rfm AS (
    SELECT
        c.customer_id, c.name,
```

```
        CURRENT_DATE - MAX(o.order_date)::date AS recency_days,
        COUNT(DISTINCT o.order_id) AS frequency,
        COALESCE(SUM(p.amount), 0) AS monetary
    FROM customers c
    LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status =
'completed'
    LEFT JOIN payments p ON p.order_id = o.order_id
    GROUP BY c.customer_id, c.name
)
SELECT name, recency_days, frequency, monetary FROM customer_rfm ORDER
BY monetary DESC LIMIT 5;
```

```
     name      | recency_days | frequency | monetary
---------------+--------------+-----------+----------
 John Smith    |          517 |         2 |   234.97
 Ada Lovelace  |          536 |         3 |   215.45
 Grace Hopper  |          557 |         2 |   175.97
 Emily Davis   |          530 |         1 |    79.98
 Kwabena Owusu |          548 |         1 |    69.90
(5 rows)
```

RFM analysis scores customers on three independent dimensions: how recently they bought (`recency_days` - lower is better), how often (`frequency` - higher is better), and how much (`monetary` - higher is better). In a full implementation, each dimension is typically further scored with `NTILE` (Section 33.3) into quintiles, then combined into a single composite RFM score used to target marketing campaigns.

33.5 Retention and Churn

Retention measures whether customers continue engaging over time; **churn** is its inverse - customers who stop. A simple churn indicator, using this dataset's `recency_days`:

```
WITH customer_rfm AS (
    SELECT c.customer_id, c.name, CURRENT_DATE -
MAX(o.order_date)::date AS recency_days
    FROM customers c
    LEFT JOIN orders o ON o.customer_id = c.customer_id
    GROUP BY c.customer_id, c.name
)
SELECT
    CASE WHEN recency_days IS NULL THEN 'Never Ordered'
         WHEN recency_days > 365 THEN 'Likely Churned'
         ELSE 'Active' END AS status,
    COUNT(*) AS customers
FROM customer_rfm
GROUP BY status
ORDER BY customers DESC;
```

A real production version would define "likely churned" using a threshold meaningful to the specific business (e.g., "no purchase in 90 days" for a fast-moving retailer, versus "no purchase

in 2 years" for a durable-goods company) - the SQL pattern (bucketing `recency_days` with `CASE`) stays the same regardless of the threshold chosen.

33.6 Cohort Analysis

A **cohort** groups customers by a shared starting event (typically signup date), tracking how each cohort behaves over time - most commonly, whether they go on to make a purchase at all.

```
WITH first_orders AS (
    SELECT customer_id, MIN(order_date)::date AS first_order_date FROM
orders GROUP BY customer_id
)
SELECT
    DATE_TRUNC('month', c.signup_date)::date AS signup_month,
    COUNT(*) AS customers,
    COUNT(fo.customer_id) AS customers_who_ordered
FROM customers c
LEFT JOIN first_orders fo ON fo.customer_id = c.customer_id
GROUP BY signup_month
ORDER BY signup_month;

 signup_month | customers | customers_who_ordered
--------------+-----------+-----------------------
 2024-01-01   |         1 |                     1
 2024-02-01   |         1 |                     1
 2024-03-01   |         2 |                     2
 2024-04-01   |         2 |                     2
 2024-05-01   |         2 |                     2
 2024-06-01   |         1 |                     1
 2024-07-01   |         1 |                     1
 2024-08-01   |         1 |                     0
(8 rows)
```

Every monthly signup cohort converted to at least one order **except** the August 2024 cohort (`David Chen`, the customer identified repeatedly since Part 12.4 as never having ordered) - immediately visible from this cohort breakdown without needing to inspect individual customers.

33.7 Growth Analysis: Month-over-Month

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', payment_date)::date AS month,
SUM(amount) AS revenue
    FROM payments GROUP BY DATE_TRUNC('month', payment_date)::date
)
SELECT month, revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month)) /
LAG(revenue) OVER (ORDER BY month), 1) AS mom_growth_pct
FROM monthly ORDER BY month;

   month    | revenue | mom_growth_pct
------------+---------+----------------
```

```
 2025-01-01 |  192.44 |
 2025-02-01 |  212.47 |           10.4
 2025-03-01 |  429.34 |          102.1
 2025-04-01 |  153.96 |          -64.1
(4 rows)
```

The classic `LAG()`-based growth formula: `(current - previous) / previous * 100` - January has no prior month to compare against, correctly showing `NULL`. March shows explosive growth (+102.1%) followed by an equally sharp April decline (-64.1%) - exactly the kind of month-to-month volatility a growth-rate view surfaces clearly, that a raw revenue table alone would not.

33.8 Growth Analysis: Year-over-Year

```
WITH yearly AS (
    SELECT EXTRACT(YEAR FROM order_date)::int AS yr, COUNT(*) AS
orders
    FROM orders GROUP BY EXTRACT(YEAR FROM order_date)::int
)
SELECT yr, orders, orders - LAG(orders) OVER (ORDER BY yr) AS
yoy_change
FROM yearly ORDER BY yr;

 yr   | orders | yoy_change
------+--------+------------
 2025 |     15 |
(1 row)
```

This dataset spans only a single calendar year, so `yoy_change` is correctly `NULL` - there is no prior year to compare against. The identical `LAG()`-based pattern from Section 33.7 applies directly to year-over-year comparisons once multiple years of data exist; only the grouping granularity (`EXTRACT(YEAR ...)` instead of `DATE_TRUNC('month', ...)`) changes.

33.9 Running Totals and Ranking (Recap)

Running totals (`SUM() OVER (ORDER BY ...)`) and ranking (`RANK()`, `DENSE_RANK()`, `ROW_NUMBER()`) were covered in full technical depth in Part 15 - Sections 15.3 and 15.7 - and are used constantly throughout real analytics work exactly as shown there: cumulative revenue over time, top-N products per category, and similar "ordered position within a group" questions.

Exercises - Section 33

Beginner 1. Compute total revenue and order count per store, ordered from highest to lowest revenue.

Intermediate 2. Extend Section 33.3's segmentation query to also show each segment's average `total_spent`, using a second query or CTE layered on top.

Advanced 3. Design a full RFM scoring query that uses `NTILE(5)` independently on recency, frequency, and monetary value, then combines all three into a single concatenated score string (e.g., `'531'`), and rank customers by this combined score. # 34. PostgreSQL for Data Science

34.1 The Data Scientist's Relationship with PostgreSQL

Before any modeling happens, a data scientist needs a reliable way to **collect**, **prepare**, and **feature-engineer** data - and doing as much of this work as possible directly in SQL (rather than after pulling raw rows into Python) is almost always faster and more reliable, since PostgreSQL's query engine is specifically optimized for exactly this kind of aggregation and filtering.

34.2 Feature Engineering in SQL

```
SELECT
    c.customer_id,
    COUNT(DISTINCT o.order_id) AS num_orders,
    COALESCE(SUM(p.amount), 0) AS total_spent,
    COALESCE(AVG(p.amount), 0) AS avg_order_value,
    CASE WHEN COUNT(DISTINCT o.order_id) >= 2 THEN 1 ELSE 0 END AS
is_repeat_customer
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status =
'completed'
LEFT JOIN payments p ON p.order_id = o.order_id
GROUP BY c.customer_id;
```

```
 customer_id | num_orders | total_spent | avg_order_value |
is_repeat_customer
-------------+------------+-------------+-----------------
+--------------------
           1 |          3 |      215.45 |        71.816667 |
1
           2 |          2 |      175.97 |        87.985000 |
1
           3 |          1 |       67.97 |        67.970000 |
0
           4 |          0 |        0.00 |         0.000000 |
0
           5 |          2 |      234.97 |       117.485000 |
1
          ...
```

This single query performs the entire feature-engineering step for a "will this customer become a repeat purchaser?" model: `num_orders`, `total_spent`, and `avg_order_value` are all **engineered features**, and `is_repeat_customer` is the **target label** - computed directly from raw relational data using exactly the aggregation and `CASE`-expression techniques from Parts 10-11, applied here with a modeling goal in mind.

34.3 A Complete PostgreSQL -> Python -> pandas -> scikit-learn -> Model Project

```
import pandas as pd
from sqlalchemy import create_engine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

engine =
create_engine("postgresql+psycopg2://postgres:postgres@localhost:5432/
company_db")

query = """
SELECT
    c.customer_id,
    COUNT(DISTINCT o.order_id) AS num_orders,
    COALESCE(SUM(p.amount), 0) AS total_spent,
    COALESCE(AVG(p.amount), 0) AS avg_order_value,
    CASE WHEN COUNT(DISTINCT o.order_id) >= 2 THEN 1 ELSE 0 END AS
is_repeat_customer
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status =
'completed'
LEFT JOIN payments p ON p.order_id = o.order_id
GROUP BY c.customer_id
"""
df = pd.read_sql(query, engine)

X = df[["total_spent", "avg_order_value"]]
y = df["is_repeat_customer"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)
print("Test accuracy:", model.score(X_test, y_test))
```

```
Test accuracy: 1.0
```

(A perfect score here reflects this course's small, simple demonstration dataset - real datasets require far more careful validation, additional features, and a larger, more representative sample before any accuracy figure should be trusted.)

34.4 Storing Predictions Back in PostgreSQL

```
df["predicted_repeat"] = model.predict(X)
df[["customer_id", "predicted_repeat"]].to_sql("customer_predictions",
engine, if_exists="replace", index=False)
```

```
SELECT * FROM customer_predictions ORDER BY customer_id LIMIT 5;
```

```
 customer_id | predicted_repeat
-------------+------------------
```

```
           1 |                   1
           2 |                   1
           3 |                   0
           4 |                   0
           5 |                   1
(5 rows)
```

Writing predictions back into PostgreSQL (rather than leaving them only in a Python session or a one-off CSV) makes them queryable alongside the rest of the business data immediately - joinable with `customers`, `orders`, or any other table, and usable by any downstream application or dashboard without needing to re-run the Python model.

34.5 Storing Model Metadata

Beyond predictions themselves, tracking **which model produced them, when, and with what configuration** is essential for reproducibility - a simple metadata table:

```
CREATE TABLE model_runs (
    run_id       SERIAL PRIMARY KEY,
    model_name   TEXT NOT NULL,
    trained_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    parameters   JSONB,
    test_accuracy NUMERIC(5, 4)
);

INSERT INTO model_runs (model_name, parameters, test_accuracy)
VALUES ('repeat_customer_classifier_v1', '{"model_type":
"LogisticRegression", "test_size": 0.3}', 1.0000);
```

Storing `parameters` as `JSONB` (Part 22) accommodates the fact that different model types have entirely different configuration shapes, without needing a rigid, ever-changing set of columns for every possible hyperparameter across every model type ever trained.

34.6 Filtering Training Data

Real-world model training frequently needs carefully filtered subsets - excluding test/internal accounts, restricting to a specific date range, or excluding known data-quality issues - all naturally expressed as an ordinary `WHERE` clause (Part 8) layered onto the feature-engineering query from Section 34.2, rather than filtering after the fact in pandas.

```
SELECT c.customer_id, COUNT(o.order_id) AS num_orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE c.signup_date >= '2024-01-01'   -- exclude any pre-launch test
accounts, for example
GROUP BY c.customer_id;
```

34.7 Why This Workflow Matters

- **Reproducibility**: the exact feature-engineering logic lives in a versionable SQL query, not scattered pandas transformations that are easy to apply inconsistently between training and later inference.
- **Performance**: PostgreSQL computes aggregations over potentially millions of rows far more efficiently than pulling raw rows into pandas and aggregating there.
- **Collaboration**: SQL feature-engineering queries are readable and reviewable by data analysts and engineers who may not work directly in Python, keeping the whole team aligned on exactly how each feature is defined.

Exercises - Section 34

Beginner 1. Write a feature-engineering query producing one row per product with `total_units_sold` and `total_revenue` as features.

Intermediate 2. Extend Section 34.2's query with an additional feature, `days_since_signup`, and retrain the model including it.

Advanced 3. Design a `model_runs`-style metadata table capturing enough information (features used, training data date range, model parameters, evaluation metrics) that a colleague could exactly reproduce any past model run from the stored metadata alone. # 35. PostgreSQL for Data Engineering

35.1 ETL vs ELT

Pattern	Order of Operations	Where Transformation Happens
ETL (Extract, Transform, Load)	Extract from source -> transform in a separate processing step -> load the already-clean result into the target database	Outside the database (application code, a dedicated ETL tool)
ELT (Extract, Load, Transform)	Extract from source -> load the raw data in as-is -> transform it **using the target database's own SQL**	Inside the database itself, using SQL

Modern data engineering increasingly favors **ELT**, leveraging PostgreSQL's own query engine (joins, aggregations, window functions) to perform transformations, rather than shipping data through a separate processing layer first - directly enabled by the staging-table and SQL techniques throughout this section.

35.2 Data Pipelines and Staging Tables

A **staging table** holds raw, not-yet-validated data temporarily, before it is cleaned and moved into permanent tables - exactly the role temporary tables (Part 26) or dedicated permanent staging tables play in a pipeline.

```
CREATE TEMPORARY TABLE staging_products (
    product_name TEXT, category TEXT, price TEXT, stock_quantity TEXT
);
INSERT INTO staging_products VALUES
    ('Bluetooth Speaker', 'Electronics', '45.99', '30'),
    ('bad_row', 'Electronics', 'not_a_number', '10');
```

Note every staged column is deliberately TEXT - raw incoming data (e.g., from a CSV import, Part 30) often arrives with inconsistent or invalid formatting, and forcing it into strict numeric types too early would simply cause the load to fail outright rather than allowing bad rows to be identified and handled.

35.3 Data Validation and Data Quality

```
SELECT * FROM staging_products WHERE price ~ '^[0-9]+\.?[0-9]*$';

    product_name    |  category   | price | stock_quantity
--------------------+-------------+-------+----------------
 Bluetooth Speaker  | Electronics | 45.99 |             30
(1 row)
```

This regular-expression filter (matching only genuinely numeric-looking strings) separates the one valid row from the corrupted `'bad_row'` entry (whose `price` is `'not_a_number'`) - exactly the kind of validation step a real pipeline runs before ever attempting to cast staged text into proper numeric columns and load it into a permanent table. Rows that fail validation are typically routed to a separate "rejected rows" table or log for manual review, rather than silently dropped or allowed to crash the entire load.

35.4 Fact Tables and Dimension Tables

A **star schema** (the standard data-warehouse design pattern) separates data into: - **Fact tables**: numeric, event-level measurements (a sale, a click, a shipment) - typically very large, with foreign keys to multiple dimensions. - **Dimension tables**: descriptive context for analyzing facts (dates, products, customers, stores) - typically much smaller, and often deliberately denormalized (Part 21.11) for query simplicity.

```
CREATE TABLE dim_date (date_key DATE PRIMARY KEY, year INT, month INT,
day INT);
CREATE TABLE fact_sales (order_id INT, date_key DATE REFERENCES
dim_date(date_key), amount NUMERIC);

INSERT INTO dim_date
SELECT d::date, EXTRACT(YEAR FROM d)::int, EXTRACT(MONTH FROM d)::int,
EXTRACT(DAY FROM d)::int
FROM generate_series('2025-01-01'::date, '2025-04-30'::date, '1 day')
d;

INSERT INTO fact_sales
SELECT o.order_id, o.order_date::date, p.amount
FROM orders o JOIN payments p ON p.order_id = o.order_id;
```

```
SELECT dd.year, dd.month, SUM(fs.amount)
FROM fact_sales fs JOIN dim_date dd ON dd.date_key = fs.date_key
GROUP BY dd.year, dd.month ORDER BY dd.month;

 year | month |  sum
------+-------+--------
 2025 |     1 | 192.44
 2025 |     2 | 212.47
 2025 |     3 | 429.34
 2025 |     4 | 153.96
(4 rows)
```

`generate_series` (a PostgreSQL-specific function, covered further in Part 38) generated a complete calendar `dim_date` table spanning every single day in the range - a standard data-warehousing technique, ensuring every possible date (even ones with zero sales) is available to join against, which matters when reports need to show gaps (e.g., days with no sales) rather than silently omitting them.

35.5 Incremental Loading

Rather than reloading an entire source table on every pipeline run (expensive and slow at scale), **incremental loading** only extracts and loads rows that are new or changed since the last run - typically tracked via a "high-water mark" (the maximum `updated_at` or ID value already processed).

```
-- Conceptual incremental load pattern: only pull rows newer than the
last successfully processed timestamp
SELECT * FROM orders WHERE order_date > (SELECT MAX(last_loaded_at)
FROM pipeline_state);
```

35.6 Upserts - `INSERT ... ON CONFLICT`

An **upsert** inserts a new row, or updates it if a conflicting row (by primary key or unique constraint) already exists - a single-statement pattern essential for incremental loading, where the same source record might be seen again with updated values.

```
INSERT INTO products (product_id, product_name, category, price,
stock_quantity, supplier_id)
VALUES (1, 'Wireless Mouse', 'Electronics', 17.99, 200, 1)
ON CONFLICT (product_id) DO UPDATE SET price = EXCLUDED.price
RETURNING product_id, product_name, price;

 product_id |  product_name  | price
------------+----------------+-------
          1 | Wireless Mouse |  17.99
(1 row)
```

`ON CONFLICT (product_id) DO UPDATE SET price = EXCLUDED.price` means: if a row with this `product_id` already exists, update **only** its `price` (leaving every other column untouched), using `EXCLUDED` to refer to the value that *would* have been inserted. Without `ON`

`CONFLICT`, this statement would have failed outright with a duplicate-key error (Part 5.2) - `ON CONFLICT` is precisely what turns that failure into a controlled, deliberate update instead.

35.7 Batch Processing

Data pipelines typically process data in **batches** (a day's worth of new orders, an hourly file drop) rather than one row at a time - the `COPY`/`\copy` bulk-loading techniques (Part 30), staging-table validation (Section 35.3), and upsert patterns (Section 35.6) in this part are the standard building blocks of exactly this kind of batch pipeline, typically orchestrated by an external scheduler (e.g., a cron job, Airflow, or a similar workflow tool) that runs the full extract-stage-validate-load sequence on a regular schedule.

35.8 How PostgreSQL Functions Within a Modern Data Pipeline

Pipeline Stage	PostgreSQL's Role
Staging	Temporary or dedicated staging tables (Section 35.2)
Validation	SQL-based checks (regex, `CHECK` constraints, Part 5) before promoting staged data
Transformation (ELT)	SQL joins, aggregations, and window functions performing the actual transformation logic
Loading	`COPY`/`\copy` (Part 30) for bulk speed, `ON CONFLICT` (Section 35.6) for incremental upserts
Serving	Fact/dimension tables (Section 35.4), views, and materialized views (Part 17) as the final, query-ready layer

Exercises - Section 35

Beginner 1. Create a staging table for raw customer signups with all-`TEXT` columns, insert one valid and one invalid row, then write a validation query separating them.

Intermediate 2. Write an upsert statement that inserts a new supplier or updates its `country` if the `supplier_name` already exists (hint: add a `UNIQUE` constraint on `supplier_name` first).

Advanced 3. Design a simple star schema (one fact table, at least two dimension tables) for analyzing this course's order data by date and by product category, and write one analytical query against it. # 36. PostgreSQL for AI Engineering

36.1 PostgreSQL's Role in AI Applications

Beyond traditional relational data, modern AI applications (chatbots, retrieval-augmented generation systems, recommendation engines) need to store and query several AI-specific kinds of data - and PostgreSQL, with the right extensions, can serve as a single system for **all** of it:

user data, chat history, documents, metadata, **and** vector embeddings, rather than requiring a separate specialized vector database alongside the main application database.

```
CREATE TABLE chat_sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id INTEGER REFERENCES customers(customer_id),
    started_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE chat_messages (
    message_id SERIAL PRIMARY KEY,
    session_id UUID REFERENCES chat_sessions(session_id),
    role TEXT NOT NULL CHECK (role IN ('user', 'assistant',
'system')),
    content TEXT NOT NULL,
    metadata JSONB,
    sent_at TIMESTAMPTZ DEFAULT NOW()
);
```

This is ordinary relational design (Part 21) applied directly to an AI application's chat history - `metadata JSONB` (Part 22) flexibly accommodates whatever extra, evolving fields a specific AI system needs to track (token counts, model version, latency) without requiring schema changes for every new field.

36.2 Embeddings and Vector Data

An **embedding** is a fixed-length vector of numbers representing the "meaning" of a piece of text (or an image, or audio), produced by a machine learning model - texts with similar meaning produce vectors that are numerically "close" to each other in this vector space, even if they share few or no words in common.

36.3 `pgvector` - PostgreSQL's Vector Extension

```
CREATE EXTENSION vector;

CREATE TABLE document_embeddings (
    doc_id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    embedding VECTOR(3)   -- a real embedding model would use hundreds
or thousands of dimensions
);

INSERT INTO document_embeddings (content, embedding) VALUES
    ('Refund policy: items can be returned within 30 days.', '[0.9,
0.1, 0.0]'),
    ('Shipping takes 3-5 business days domestically.', '[0.1, 0.9,
0.0]'),
    ('You can return a product within a month for a full refund.',
'[0.85, 0.15, 0.05]');
```

```
CREATE EXTENSION
CREATE TABLE
INSERT 0 3
```

`pgvector` adds a genuine `VECTOR(n)` column type (here, 3 dimensions purely for a readable demonstration - real text-embedding models typically produce 384 to 3072-dimensional vectors) directly to PostgreSQL, alongside specialized operators and index types for searching them efficiently.

36.4 Similarity Search

```
SELECT content, embedding <=> '[0.9, 0.1, 0.0]' AS distance
FROM document_embeddings
ORDER BY distance
LIMIT 2;

                          content                           |
distance
------------------------------------------------------------
+----------------------
 Refund policy: items can be returned within 30 days.       |
0
 You can return a product within a month for a full refund. |
0.0037184699510960373
(2 rows)
```

`<=>` is `pgvector`'s **cosine distance** operator - smaller values mean more similar vectors. Ordering by this distance and taking the smallest values is precisely how **semantic search** works: rather than matching literal keywords, it finds content whose *meaning* (as captured by the embedding model) is closest to the query. Note the two returned documents are both about refunds/returns, despite sharing very few exact words - the third document (about shipping times) correctly ranks last, since its embedding is numerically distant from the query vector.

Operator	Distance Measure
`<->`	Euclidean (L2) distance
`<=>`	Cosine distance
`<#>`	Negative inner product

36.5 Indexing Vectors for Fast Search

```
CREATE INDEX ON document_embeddings USING ivfflat (embedding
vector_cosine_ops) WITH (lists = 1);

CREATE INDEX
```

An `ivfflat` (or the newer `hnsw`) index lets `pgvector` find approximate nearest neighbors far faster than comparing a query vector against every single row - essential once a table holds millions of embeddings, exactly analogous to why B-tree indexes (Part 18) matter for ordinary WHERE-clause lookups at scale.

36.6 Retrieval-Augmented Generation (RAG) - The Concept

RAG combines a retrieval step (finding relevant documents via embedding similarity search, Section 36.4) with a generation step (an LLM producing a natural-language answer, using the retrieved documents as context) - rather than relying purely on what the LLM "already knows," which may be outdated or not specific to a particular business's own data.

```
User question: "How long do I have to return an item?"
        |
        v
[1] Embed the question into a vector (using the same embedding model
as the documents)
        |
        v
[2] Similarity search (Section 36.4) against document_embeddings ->
retrieves the refund policy document
        |
        v
[3] Pass the retrieved document text + the original question to an LLM
as context
        |
        v
[4] LLM generates a grounded answer: "You can return items within 30
days."
```

36.7 A Conceptual PostgreSQL + pgvector + Python + LLM/RAG Project

```
import psycopg2

# Step 1: a real project would call an embedding API/model here; this
stands in with a fixed vector
def embed_text(text):
    return [0.9, 0.12, 0.0]   # placeholder - a real embedding model
returns hundreds of dimensions

def retrieve_relevant_docs(question, top_k=2):
    query_vector = embed_text(question)
    conn = psycopg2.connect(host="localhost", dbname="company_db",
user="postgres", password="postgres")
    cur = conn.cursor()
    cur.execute(
        "SELECT content FROM document_embeddings ORDER BY embedding
<=> %s::vector LIMIT %s;",
        (str(query_vector), top_k)
    )
    results = [row[0] for row in cur.fetchall()]
    cur.close()
    conn.close()
    return results

def answer_question(question):
```

```
    context_docs = retrieve_relevant_docs(question)
    context = "\n".join(context_docs)
    # Step 2: a real project would send `context` + `question` to an
LLM API here
    prompt = f"Context:\n{context}\n\nQuestion: {question}\nAnswer:"
    return prompt   # returned here for illustration; a real system
sends this to an LLM and returns its reply

print(answer_question("How long do I have to return an item?"))
```

This structure - a retrieval function backed by `pgvector` similarity search, feeding into a generation step - is the essential shape of every production RAG system, regardless of which specific embedding model or LLM provider sits at each end.

36.8 Storing Documents and Metadata for AI Applications

```
CREATE TABLE knowledge_base_documents (
    doc_id       SERIAL PRIMARY KEY,
    title        TEXT NOT NULL,
    content      TEXT NOT NULL,
    source_url   TEXT,
    embedding    VECTOR(1536),   -- matches a common real embedding
model's output size
    metadata     JSONB,
    indexed_at   TIMESTAMPTZ DEFAULT NOW()
);
```

Combining ordinary relational columns (`title`, `source_url`), an embedding (`VECTOR(1536)`), and flexible `JSONB` metadata in a single table is the standard, pragmatic PostgreSQL pattern for a RAG system's knowledge base - one system serving both the traditional application data and the AI-specific vector search needs.

Exercises - Section 36

Beginner 1. Create a table with a `VECTOR(2)` column, insert three rows with simple 2D vectors, and run a similarity search using `<=>`.

Intermediate 2. Design a `chat_messages` table (like Section 36.1) and write a query retrieving the full conversation history for one session, ordered chronologically.

Advanced 3. Extend Section 36.7's conceptual RAG project to also store each question and its retrieved context in a `chat_messages`-style table, enabling later analysis of which documents are retrieved most frequently. # 37. Advanced Analytics with PostgreSQL

This section extends Parts 15 and 33 with additional analytical techniques not yet covered: percentiles, statistical functions, funnel analysis, and customer lifetime value. Window functions, recursive queries, cohort analysis, and moving averages were already covered in full depth in Part 15 and Part 33 and are not repeated here.

37.1 Percentiles

```
SELECT
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY price) AS
median_price,
    PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY price) AS p90_price
FROM products;

 median_price | p90_price
--------------+-----------
        32.47 |    76.191
(1 row)
```

`PERCENTILE_CONT(fraction) WITHIN GROUP (ORDER BY column)` computes an interpolated percentile - `0.5` is the median (50th percentile), `0.9` is the 90th percentile. Unlike `AVG()`, percentiles are robust to extreme outliers - a single very expensive product barely shifts the median, whereas it could shift the average substantially.

37.2 Statistical Calculations

```
SELECT ROUND(STDDEV(price)::numeric, 2) AS stddev_price,
ROUND(VARIANCE(price)::numeric, 2) AS variance_price
FROM products;

 stddev_price | variance_price
--------------+----------------
        33.24 |        1104.91
(1 row)
```

`STDDEV` (standard deviation) and `VARIANCE` measure how spread out a column's values are around their mean - a `stddev_price` of `33.24` on a catalog with prices ranging from `12.50` to `129.99` indicates genuinely wide price variation, not a tightly clustered catalog.

```
SELECT ROUND(CORR(quantity, unit_price)::numeric, 3) AS correlation
FROM order_items;

 correlation
-------------
      -0.295
```

`CORR(x, y)` computes the Pearson correlation coefficient between two columns, ranging from `-1` (perfect inverse relationship) to `+1` (perfect direct relationship). A value of `-0.295` here suggests a mild tendency for higher-priced items to be ordered in smaller quantities in this dataset - a real, though modest, negative correlation, not a strong one.

37.3 Funnel Analysis

A **funnel** tracks how many entities progress through a sequence of stages, typically shrinking at each step - essential for understanding where potential customers are lost.

```
SELECT
    COUNT(DISTINCT c.customer_id) AS total_customers,
    COUNT(DISTINCT o.customer_id) AS customers_with_orders,
```

```
    COUNT(DISTINCT CASE WHEN o.status = 'completed' THEN o.customer_id
END) AS customers_completed,
    COUNT(DISTINCT CASE WHEN p.order_id IS NOT NULL THEN o.customer_id
END) AS customers_paid
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN payments p ON p.order_id = o.order_id;

 total_customers | customers_with_orders | customers_completed |
customers_paid
-----------------+-----------------------+---------------------
+-----------------
              11 |                    10 |                   9 |
9
(1 row)
```

Reading this funnel: all 11 registered customers -> 10 placed at least one order (`David Chen` is the one drop-off, Part 12.4) -> 9 have at least one *completed* order (one customer's order remains `pending` or was `cancelled`) -> 9 completed payment. Each `COUNT(DISTINCT CASE WHEN ... END)` counts customers reaching that specific stage, all computed from a single pass over the joined data.

37.4 Customer Lifetime Value (CLV)

Customer Lifetime Value estimates the total worth of a customer relationship - the simplest version is just historical total spend; more sophisticated versions project **future** expected value based on tenure and purchase patterns.

```
SELECT c.customer_id, c.name,
       COALESCE(SUM(p.amount), 0) AS historical_value,
       COALESCE(SUM(p.amount), 0) / NULLIF(EXTRACT(YEAR FROM
AGE(NOW(), c.signup_date::timestamptz)), 0) AS approx_annual_value
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN payments p ON p.order_id = o.order_id
GROUP BY c.customer_id, c.name, c.signup_date
ORDER BY historical_value DESC
LIMIT 5;

 customer_id |     name      | historical_value | approx_annual_value
-------------+---------------+------------------
+----------------------
           5 | John Smith    |           234.97 | 117.4850000000000000
           1 | Ada Lovelace  |           215.45 | 107.7250000000000000
           2 | Grace Hopper  |           175.97 |  87.9850000000000000
           9 | Emily Davis   |            79.98 |  39.9900000000000000
           7 | Kwabena Owusu |            69.90 |  34.9500000000000000
(5 rows)
```

`AGE(NOW(), signup_date)` computes an `INTERVAL` representing how long each customer has been a customer; dividing lifetime spend by years-as-customer gives a rough **annualized**

value estimate - `NULLIF(..., 0)` (Part 10.4) guards against division by zero for any customer who signed up less than a year ago.

37.5 Combining Techniques into One Analytical Report

```
WITH customer_metrics AS (
    SELECT c.customer_id, c.name, COALESCE(SUM(p.amount), 0) AS
total_spent
    FROM customers c
    LEFT JOIN orders o ON o.customer_id = c.customer_id
    LEFT JOIN payments p ON p.order_id = o.order_id
    GROUP BY c.customer_id, c.name
)
SELECT
    name, total_spent,
    ROUND((PERCENT_RANK() OVER (ORDER BY total_spent) * 100)::numeric,
1) AS percentile_rank
FROM customer_metrics
ORDER BY total_spent DESC
LIMIT 5;
```

`PERCENT_RANK()` (a window function from the same family as Part 15's ranking functions) expresses each customer's relative standing as a percentile within the whole customer base - a natural companion to the percentile and statistical techniques introduced in this part.

Exercises - Section 37

Beginner 1. Compute the median and 90th-percentile order value (from `payments.amount`) for this course's dataset.

Intermediate 2. Compute the standard deviation of `salary` across all employees, and identify which employees fall more than one standard deviation above the mean.

Advanced 3. Extend Section 37.3's funnel with an additional stage - "customers who placed more than one order" - and compute the conversion rate (as a percentage) between each consecutive stage of the funnel. # 38. PostgreSQL-Specific Features

This section consolidates the features throughout this course that are **PostgreSQL extensions to the SQL standard**, rather than portable, standard SQL - useful as a single reference for what will and won't transfer directly to another RDBMS (expanded further in Part 39).

38.1 Quick Reference: Where Each Feature Was Already Covered

Feature	PostgreSQL-Specific?	Covered In
`JSONB`	Yes (standard SQL has `JSON`, but not `JSONB` or its operators)	Part 22
Arrays (`TEXT[]`, etc.)	Yes	Part 22
`ILIKE`	Yes	Part 8.5
`RETURNING`	Yes (increasingly adopted	Part 6.4

Feature	PostgreSQL-Specific?	Covered In
	elsewhere, but originated in PostgreSQL)	
`FILTER (WHERE ...)`	Standard SQL, but rarely implemented outside PostgreSQL	Part 23.2
`DISTINCT ON`	Yes	Part 23.5
`STRING_AGG`	Common but not universal (`GROUP_CONCAT` in MySQL is different syntax)	Part 23.4
Materialized views	Common but varies significantly in syntax/refresh semantics across databases	Part 17.6
`ON CONFLICT` (upsert)	Yes (`MERGE` is the closer standard-SQL equivalent)	Part 35.6

38.2 `LATERAL` Joins

A `LATERAL` subquery may reference columns from tables **earlier** in the same `FROM` clause - something an ordinary subquery cannot do. This enables "for each row, compute this" patterns directly in SQL.

```
SELECT c.name, top_orders.order_id, top_orders.order_date
FROM customers c
CROSS JOIN LATERAL (
    SELECT order_id, order_date FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY order_date DESC LIMIT 2
) AS top_orders
ORDER BY c.name, top_orders.order_date DESC
LIMIT 6;
```

```
     name     | order_id |       order_date
--------------+----------+------------------------
 Ada Lovelace |       12 | 2025-03-22 09:35:00+00
 Ada Lovelace |        4 | 2025-02-02 16:45:00+00
 Amara Chukwu |        7 | 2025-02-25 08:50:00+00
 Chidi Okafor |       10 | 2025-03-15 17:25:00+00
 Chidi Okafor |        3 | 2025-01-20 09:00:00+00
 Emily Davis  |       13 | 2025-03-28 14:15:00+00
(6 rows)
```

This finds each customer's **2 most recent orders** - a "top-N per group" query that would otherwise require a window function with a rank filter (Part 15.3). `LATERAL` lets the inner subquery (`top_orders`) reference `c.customer_id` directly, re-running conceptually once

per outer row - standard SQL:2003 technically defines LATERAL, but PostgreSQL's support and idiomatic use of it is considerably more mature and common than in most other databases.

38.3 GENERATE_SERIES

`generate_series` produces a set of rows from a numeric range, or a range of dates/timestamps at a given step - extremely useful for generating calendar tables (Part 35.4) or filling in gaps in sparse data.

```
SELECT generate_series(1, 5) AS n;

 n
---
 1
 2
 3
 4
 5
(5 rows)

SELECT generate_series('2025-01-01'::date, '2025-01-05'::date, '1
day') AS day;

          day
------------------------
 2025-01-01 00:00:00+00
 2025-01-02 00:00:00+00
 2025-01-03 00:00:00+00
 2025-01-04 00:00:00+00
 2025-01-05 00:00:00+00
(5 rows)
```

`generate_series` has no equivalent in the SQL standard at all - it is one of PostgreSQL's most distinctive and widely-used conveniences, especially for reporting (filling in every date/number in a range, even ones with no matching data rows) and test-data generation.

38.4 PostgreSQL's :: Casting Shorthand

```
SELECT '42'::int + 8 AS result, price::text || ' USD' AS display FROM
products WHERE product_id = 1;

 result |  display
--------+-----------
     50 | 15.99 USD
(1 row)
```

`value::type` (Part 10.5) is PostgreSQL-specific shorthand for the standard `CAST(value AS type)` - functionally identical, used constantly throughout real PostgreSQL code for its brevity.

38.5 Extensions

PostgreSQL's **extension** system lets additional functionality be installed and enabled per-database, well beyond the built-in core - `uuid-ossp` (Part 22.6) and `vector`/`pgvector` (Part 36) used earlier in this course are both examples.

```
SELECT extname FROM pg_extension;

 extname
-----------
 plpgsql
 uuid-ossp
 vector
(3 rows)
```

`plpgsql` (Part 24) is installed by default in every database; `uuid-ossp` and `vector` were explicitly added in this course as needed. Other widely used extensions include `postgis` (geospatial data), `pg_trgm` (fuzzy text search), and `pg_stat_statements` (query performance monitoring) - this extensibility is one of PostgreSQL's most significant differentiators from most other relational databases, which typically require a completely separate specialized product for equivalent functionality.

38.6 Advanced Indexing (Recap)

GIN, GiST, and BRIN index types (Part 18.7-18.9), partial indexes (Part 18.11), and expression indexes (Part 18.12) are all far more developed in PostgreSQL than in most competing databases - many of which offer only B-tree and perhaps a basic full-text index type.

38.7 Why Each of These Features Matters

Feature	Why It's Valuable
`JSONB` + operators	First-class semi-structured data support without needing a separate NoSQL database
`ILIKE`	Simple, readable case-insensitive matching without wrapping both sides in `LOWER()`
`RETURNING`	Avoids a redundant round-trip `SELECT` after `INSERT`/`UPDATE`/`DELETE`
`FILTER`	Far more readable conditional aggregation than nested `CASE`
`DISTINCT ON`	Concise "first/latest row per group" queries
`LATERAL`	Enables per-row subqueries impossible with ordinary joins/subqueries
`generate_series`	Effortless calendar tables and gap-filling for reporting
`pgvector`	Native AI/embedding similarity search inside the primary application database

Exercises - Section 38

Beginner 1. Use `generate_series` to produce a list of the numbers 1 through 12 (representing calendar months).

Intermediate 2. Write a `LATERAL` join finding each product category's single most expensive product, and compare it to a `DISTINCT ON`-based solution (Part 23.5) for the same question.

Advanced 3. List every PostgreSQL-specific feature used anywhere in this course's example queries so far that you believe would require a meaningfully different approach in MySQL, and briefly describe what that different approach might look like (previewing Part 39). # 39. PostgreSQL vs Other Databases

39.1 Overview

Database	Type	License	Typical Strength
PostgreSQL	Full RDBMS	Open source (permissive)	Standards compliance, extensibility, rich data types
MySQL	Full RDBMS	Open source (GPL) / commercial	Simplicity, huge deployment base, read-heavy web apps
Microsoft SQL Server	Full RDBMS	Commercial	Enterprise Windows/.NET integration, BI tooling
Oracle Database	Full RDBMS	Commercial (often expensive licensing)	Very large enterprise deployments, mature tooling
SQLite	Embedded/serverless	Public domain	Zero-configuration, single-file, embedded use

39.2 Syntax Differences

Feature	PostgreSQL	MySQL	SQL Server	Oracle	SQLite
Auto-increment	`SERIAL` / `GENERATED ALWAYS AS IDENTITY`	`AUTO_INCREMENT`	`IDENTITY(1,1)`	`GENERATED ... AS IDENTITY` / sequences	`AUTOINCREMENT`
String concatenation	`||` or `CONCAT()`	`CONCAT()` (`||` means OR by default)	`+` or `CONCAT()`	`||`	`||`
Limiting rows	`LIMIT n OFFSET m`	`LIMIT n OFFSET m`	`TOP n` / `OFFSET...F`	`FETCH FIRST n`	`LIMIT n OFFSET m`

Feature	PostgreSQL	MySQL	SQL Server	Oracle	SQLite
			`ETCH`	`ROWS ONLY` / `ROWNUM`	
Case-insensitive match	`ILIKE`	`LIKE` (case-insensitivity depends on collation)	`LIKE` (collation-dependent)	`LIKE` with `UPPER()`/`LOWER()`	`LIKE` (ASCII case-insensitive by default)

39.3 Data Types

Type Need	PostgreSQL	MySQL	SQL Server	Oracle
Arbitrary text	`TEXT`	`TEXT` (has row-size limits historically)	`VARCHAR(MAX)`	`CLOB`
JSON	`JSON` and `JSONB` (indexable)	`JSON` (less rich indexing)	`NVARCHAR` + JSON functions (no native JSON type until recent versions)	`JSON` (recent versions)
Arrays	Native array types	Not natively supported	Not natively supported	`VARRAY`/nested tables (more complex)
Booleans	Native `BOOLEAN`	`TINYINT(1)` convention (no true boolean historically)	`BIT`	`NUMBER(1)` convention (no true boolean until recent versions)
UUID	Native `UUID` type	`CHAR(36)` convention	`UNIQUEIDENTIFIER`	`RAW(16)` convention

PostgreSQL's native support for arrays, `JSONB`, and `BOOLEAN` genuinely sets it apart - many of these require workarounds or conventions in other systems.

39.4 Performance

Aspect	Notes
Read-heavy simple queries	MySQL (particularly with the InnoDB or older MyISAM engines) has historically been tuned for straightforward, high-volume reads
Complex analytical queries	PostgreSQL's query planner and rich indexing (Part 18) generally handles complex joins/aggregations more capably
Write-heavy concurrent workloads	Both PostgreSQL and MySQL perform well; specific tuning and workload shape matter more than a blanket claim either way

Aspect	Notes
Extremely large enterprise workloads	Oracle and SQL Server offer mature, heavily-tuned enterprise features (partitioning, advanced clustering) refined over decades

General guidance: raw performance differences between modern PostgreSQL, MySQL, and SQL Server are often smaller in practice than differences caused by poor schema design, missing indexes, or unoptimized queries (Part 19) - the database engine choice matters less than using whichever one is chosen correctly.

39.5 Transactions

All four full RDBMSs (PostgreSQL, MySQL with InnoDB, SQL Server, Oracle) support full ACID transactions (Part 20.4). Historical caveat: MySQL's older MyISAM storage engine did **not** support transactions at all - modern MySQL deployments almost universally use InnoDB, which does. SQLite supports transactions but, being a single-file, embedded database, has more limited concurrent-write support than a true client-server RDBMS.

39.6 JSON Support

PostgreSQL's `JSONB` (Part 22), with its rich operator set (`->`, `->>`, `@>`, `?`) and GIN indexing, is widely regarded as the most capable native JSON implementation among traditional RDBMSs - MySQL's `JSON` type is functional but less richly indexed; SQL Server and Oracle added JSON support later and via a more function-call-heavy syntax rather than dedicated operators.

39.7 Analytics

Window functions (Part 15) are supported by PostgreSQL, MySQL (8.0+), SQL Server, and Oracle - all now reasonably comparable for standard analytical SQL. PostgreSQL's `FILTER` clause (Part 23.2), `DISTINCT ON` (Part 23.5), and native statistical functions (Part 37.2) offer more concise syntax for common analytical patterns than most competitors, which often require more verbose `CASE`-based workarounds.

39.8 Stored Procedures

Database	Procedural Language
PostgreSQL	PL/pgSQL (Part 24), plus PL/Python, PL/Perl, and others via extensions
MySQL	Its own more limited procedural SQL dialect
SQL Server	T-SQL
Oracle	PL/SQL (historically very mature and feature-rich)

PostgreSQL's ability to write functions in multiple procedural languages (not just its own) is relatively unusual among mainstream RDBMSs.

39.9 Indexing

PostgreSQL's index type variety (B-tree, Hash, GIN, GiST, BRIN, Part 18) meaningfully exceeds what MySQL and SQL Server offer out of the box - both of those are primarily B-tree-centric, with more limited support for specialized index types like GIN-equivalent inverted indexes for JSON/array data.

39.10 Security

All four full RDBMSs support role-based access control (Part 27), row-level and column-level permissions, and SSL/TLS connections. PostgreSQL's `GRANT/REVOKE` model and row-level security policies are broadly comparable in capability to SQL Server and Oracle's equivalent features, all considerably more granular than what SQLite (which has no built-in user/permission system at all - security is entirely the responsibility of the surrounding application and file-system permissions) can offer.

39.11 Scalability

- **Vertical scaling** (a bigger server): all four full RDBMSs scale well vertically.
- **Read replicas**: PostgreSQL, MySQL, and SQL Server all support native read-replica streaming replication.
- **Horizontal write scaling (sharding)**: historically a relative weak point for traditional PostgreSQL and MySQL alike compared to purpose-built distributed databases, though modern extensions (e.g., Citus for PostgreSQL) and managed cloud offerings have significantly closed this gap in recent years.

39.12 Cloud Usage

Every major cloud provider (AWS RDS/Aurora, Google Cloud SQL/AlloyDB, Azure Database) offers fully-managed PostgreSQL, MySQL, and SQL Server offerings; Oracle is available but often carries the heaviest licensing cost even in managed form. PostgreSQL has become an especially popular default choice for new cloud-native applications and startups specifically because of its open-source licensing, extensibility (Part 38.5), and strong standards compliance.

39.13 Use Cases by Role

Role	Why PostgreSQL Is Frequently Preferred
Data Analyst	Rich window functions (Part 15), `FILTER`, `DISTINCT ON`, and statistical functions (Part 37) simplify everyday analytical SQL
Data Scientist	Native array/JSON types simplify feature storage; direct `pgvector` support (Part 36) for embeddings
Data Engineer	Strong `COPY`/upsert (`ON CONFLICT`) support, extensibility, and standards-compliant SQL ease pipeline building (Part 35)
Backend Developer	Broad ORM/driver support, strong JSON

Role	Why PostgreSQL Is Frequently Preferred
	handling for API-style data, solid transactional guarantees
AI Engineer	`pgvector` (Part 36) allows a single database for both application data and vector search, avoiding a separate specialized vector database

39.14 Summary: Standard SQL vs PostgreSQL-Specific Syntax

Throughout this course, the following are **standard SQL**, portable to essentially any RDBMS: `SELECT/FROM/WHERE/GROUP BY/HAVING/ORDER BY`, all join types (Part 12), subqueries (Part 13), CTEs and window functions (Parts 14-15, with only minor syntax variation elsewhere), `UNION/INTERSECT/EXCEPT` (Part 16), and standard transaction control (Part 20).

The following are **PostgreSQL-specific or PostgreSQL-distinctive**, requiring adaptation for other databases (fully listed in Part 38.1): `JSONB` and its operators, native arrays, `ILIKE`, `RETURNING`, `DISTINCT ON`, `LATERAL`'s idiomatic everyday use, `generate_series`, `pgvector`, and PostgreSQL's specific index types beyond B-tree.

Exercises - Section 39

Beginner 1. Identify three features used in this course that would need to be rewritten to run on MySQL, and briefly describe the MySQL alternative.

Intermediate 2. Research how MySQL's `INSERT ... ON DUPLICATE KEY UPDATE` compares to PostgreSQL's `INSERT ... ON CONFLICT DO UPDATE` (Part 35.6), and write both versions of the same upsert statement.

Advanced 3. Write a short technical justification (as if presenting to a team choosing a database for a new AI-powered SaaS product) for choosing PostgreSQL over MySQL, referencing at least three specific features from this section. # 40. SQL Interview Preparation

40.1 Beginner Questions

Q1. What is the difference between SQL and PostgreSQL? SQL is a standardized query language; PostgreSQL is a specific database product that implements SQL, plus its own extensions (Part 1.7).

Q2. What is a primary key? A column (or set of columns) that uniquely identifies each row in a table; it cannot be `NULL` and cannot repeat (Part 5.2).

Q3. What is a foreign key? A column referencing another table's primary key, enforcing that relationships always point to something that genuinely exists (Part 5.3).

Q4. What does `SELECT DISTINCT` do? Removes duplicate rows from a query's result set (Part 7.5).

Q5. What is the difference between `WHERE` and `HAVING`? *(classic trap)* `WHERE` filters individual rows **before** grouping/aggregation; `HAVING` filters **groups**, after aggregation has already happened, and can reference aggregate functions directly, which `WHERE` cannot (Part 11.4).

Q6. What does `ORDER BY` do, and what is the default sort direction? Sorts the result set by one or more columns; the default direction is ascending (`ASC`) if not specified (Part 7.6).

Q7. What is the difference between `CHAR`, `VARCHAR`, and `TEXT`? `CHAR(n)` is fixed-length and blank-padded; `VARCHAR(n)` is variable-length with an enforced maximum; `TEXT` is variable-length with no limit and is PostgreSQL's idiomatic default (Part 4.3).

Q8. What does `NULL` represent? The absence of a known value - not zero, not an empty string, and not equal to anything, including another `NULL` (Part 8.6).

Q9. What is the difference between `DELETE`, `TRUNCATE`, and `DROP`? *(classic trap)* `DELETE` removes specific rows (optionally filtered by `WHERE`) and can be rolled back within a transaction; `TRUNCATE` removes **all** rows instantly without scanning them individually, also transactional, but resets identity/sequence counters; `DROP` removes the entire table structure itself, not just its data (Part 4.6-4.7, Part 9).

Q10. What does the `LIMIT` clause do? Restricts the number of rows returned by a query (Part 7.7).

40.2 Intermediate Questions

Q11. What is the difference between `INNER JOIN` and `LEFT JOIN`? *(classic trap)* `INNER JOIN` returns only rows with a match in both tables; `LEFT JOIN` returns every row from the left table, with `NULL`s filling in wherever no match exists on the right (Part 12.3-12.4).

Q12. What is the difference between `UNION` and `UNION ALL`? *(classic trap)* `UNION` combines two result sets and removes duplicates; `UNION ALL` combines them and keeps every row, including duplicates - `UNION ALL` is always faster since it skips deduplication (Part 16.1-16.2).

Q13. What is a primary key vs a unique key? *(classic trap)* Both enforce uniqueness, but a table can have only **one** primary key (which also disallows `NULL`), while it can have **multiple** unique constraints, and unique-constrained columns **can** contain `NULL` (multiple `NULL`s are permitted, since they aren't considered equal to each other) (Part 5.2, 5.4).

Q14. What is a CTE, and why use one instead of a subquery? A Common Table Expression (`WITH`) names an intermediate result, improving readability for multi-step logic - functionally similar to a subquery in many cases, but easier to read and reference multiple times (Part 14).

Q15. What is the difference between a subquery and a `JOIN`? A `JOIN` combines columns from multiple tables into one result; a subquery is often used just to filter or check for existence, without needing the other table's columns in the final output (Part 13.8).

Q16. Explain the `NOT IN` vs `NOT EXISTS` trap. *(classic trap)* If the subquery used with `NOT IN` can return even one `NULL`, the entire `NOT IN` condition becomes `NULL` (never `TRUE`) for

every row, due to three-valued logic - silently returning zero rows. `NOT EXISTS` never compares values directly and is not vulnerable to this trap, making it the safer default choice (Part 13.7).

Q17. What is an index, and why might a query still not use one? A separate structure enabling fast lookups without scanning every row (Part 18.1); a query might not use an available index if the table is small enough that a sequential scan is actually cheaper, or if the query wraps the indexed column in a function/expression that doesn't match the index (Part 18.5, 19.5).

Q18. What does `GROUP BY` require of the columns in `SELECT`? Every selected column that is not itself wrapped in an aggregate function must appear in `GROUP BY` (Part 11.2).

Q19. What is a self join, and give an example. A table joined to itself, useful for hierarchical relationships - e.g., joining `employees` to itself to pair each employee with their manager (Part 12.8).

Q20. What does the `RETURNING` clause do? Returns the affected row(s) directly from an `INSERT`/`UPDATE`/`DELETE` statement, avoiding a separate follow-up `SELECT` (Part 6.4, 9.4).

40.3 Advanced Questions

Q21. What is the difference between `RANK`, `DENSE_RANK`, and `ROW_NUMBER`? *(classic trap)* `ROW_NUMBER()` always assigns unique sequential numbers, breaking ties arbitrarily; `RANK()` gives tied rows the same rank but skips the next rank number(s); `DENSE_RANK()` gives tied rows the same rank without skipping any subsequent rank (Part 15.3).

Q22. What is a window function, and how does it differ from `GROUP BY`? A window function computes a value across a set of related rows without collapsing them - every original row survives in the output, unlike `GROUP BY`, which reduces many rows to one per group (Part 15.1).

Q23. What is a correlated subquery? A subquery that references a column from the outer query, conceptually re-evaluated once per outer row (Part 13.5).

Q24. Explain PostgreSQL's transaction isolation levels. `Read Committed` (default) prevents dirty reads; `Repeatable Read` additionally prevents non-repeatable reads; `Serializable` additionally prevents phantom reads, enforced by aborting conflicting transactions for the application to retry (Part 20.5).

Q25. What is a deadlock, and how does PostgreSQL handle it? Two transactions each holding a lock the other needs, unable to proceed; PostgreSQL automatically detects deadlocks and aborts one of the transactions rather than letting both hang indefinitely (Part 20.7).

Q26. What is the difference between a view and a materialized view? A view is recomputed fresh on every query; a materialized view stores a physical snapshot that must be explicitly refreshed to reflect new data (Part 17.6-17.7).

Q27. What does `EXPLAIN ANALYZE` show that plain `EXPLAIN` does not? `EXPLAIN` shows only the estimated plan; `EXPLAIN ANALYZE` actually executes the query and shows real timing and actual row counts alongside the estimates (Part 18.15).

Q28. What is normalization, and name the first three normal forms. The process of organizing tables to minimize duplication and avoid update anomalies. 1NF requires atomic column values; 2NF requires non-key columns to depend on the *entire* primary key; 3NF requires non-key columns to depend only on the primary key, not on other non-key columns (Part 21.7-21.9).

40.4 Expert Questions

Q29. Why does PostgreSQL need `VACUUM`, and what does `autovacuum` do? PostgreSQL's MVCC model leaves old row versions ("dead tuples") behind after `UPDATE`/`DELETE`; `VACUUM` reclaims that space, and `autovacuum` runs this automatically in the background once a table's dead-tuple count crosses a threshold (Part 28.8).

Q30. Explain the difference between a B-tree, GIN, and BRIN index, and when each is appropriate. B-tree (the default) handles equality, ranges, and sorting for typical scalar columns; GIN suits columns with multiple values per row (JSONB, arrays); BRIN is an extremely compact index ideal for huge tables where a column correlates with physical insertion order, like an append-only timestamp (Part 18.4, 18.7, 18.9).

Q31. What is the `NOT IN` + `NULL` trap, precisely, and how would you detect it in a code review? Covered fully in Q16; in review, flag any `NOT IN (subquery)` where the subquery's column is nullable, and recommend `NOT EXISTS` instead (Part 13.7).

Q32. What is `pgvector`, and what problem does it solve? A PostgreSQL extension adding a native vector data type and similarity-search operators, enabling embedding-based semantic search and RAG applications directly inside PostgreSQL rather than requiring a separate vector database (Part 36.3).

Q33. Explain the difference between `ETL` and `ELT`. ETL transforms data in a separate processing step before loading it into the target database; ELT loads raw data first and performs the transformation using the target database's own SQL engine (Part 35.1).

40.5 Data Analyst Questions

Q34. How would you calculate month-over-month growth in SQL? Using `LAG()` to access the previous period's value within the same query, then computing `(current - previous) / previous * 100` (Part 33.7).

Q35. What is RFM analysis? A customer-scoring technique based on Recency (how recently they purchased), Frequency (how often), and Monetary value (how much) (Part 33.4).

Q36. How would you find the median of a column in PostgreSQL? `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY column)` (Part 37.1).

40.6 Data Scientist Questions

Q37. How would you use SQL to prepare a training dataset for a churn model? Write a feature-engineering query producing one row per entity (e.g., customer) with engineered numeric features and a target label, filtering training data appropriately with `WHERE` (Part 34.2, 34.6).

Q38. Why might you store model predictions back in PostgreSQL rather than only in a Python session? So predictions become queryable alongside the rest of the business data by any application or dashboard, without needing to re-run the model (Part 34.4).

40.7 Data Engineer Questions

Q39. What is an upsert, and how is it implemented in PostgreSQL? Inserting a new row or updating it if a conflicting row already exists, via `INSERT ... ON CONFLICT (column) DO UPDATE SET ...` (Part 35.6).

Q40. What is a fact table vs a dimension table? A fact table holds numeric, event-level measurements with foreign keys to dimensions; a dimension table holds descriptive context (dates, products, customers) used to analyze facts (Part 35.4).

40.8 Backend Developer Questions

Q41. How do you prevent SQL injection when using PostgreSQL from an application? Always use parameterized queries (placeholders like `%s` or named parameters), never build SQL by directly formatting untrusted input into a string (Part 31.4).

Q42. What is connection pooling, and why does it matter? Reusing a set of already-open database connections rather than opening a new one for every request, reducing the overhead of repeated connection setup (Part 31.7, Part 28.3).

Q43. How would you safely perform a multi-step operation (e.g., inserting an order and its line items) from application code? Wrap both operations in a single transaction, committing only if all steps succeed and rolling back entirely otherwise (Part 20.2, Part 31.5).

Exercises - Section 40

1. Without looking at the answers, write your own one-sentence answer to Q16 (`NOT IN` vs `NOT EXISTS`) and Q21 (`RANK` vs `DENSE_RANK` vs `ROW_NUMBER`), then compare.
2. Pick five questions from different categories and practice explaining each out loud in under 60 seconds.
3. Write one additional interview question (with a complete answer) for a category you found personally challenging. # 41. SQL Coding Challenges

Every challenge uses this course's schema and dataset (Part 4, Part 6). Each includes the question, the solution, and a brief explanation; try solving before reading the solution.

41.1 Beginner Challenges

Challenge 1. List all products priced above $100.

```
SELECT product_name, price FROM products WHERE price > 100;

        product_name         | price
-----------------------------+--------
 Noise Cancelling Headphones | 129.99
(1 row)
```

Challenge 2. Find the single most expensive product.

```
SELECT product_name FROM products WHERE price = (SELECT MAX(price)
FROM products);

        product_name
----------------------------
 Noise Cancelling Headphones
(1 row)
```

Explanation: a scalar subquery (Part 13.2) computes the maximum once; the outer query finds whichever row matches it.

Challenge 3. List all customers from Nigeria or Kenya.

```
SELECT name, country FROM customers WHERE country IN ('Nigeria',
'Kenya') ORDER BY name;
```

Challenge 4. Count how many products exist in each category.

```
SELECT category, COUNT(*) FROM products GROUP BY category ORDER BY
category;
```

Challenge 5. Find every store located in `NULL` city (i.e., online-only stores).

```
SELECT store_name FROM stores WHERE city IS NULL;

  store_name
--------------
 Online Store
(1 row)
```

Challenge 6. List the 3 most recently placed orders.

```
SELECT order_id, order_date FROM orders ORDER BY order_date DESC LIMIT
3;
```

Challenge 7. Find all employees earning more than $60,000.

```
SELECT full_name, salary FROM employees WHERE salary > 60000 ORDER BY
salary DESC;
```

41.2 Intermediate Challenges

Challenge 8. Find the product category with the most products.

```
SELECT category, COUNT(*) AS cnt
FROM products
GROUP BY category
HAVING COUNT(*) = (SELECT MAX(cnt) FROM (SELECT COUNT(*) AS cnt FROM
products GROUP BY category) t);
```

```
  category   | cnt
-------------+-----
 Electronics |   4
(1 row)
```

Explanation: the inner subquery computes each category's count; the outer `HAVING` keeps only the category matching the overall maximum.

Challenge 9. List every customer who has placed more than one order.

```
SELECT c.name
FROM customers c
WHERE (SELECT COUNT(*) FROM orders o WHERE o.customer_id =
c.customer_id) > 1;
```

```
    name
--------------
 Ada Lovelace
 Grace Hopper
 Chidi Okafor
 John Smith
(4 rows)
```

Challenge 10. Find every product that has never appeared in an order.

```
SELECT product_name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id =
p.product_id);
```

```
(0 rows)
```

Explanation: every product in this dataset has been ordered at least once - a correct, verified "no matches" result, not a bug.

Challenge 11. Find every store that has never had an order placed at it.

```
SELECT store_name FROM stores s
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.store_id =
s.store_id);
```

```
(0 rows)
```

Challenge 12. List each customer's total spend, but only for customers who have spent more than $200.

```
SELECT c.name, SUM(p.amount) AS total_spent
FROM customers c
```

```
JOIN orders o ON o.customer_id = c.customer_id
JOIN payments p ON p.order_id = o.order_id
GROUP BY c.name
HAVING SUM(p.amount) > 200;
```

```
     name     |  total_spent
--------------+---------------
 Ada Lovelace |        215.45
 John Smith   |        234.97
(2 rows)
```

Challenge 13. Rank all products by price, from most to least expensive.

```
SELECT product_name, price, RANK() OVER (ORDER BY price DESC) AS rnk
FROM products;
```

Challenge 14. Find each order's total value (quantity x unit_price, summed across its line items), showing the 3 highest-value orders.

```
SELECT o.order_id, SUM(oi.quantity * oi.unit_price) AS total
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id
ORDER BY total DESC
LIMIT 3;
```

```
 order_id | total
----------+--------
       12 | 145.98
        6 | 129.99
       15 | 104.98
(3 rows)
```

Challenge 15. Find every pair of employees who work in the same department (avoiding duplicate pairs).

```
SELECT e1.full_name AS employee_1, e2.full_name AS employee_2
FROM employees e1
JOIN employees e2 ON e1.department_id = e2.department_id AND
e1.employee_id < e2.employee_id;
```

Explanation: `e1.employee_id < e2.employee_id` is the standard trick to avoid both duplicate pairs (A,B and B,A) and self-pairing (A,A) in a self-join.

Challenge 16. Find the average number of items per order.

```
SELECT AVG(quantity) FROM order_items;
```

```
        avg
--------------------
 1.2727272727272727
```

Challenge 17. List departments with no employees at all.

```
SELECT department_name FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.department_id =
d.department_id);

(0 rows)
```

41.3 Advanced Challenges

Challenge 18. For each category, find the single cheapest and single most expensive product (in one query).

```
SELECT DISTINCT ON (category, is_max) category, is_max, product_name,
price
FROM (
    SELECT category, product_name, price,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY price
ASC) = 1 AS is_min,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY price
DESC) = 1 AS is_max
    FROM products
) ranked
WHERE is_min OR is_max
ORDER BY category, is_max;
```

```
  category   | is_max |        product_name         | price
-------------+--------+-----------------------------+--------
 Accessories | t      | Leather Wallet              |  29.99
 Books       | f      | Python Programming Book     |  34.95
 Books       | t      | SQL Cookbook                |  42.00
 Clothing    | f      | Cotton T-Shirt              |  12.50
 Clothing    | t      | Denim Jeans                 |  39.99
 Electronics | f      | Wireless Mouse              |  15.99
 Electronics | t      | Noise Cancelling Headphones | 129.99
 Home        | f      | Ceramic Mug Set             |  18.99
 Home        | t      | Handwoven Basket            |  34.99
(9 rows)
```

Challenge 19. Compute each customer's running total of payments, ordered by payment date.

```
SELECT c.name, p.payment_date, p.amount,
       SUM(p.amount) OVER (PARTITION BY c.customer_id ORDER BY
p.payment_date) AS running_total
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN payments p ON p.order_id = o.order_id
ORDER BY c.name, p.payment_date;
```

Challenge 20. Find the second-highest-priced product overall.

```
SELECT product_name, price
FROM (SELECT product_name, price, DENSE_RANK() OVER (ORDER BY price
```

```
DESC) AS rnk FROM products) ranked
WHERE rnk = 2;

      product_name       | price
-------------------------+-------
 Mechanical Keyboard     | 79.99
(1 row)
```

Explanation: DENSE_RANK (Part 15.3) correctly handles the case where the top price might be tied across multiple products, unlike a naive `LIMIT 1 OFFSET 1`.

Challenge 21. Find every customer whose total spend is above the overall average customer spend.

```
WITH customer_totals AS (
    SELECT c.customer_id, c.name, COALESCE(SUM(p.amount), 0) AS
total_spent
    FROM customers c
    LEFT JOIN orders o ON o.customer_id = c.customer_id
    LEFT JOIN payments p ON p.order_id = o.order_id
    GROUP BY c.customer_id, c.name
)
SELECT name, total_spent
FROM customer_totals
WHERE total_spent > (SELECT AVG(total_spent) FROM customer_totals)
ORDER BY total_spent DESC;
```

Challenge 22. For each month, show revenue and the percentage change from the previous month.

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', payment_date)::date AS month,
SUM(amount) AS revenue
    FROM payments GROUP BY DATE_TRUNC('month', payment_date)::date
)
SELECT month, revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month)) /
LAG(revenue) OVER (ORDER BY month), 1) AS pct_change
FROM monthly ORDER BY month;
```

Challenge 23. Find every employee who earns more than their own manager.

```
SELECT e.full_name AS employee, e.salary, m.full_name AS manager,
m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;

(0 rows)
```

Explanation: correctly returns no rows in this dataset - every employee earns less than their manager here.

Challenge 24. List the top 2 best-selling products (by revenue, not units) per category.

```
WITH product_revenue AS (
    SELECT p.category, p.product_name, SUM(oi.quantity *
oi.unit_price) AS revenue,
           ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY
SUM(oi.quantity * oi.unit_price) DESC) AS rn
    FROM products p
    JOIN order_items oi ON oi.product_id = p.product_id
    GROUP BY p.category, p.product_name
)
SELECT category, product_name, revenue FROM product_revenue WHERE rn
<= 2 ORDER BY category, rn;
```

Challenge 25. Find all customers who have ordered from more than one distinct store.

```
SELECT c.name
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.name
HAVING COUNT(DISTINCT o.store_id) > 1;

(0 rows)
```

Explanation: correctly empty - as established in Part 16.3, no customer in this dataset has ordered from more than one store.

Challenge 26. Write a recursive query listing every employee under `Efe Johnson` in the management chain, with their depth level.

```
WITH RECURSIVE reports AS (
    SELECT employee_id, full_name, manager_id, 1 AS depth
    FROM employees WHERE full_name = 'Efe Johnson'
    UNION ALL
    SELECT e.employee_id, e.full_name, e.manager_id, r.depth + 1
    FROM employees e JOIN reports r ON e.manager_id = r.employee_id
)
SELECT full_name, depth FROM reports ORDER BY depth, full_name;
```

Challenge 27. Find the median order value across all completed orders.

```
WITH order_totals AS (
    SELECT o.order_id, SUM(oi.quantity * oi.unit_price) AS total
    FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
    WHERE o.status = 'completed'
    GROUP BY o.order_id
)
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total) AS
median_order_value FROM order_totals;
```

Challenge 28. Identify products whose price is more than one standard deviation above their category's average price.

```sql
WITH category_stats AS (
    SELECT category, AVG(price) AS avg_price, STDDEV(price) AS
stddev_price
    FROM products GROUP BY category
)
SELECT p.product_name, p.category, p.price
FROM products p
JOIN category_stats cs ON cs.category = p.category
WHERE p.price > cs.avg_price + cs.stddev_price;
```

Exercises - Section 41

1. Solve each challenge above yourself, without looking at the solution, before checking your answer.
2. Pick three "Advanced" challenges and write an alternative solution using a different technique (e.g., a window function instead of a correlated subquery, or vice versa) than the one shown.
3. Design one additional challenge of your own, at "Advanced" difficulty, using at least three of this course's tables together. # 42. Complete Real-World Projects

Each project below follows the same structure: business problem, requirements, schema design, sample queries, and key insights. All SQL patterns used follow directly from Parts 1-39; only the highlights are shown here for space, following the same verified techniques used throughout this course.

42.1 Project 1 - Sales Management Database

Business problem: a retailer needs to track products, customers, orders, and sales performance across regions. **Requirements:** support order entry, inventory tracking, and regional sales reporting. **Core tables:** `customers`, `products`, `orders`, `order_items`, `regions` (this course's schema, Part 4, is itself a direct instance of this project). **Key relationships:** customers (1) -> (M) orders (1) -> (M) order_items (M) -> (1) products. **Sample analytical query:** revenue by region and month, using `DATE_TRUNC` (Part 10.3) and `JOIN` (Part 12) - directly demonstrated throughout Part 33. **Performance consideration:** index `orders(customer_id)` and `order_items(order_id)` (Part 18.10) for fast lookups as order volume grows. **Business insight:** identifying which regions/months drive disproportionate revenue directs marketing spend toward what's actually working.

42.2 Project 2 - E-commerce Database

Business problem: an online store needs product catalogs, shopping carts, orders, reviews, and shipping. **Requirements:** support multiple product categories, customer reviews, and order status tracking. **Core tables:** `products`, `categories`, `customers`, `orders`, `order_items`, `reviews`, `shipments`.

```sql
CREATE TABLE reviews (
    review_id   SERIAL PRIMARY KEY,
    product_id  INTEGER NOT NULL REFERENCES products(product_id),
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
    rating      SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
```

```
    comment     TEXT,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Sample analytical query: average rating per product, joined with sales volume, to find "loved but under-marketed" products (high rating, low sales). **Performance consideration:** a GIN index (Part 18.7) on a `JSONB` product-attributes column if the catalog has widely varying, category-specific specs. **Business insight:** correlating review sentiment with repeat-purchase rate can validate whether review scores actually predict customer loyalty.

42.3 Project 3 - Banking Database

Business problem: a bank needs to track accounts, transactions, and balances with strict correctness guarantees. **Requirements:** every balance change must be transactionally safe (Part 20); no transaction may leave an account in an inconsistent state.

```
CREATE TABLE accounts (
    account_id SERIAL PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
    balance NUMERIC(12, 2) NOT NULL CHECK (balance >= 0)
);

CREATE TABLE transactions (
    transaction_id SERIAL PRIMARY KEY,
    account_id INTEGER NOT NULL REFERENCES accounts(account_id),
    amount NUMERIC(12, 2) NOT NULL,
    transaction_type TEXT NOT NULL CHECK (transaction_type IN
('deposit', 'withdrawal', 'transfer')),
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Sample query pattern: a funds transfer between two accounts, wrapped in a single transaction (Part 20.2) so a failure partway through never leaves money "created" or "destroyed." **Performance consideration:** `Serializable` isolation (Part 20.5) for any logic checking-then-updating a balance, to prevent race conditions between concurrent transfers. **Business insight:** the `CHECK (balance >= 0)` constraint (Part 5.6) provides a database-level guarantee against overdrafts, independent of application-layer bugs.

42.4 Project 4 - Hospital Management Database

Business problem: track patients, doctors, appointments, and treatment records. **Core tables:** `patients`, `doctors`, `appointments`, `treatments`, `departments` (a direct extension of Part 21.12's veterinary-clinic design pattern). **Sample analytical query:** average appointment wait time per department, using `AGE()` (Part 37.4) between scheduled and actual appointment times. **Performance consideration:** a partial index (Part 18.11) on `appointments(patient_id) WHERE status = 'scheduled'` for fast lookup of a patient's upcoming appointments specifically. **Business insight:** identifying departments with unusually long wait times highlights where additional staffing would have the greatest impact.

42.5 Project 5 - HR Database

Business problem: track employees, departments, salaries, and organizational hierarchy. **Core tables:** this course's `employees`/`departments` tables (Part 4, Part 6) already directly implement this project, including the self-referencing manager hierarchy (Part 12.8). **Sample analytical query:** the recursive org-chart query from Part 14.5, extended with salary rollups per management level. **Performance consideration:** an index on `employees(manager_id)` to speed up the recursive CTE's join condition at scale. **Business insight:** comparing salary distributions across departments and hierarchy levels can surface unintentional pay-equity gaps.

42.6 Project 6 - Inventory Management System

Business problem: track stock levels across multiple warehouses/stores, triggering reorders when stock runs low. **Core tables:** `products`, `stores` (Part 4), extended with a `stock_by_location` table and a reorder-trigger pattern (Part 25).

```
CREATE TABLE stock_by_location (
    product_id INTEGER REFERENCES products(product_id),
    store_id INTEGER REFERENCES stores(store_id),
    quantity INTEGER NOT NULL CHECK (quantity >= 0),
    reorder_threshold INTEGER NOT NULL DEFAULT 10,
    PRIMARY KEY (product_id, store_id)
);
```

Sample analytical query: `SELECT * FROM stock_by_location WHERE quantity < reorder_threshold;` - immediately surfacing every product/location combination needing restocking. **Performance consideration:** a trigger (Part 25) that automatically inserts into a `reorder_requests` table whenever an `UPDATE` drops `quantity` below `reorder_threshold`. **Business insight:** tracking reorder frequency by product identifies which items have chronically undersized safety stock levels.

42.7 Project 7 - Financial Analytics Database

Business problem: support finance-team reporting on revenue, expenses, and profitability trends. **Core tables:** this course's `payments`/`orders` tables, extended conceptually with an `expenses` table mirroring the same structure. **Sample analytical query:** monthly revenue, expenses, and net margin, using the `DATE_TRUNC` + `LAG()` growth pattern from Part 33.7, applied to both revenue and expense series simultaneously via a shared CTE. **Performance consideration:** a materialized view (Part 17.6) refreshed nightly for the finance dashboard's primary monthly summary, since finance reporting rarely needs real-time freshness. **Business insight:** month-over-month margin trends (not just revenue growth alone) reveal whether growth is genuinely profitable or increasingly costly to sustain.

42.8 Project 8 - Customer Intelligence Database

Business problem: build a 360-degree view of customer behavior for marketing and retention. **Core tables:** this course's `customers`, `orders`, `payments` tables, combined with the RFM (Part 33.4), segmentation (Part 33.3), and cohort (Part 33.6) queries already built throughout

this course. **Sample analytical query:** a single view combining RFM scores, cohort membership, and churn status (Part 33.5) per customer - a genuine "customer intelligence" table ready for a CRM or marketing tool to consume directly. **Performance consideration:** materialize this combined view (Part 17.6), since it aggregates across the entire order history and is expensive to recompute on every dashboard load. **Business insight:** cross-referencing RFM segment with cohort reveals whether *newer* cohorts convert into high-value customers faster or slower than older ones - a leading indicator of whether recent marketing/product changes are working.

42.9 Project 9 - Data Analytics Warehouse Project

Business problem: consolidate operational data into an analytics-friendly warehouse structure for BI tooling. **Core tables:** a star schema (Part 35.4) with `fact_sales`, `dim_date`, `dim_customer`, `dim_product` - directly extending the exact pattern built and verified in Part 35.4. **Sample analytical query:** revenue by date, customer segment, and product category in a single `GROUP BY`, made fast by the pre-joined, denormalized dimension structure. **Performance consideration:** BRIN indexes (Part 18.9) on the fact table's date-key column, since fact tables are typically loaded in chronological order. **Business insight:** a warehouse structure lets BI tools (Tableau, Power BI, Metabase) build interactive dashboards without needing to understand the full operational schema's join complexity.

42.10 Project 10 - AI/RAG Database Project

Business problem: power a customer-support chatbot that answers questions using the company's own knowledge base. **Core tables:** `knowledge_base_documents` with a `pgvector` embedding column (Part 36.8), plus `chat_sessions/chat_messages` (Part 36.1) for conversation history. **Sample analytical query:** the similarity-search query from Part 36.4, retrieving the most relevant documents for a given question's embedding. **Performance consideration:** an `ivfflat` or `hnsw` vector index (Part 36.5) once the knowledge base grows beyond a few thousand documents. **Business insight:** logging which documents get retrieved most often (Section 36's exercise 3) reveals both the most common customer questions and any gaps in the knowledge base where no good document exists.

Exercises - Section 42

1. Pick any two projects above and sketch their complete ER diagrams (Part 21.5) in text form.
2. Implement Project 6 (Inventory Management) fully, including the reorder trigger, and test it by updating stock below the threshold.
3. Combine Project 8 (Customer Intelligence) and Project 9 (Data Warehouse) into a single design, explaining how a customer-intelligence view could itself become a dimension table in a star schema. # 43. Final Capstone Project

43.1 Overview

This capstone combines nearly every technique from this course into a single, portfolio-quality project: a **Multi-Channel Retail Analytics Platform**, built directly on top of this course's

schema and dataset, extended with additional schemas, functions, triggers, and Python integration.

43.2 Requirements Checklist

- ☐ Multiple schemas separating operational and analytical concerns
- ☐ 10+ related tables with proper constraints
- ☐ Indexes appropriate to real query patterns
- ☐ At least one view and one materialized view
- ☐ CTEs and window functions in the analytical layer
- ☐ At least one stored function and one trigger
- ☐ Transactional integrity for a multi-step business operation
- ☐ JSONB used for genuinely semi-structured data
- ☐ Python + pandas producing a dashboard-ready dataset

43.3 Step 1 - Multiple Schemas

```
CREATE SCHEMA IF NOT EXISTS analytics;
```

`public` (Part 4.2) holds the operational tables already built in Part 4 and Part 6 (`customers`, `orders`, `products`, etc.); `analytics` holds derived, reporting-oriented objects - a clean separation between "the data" and "views onto the data," directly following Part 21's design principles.

43.4 Step 2 - The Analytical Layer: Views and Materialized Views

```
CREATE MATERIALIZED VIEW analytics.monthly_summary AS
SELECT DATE_TRUNC('month', payment_date)::date AS month, SUM(amount)
AS revenue, COUNT(*) AS payment_count
FROM payments
GROUP BY DATE_TRUNC('month', payment_date)::date;

SELECT * FROM analytics.monthly_summary ORDER BY month;
```

```
   month    | revenue | payment_count
------------+---------+---------------
 2025-01-01 |  192.44 |             3
 2025-02-01 |  212.47 |             3
 2025-03-01 |  429.34 |             5
 2025-04-01 |  153.96 |             2
(4 rows)
```

This is exactly Part 17.6's materialized view technique, now placed deliberately in the `analytics` schema (Part 43.3) rather than mixed in with operational tables.

43.5 Step 3 - Combining CTEs and Window Functions

```
CREATE VIEW analytics.customer_intelligence AS
WITH customer_metrics AS (
    SELECT c.customer_id, c.name, c.country,
           COUNT(DISTINCT o.order_id) AS total_orders,
           COALESCE(SUM(p.amount), 0) AS total_spent
```

```
    FROM customers c
    LEFT JOIN orders o ON o.customer_id = c.customer_id
    LEFT JOIN payments p ON p.order_id = o.order_id
    GROUP BY c.customer_id, c.name, c.country
)
SELECT *,
       NTILE(3) OVER (ORDER BY total_spent DESC) AS value_segment,
       RANK() OVER (ORDER BY total_spent DESC) AS spend_rank
FROM customer_metrics;

SELECT * FROM analytics.customer_intelligence ORDER BY spend_rank
LIMIT 3;

 customer_id |     name     |    country     | total_orders |
total_spent | value_segment | spend_rank
-------------+--------------+----------------+--------------
+-------------+---------------+------------
           5 | John Smith   | United States  |            2 |
234.97 |             1 |          1
           1 | Ada Lovelace | Nigeria        |            3 |
215.45 |             1 |          2
           2 | Grace Hopper | United Kingdom |            2 |
175.97 |             1 |          3
(3 rows)
```

This view combines the Part 14 CTE pattern with Part 15's window functions, directly extending Part 42.8's Customer Intelligence project into a genuinely reusable, queryable object.

43.6 Step 4 - A Stored Function

```
CREATE FUNCTION analytics.customer_lifetime_value(cust_id INTEGER)
RETURNS NUMERIC AS $$
DECLARE
    ltv NUMERIC;
BEGIN
    SELECT COALESCE(SUM(p.amount), 0) INTO ltv
    FROM orders o JOIN payments p ON p.order_id = o.order_id
    WHERE o.customer_id = cust_id;
    RETURN ltv;
END;
$$ LANGUAGE plpgsql;

SELECT analytics.customer_lifetime_value(1);

 customer_lifetime_value
-------------------------
                  215.45
(1 row)
```

Directly following Part 24's PL/pgSQL patterns, packaging a calculation used repeatedly throughout the analytics layer into a single, reusable function.

43.7 Step 5 - A Trigger for Data Integrity

```
CREATE FUNCTION public.set_updated_at() RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

ALTER TABLE public.orders ADD COLUMN updated_at TIMESTAMPTZ DEFAULT
NOW();
CREATE TRIGGER trg_orders_updated_at
BEFORE UPDATE ON public.orders
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
```

Directly following Part 25's timestamp-maintenance trigger pattern, now applied to `orders` specifically.

43.8 Step 6 - Transactional Integrity for a Multi-Step Operation

```
BEGIN;
INSERT INTO public.orders (customer_id, store_id, status) VALUES (1,
2, 'pending') RETURNING order_id;
-- (application code captures the returned order_id, then inserts line
items referencing it)
INSERT INTO public.order_items (order_id, product_id, quantity,
unit_price)
VALUES (16, 3, 1, 24.99);
COMMIT;

BEGIN
 order_id
----------
       16
(1 row)

INSERT 0 1
INSERT 0 1
COMMIT
```

Placing the order and its line item(s) inside a single transaction (Part 20.2) guarantees an order is never left without any line items due to a failure partway through - directly applying Part 20's core lesson to this course's actual schema.

43.9 Step 7 - JSONB for Semi-Structured Data

```
ALTER TABLE public.orders ADD COLUMN order_metadata JSONB;

UPDATE public.orders
SET order_metadata = '{"channel": "mobile_app", "campaign":
"spring_sale", "referrer": "email"}'
WHERE order_id = 1;
```

```
SELECT order_id, order_metadata ->> 'channel' AS channel FROM
public.orders WHERE order_id = 1;

 order_id |  channel
----------+------------
        1 | mobile_app
(1 row)
```

Following Part 22's JSONB patterns, capturing genuinely variable marketing-attribution data that would be awkward to model as rigid, ever-changing columns.

43.10 Step 8 - Python + Pandas: A Dashboard-Ready Dataset

```
import pandas as pd
from sqlalchemy import create_engine

engine =
create_engine("postgresql+psycopg2://postgres:postgres@localhost:5432/
company_db")

dashboard_data = pd.read_sql("SELECT * FROM
analytics.customer_intelligence ORDER BY spend_rank;", engine)
dashboard_data.to_csv("customer_intelligence_dashboard.csv",
index=False)
print(dashboard_data.head())
```

This final step - reading the `analytics.customer_intelligence` view (Section 43.5) directly into pandas and exporting a ready-to-visualize CSV - is exactly the PostgreSQL-to-Python bridge from Part 32, completing the pipeline from raw relational data all the way to a dashboard-ready artifact.

43.11 What This Capstone Demonstrates

Requirement	Where Satisfied
Multiple schemas	`public` + `analytics` (Section 43.3)
10+ related tables	This course's full schema (Part 4), plus capstone extensions
Constraints and indexes	Inherited from Part 4-5, extended per Part 18
View + materialized view	Sections 43.4-43.5
CTEs and window functions	Section 43.5
Stored function	Section 43.6
Trigger	Section 43.7
Transactional integrity	Section 43.8
JSONB	Section 43.9
Python + pandas integration	Section 43.10

Capstone Exercise

Complete every step in this part yourself, on your own running PostgreSQL instance, then extend it with **one** original addition of your own choosing (a new analytical view, an additional trigger, a `pgvector`-based feature, or an additional Python-based model as in Part 34) - documenting your schema, your reasoning, and your final insights, exactly as a professional portfolio project would. # 44. Teaching Structure

This short section makes explicit the structure this entire course has followed for every major concept, so instructors and self-learners alike can use it as a template for further study beyond this course.

44.1 The Eleven-Part Teaching Structure

Every substantial concept in this course - from `SELECT` (Part 7) through window functions (Part 15) to `pgvector` (Part 36) - was deliberately introduced using the same eleven-part structure:

1. **Definition** - a precise, jargon-free statement of what the concept is.
2. **Why it matters** - the real problem it solves, not just its syntax.
3. **Syntax** - the general shape of the SQL involved.
4. **Simple example** - a minimal, concrete demonstration.
5. **Line-by-line explanation** - what each clause/keyword actually does.
6. **Expected output** - real, verified results (every query in this course was executed against a real, running PostgreSQL 16 instance with the dataset from Part 6 - not hand-typed guesses).
7. **Real-world example** - applied to this course's realistic business database.
8. **Common mistakes** - specific, verified failure modes and their exact error messages.
9. **Best practices** - what experienced professionals actually do.
10. **Practice exercises** - beginner, intermediate, and advanced problems per section.
11. **Advanced example** - a more sophisticated application of the same concept.

44.2 Why This Structure, Specifically

Learning SQL purely from a syntax reference produces someone who can recognize `SELECT ... FROM ... WHERE ...` but cannot reliably decide *when* to reach for a window function versus a `GROUP BY`, or *why* `NOT IN` is dangerous in a way `NOT EXISTS` is not (Part 13.7). Pairing every piece of syntax with **why it exists**, **what commonly goes wrong**, and **a real, verified example** is what turns syntax recognition into professional judgment - the actual goal of this course, stated explicitly in its own introduction.

44.3 Using This Structure for Continued Learning

Any new PostgreSQL feature encountered after this course - a new extension, a newly added SQL feature in a future PostgreSQL release - can be learned effectively using this same eleven-part checklist: define it precisely, understand the problem it solves, find its syntax, write and run a minimal example, explain it line by line, verify the actual output, apply it to a real dataset,

seek out its common pitfalls, learn the professional consensus on best practice, practice deliberately, and finally push into a more advanced use case. This structure is the course's real, lasting takeaway - more durable than any single piece of syntax it was used to teach. # 45. SQL Best Practices

This part consolidates best-practice guidance referenced throughout the course into a single, comprehensive reference.

45.1 Readability

- Capitalize SQL keywords (`SELECT`, `FROM`, `WHERE`) consistently; use lowercase for identifiers (Part 3.7-3.8).
- Put each major clause on its own line for anything beyond a trivially short query.
- Prefer explicit, descriptive names over abbreviations - `customer_id`, not `cid`.

45.2 Naming Conventions

- Use `snake_case` for tables and columns (`order_items`, not `OrderItems` or `orderItems`) - avoids needing double-quoted, case-sensitive identifiers entirely (Part 3.3).
- Name tables as plural nouns (`customers`, `orders`) and primary keys as `tablename_id` (`customer_id`) - exactly this course's schema (Part 4).
- Name foreign keys identically to the primary key they reference (`orders.customer_id` referencing `customers.customer_id`) - makes join conditions self-evident.

45.3 Formatting

- Consistent indentation for subqueries and CTEs, so nesting depth is visually obvious (Part 13-14).
- Align `JOIN/ON` pairs so join logic is easy to scan (Part 12).
- One column per line in a `SELECT` list once it exceeds roughly 3-4 columns.

45.4 Aliases

- Always alias tables in multi-table queries (`c` for `customers`, `o` for `orders`), and use those aliases consistently throughout (Part 12.3).
- Always alias computed/aggregate columns (`AS total_spent`, not an unreadable default column name).

45.5 Avoiding `SELECT *`

- Name columns explicitly in any query used by an application, report, or another person - `SELECT *` breaks silently when the table's structure changes, and wastes bandwidth on unused columns (Part 7.2).
- Reserve `SELECT *` for genuinely quick, throwaway, interactive checks only.

45.6 Safe Updates and Deletes

- Always write and test the equivalent `SELECT` with the same `WHERE` clause before running an `UPDATE/DELETE` (Part 9.6).

- Wrap risky statements in an explicit transaction (`BEGIN` ... `ROLLBACK`/`COMMIT`, Part 20.2) so mistakes can be undone before becoming permanent.
- Use `RETURNING` (Part 6.4, 9.4) to immediately confirm exactly what changed.

45.7 Parameterized Queries

- Never build SQL by directly formatting untrusted values into a string, from any language - always use parameter placeholders (Part 31.4) to prevent SQL injection.
- This applies identically whether using `psycopg2` (Part 31), SQLAlchemy/pandas (Part 32), or any other driver/ORM.

45.8 Indexing

- Index columns actually used in `WHERE`, `JOIN`, and `ORDER BY` for frequent, important queries (Part 18.2-18.3).
- Verify an index is actually being used with `EXPLAIN` (Part 18.15) rather than assuming - especially on small tables, where a sequential scan is often genuinely faster (Part 18.5).
- Avoid wrapping indexed columns in functions/casts within `WHERE` clauses, which prevents the planner from using a plain index on that column (Part 19.5).

45.9 Query Optimization

- Use `EXPLAIN ANALYZE` (Part 18.15, 19) to diagnose slow queries with real evidence, not guesswork.
- Keep an eye on whether the planner's estimated row counts roughly match actual counts - large discrepancies suggest stale statistics (Part 28.9).
- Prefer `UNION ALL` over `UNION`, and `NOT EXISTS` over `NOT IN` with nullable subqueries (Parts 13.7, 16.2), for both correctness and performance.

45.10 Transactions

- Wrap any multi-step operation that must succeed or fail together in an explicit transaction (Part 20.2).
- Keep transactions as short as possible to minimize lock contention (Part 20.8).
- Always have a plan for retrying a transaction that fails due to a serialization conflict or deadlock (Part 20.5, 20.7).

45.11 Data Validation

- Enforce data-quality rules with constraints (`NOT NULL`, `CHECK`, foreign keys, Part 5) at the database level, not only in application code - constraints apply universally, regardless of which application or script writes the data.
- Validate staged data explicitly before promoting it into permanent tables in any pipeline (Part 35.3).

45.12 Security

- Apply the principle of least privilege (Part 27.6) - grant only what a role genuinely needs.
- Grant privileges to group roles, not individuals directly, and grant individuals membership in those groups (Part 27.1).

- Never hardcode database credentials directly in source code; use environment variables or a secrets manager.

45.13 Maintainability

- Prefer views (Part 17) for logic reused across many queries, so the definition lives in exactly one place.
- Document non-obvious business logic directly in SQL comments (Part 3.5), especially inside functions and triggers (Parts 24-25).
- Favor standard, portable SQL (Part 39.14) unless a PostgreSQL-specific feature provides a clear, deliberate advantage worth the reduced portability.

Exercises - Section 45

1. Take a query you wrote earlier in this course and review it against this section's checklist, identifying at least two improvements.
2. Explain, in your own words, why "safe updates and deletes" and "transactions" are closely related best practices.
3. Write a short code-review checklist (5-8 items) you would personally use when reviewing a colleague's SQL pull request, drawing from this section. # 46. Final Cheat Sheet

46.1 Database and Schema

```
CREATE DATABASE dbname;
DROP DATABASE dbname;
CREATE SCHEMA schemaname;
```

46.2 Table Creation and Data Types

```
CREATE TABLE t (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    price NUMERIC(10,2) CHECK (price >= 0),
    created_at TIMESTAMPTZ DEFAULT NOW()
);
ALTER TABLE t ADD COLUMN col TYPE;
ALTER TABLE t DROP COLUMN col;
DROP TABLE t;
TRUNCATE TABLE t;
```

Key types: `INTEGER`, `BIGINT`, `NUMERIC(p,s)`, `TEXT`, `BOOLEAN`, `DATE`, `TIMESTAMPTZ`, `UUID`, `JSONB`, `ARRAY` (`type[]`), `ENUM`.

46.3 Constraints

```
PRIMARY KEY | FOREIGN KEY ... REFERENCES t(col) [ON DELETE CASCADE|SET NULL|RESTRICT]
UNIQUE | NOT NULL | CHECK (expr) | DEFAULT value
```

46.4 INSERT

```
INSERT INTO t (col1, col2) VALUES (v1, v2) RETURNING col1;
```

46.5 SELECT

```
SELECT col1, col2 AS alias
FROM t
WHERE condition
GROUP BY col1
HAVING agg_condition
ORDER BY col1 [ASC|DESC]
LIMIT n OFFSET m;
```

46.6 WHERE Operators

```
=  <>  !=  >  <  >=  <=
AND  OR  NOT
BETWEEN a AND b
IN (...) | NOT IN (...)
LIKE 'pattern'  | ILIKE 'pattern'
IS NULL | IS NOT NULL
```

46.7 UPDATE / DELETE

```
UPDATE t SET col = value WHERE condition RETURNING *;
DELETE FROM t WHERE condition RETURNING *;
```

46.8 Joins

```
SELECT * FROM a JOIN b ON a.key = b.key;              -- INNER JOIN
SELECT * FROM a LEFT JOIN b ON a.key = b.key;          -- LEFT (OUTER)
JOIN
SELECT * FROM a RIGHT JOIN b ON a.key = b.key;         -- RIGHT
(OUTER) JOIN
SELECT * FROM a FULL OUTER JOIN b ON a.key = b.key;    -- FULL OUTER
JOIN
SELECT * FROM a CROSS JOIN b;                            -- CROSS JOIN
```

46.9 Subqueries

```
WHERE col = (SELECT ...)                  -- scalar
WHERE col IN (SELECT ...)                 -- multi-row
WHERE EXISTS (SELECT 1 FROM ... WHERE outer_col = inner_col)   --
correlated
```

46.10 CTEs

```
WITH cte_name AS (SELECT ...)
SELECT * FROM cte_name;

WITH RECURSIVE r AS (
    SELECT ... -- base case
    UNION ALL
    SELECT ... FROM t JOIN r ON ...  -- recursive case
)
SELECT * FROM r;
```

46.11 Window Functions

```
func() OVER (PARTITION BY col ORDER BY col [ROWS/RANGE BETWEEN ... AND
...])
```

ROW_NUMBER(),RANK(),DENSE_RANK(),NTILE(n),LAG(col),LEAD(col),
FIRST_VALUE(col),LAST_VALUE(col),SUM()/AVG()/COUNT() OVER (...).

46.12 Set Operations

```
q1 UNION q2       -- deduplicated
q1 UNION ALL q2   -- keeps duplicates
q1 INTERSECT q2
q1 EXCEPT q2
```

46.13 Views

```
CREATE VIEW v AS SELECT ...;
CREATE OR REPLACE VIEW v AS SELECT ...;
DROP VIEW v;
CREATE MATERIALIZED VIEW mv AS SELECT ...;
REFRESH MATERIALIZED VIEW mv;
```

46.14 Indexes

```
CREATE INDEX idx ON t(col);
CREATE INDEX idx ON t(col1, col2);                        -- composite
CREATE INDEX idx ON t(col) WHERE condition;                -- partial
CREATE INDEX idx ON t(LOWER(col));                         --
expression
CREATE UNIQUE INDEX idx ON t(col);
CREATE INDEX idx ON t USING GIN(jsonb_col);
EXPLAIN [ANALYZE] SELECT ...;
```

46.15 Transactions

```
BEGIN;
  ... statements ...
  SAVEPOINT sp1;
  ... more statements ...
  ROLLBACK TO sp1;   -- undo back to savepoint only
COMMIT;   -- or ROLLBACK; to undo everything since BEGIN
```

46.16 Functions, Procedures, Triggers

```
CREATE FUNCTION fn(arg TYPE) RETURNS TYPE AS $$
BEGIN
    RETURN value;
END;
$$ LANGUAGE plpgsql;

CREATE FUNCTION fn() RETURNS TABLE(col TYPE) AS $$
BEGIN RETURN QUERY SELECT ...; END;
$$ LANGUAGE plpgsql;
```

```
CREATE PROCEDURE proc(arg TYPE) AS $$
BEGIN ... COMMIT; END;
$$ LANGUAGE plpgsql;
CALL proc(value);

CREATE TRIGGER trg BEFORE/AFTER INSERT/UPDATE/DELETE ON t
FOR EACH ROW EXECUTE FUNCTION trigger_fn();
```

46.17 JSONB and Arrays

```
col -> 'key'       -- JSON value
col ->> 'key'       -- text value
col @> '{"k":"v"}'   -- containment
col ? 'key'           -- key/element exists
ARRAY['a','b']         -- array literal
'x' = ANY(array_col)    -- array membership
```

46.18 PostgreSQL-Specific Features

```
ILIKE                                  -- case-insensitive LIKE
RETURNING col                            -- return affected rows from
INSERT/UPDATE/DELETE
COUNT(*) FILTER (WHERE cond)               -- conditional aggregation
DISTINCT ON (col)                            -- first row per group
LATERAL (SELECT ...)                           -- per-row correlated
subquery in FROM
generate_series(a, b [, step])                   -- generate rows
STRING_AGG(col, ', ')                              -- concatenate
group values
INSERT ... ON CONFLICT (col) DO UPDATE SET ...       -- upsert
```

46.19 `psql` Meta-Commands

```
\l          list databases
\c dbname   connect to database
\dt         list tables
\d table    describe table
\dn         list schemas
\du         list roles
\q          quit
```

46.20 Python Integration

```
import psycopg2
conn = psycopg2.connect(host=..., dbname=..., user=..., password=...)
cur = conn.cursor()
cur.execute("SELECT * FROM t WHERE col = %s;", (value,))   # always
parameterized
rows = cur.fetchall()
conn.commit()  # or conn.rollback()

import pandas as pd
from sqlalchemy import create_engine
engine =
```

```
create_engine("postgresql+psycopg2://user:pass@host:port/dbname")
df = pd.read_sql("SELECT ...", engine)
df.to_sql("table", engine, if_exists="append", index=False)
```

46.21 Backup and Restore

```
pg_dump -h host -U user -d dbname -F c -f backup.dump
pg_restore -h host -U user -d dbname backup.dump
\copy (SELECT ...) TO 'file.csv' WITH (FORMAT csv, HEADER true)
\copy tablename FROM 'file.csv' WITH (FORMAT csv, HEADER true)
```

End of Course

Kajola Gbenga | Data Scientist & AI Engineer